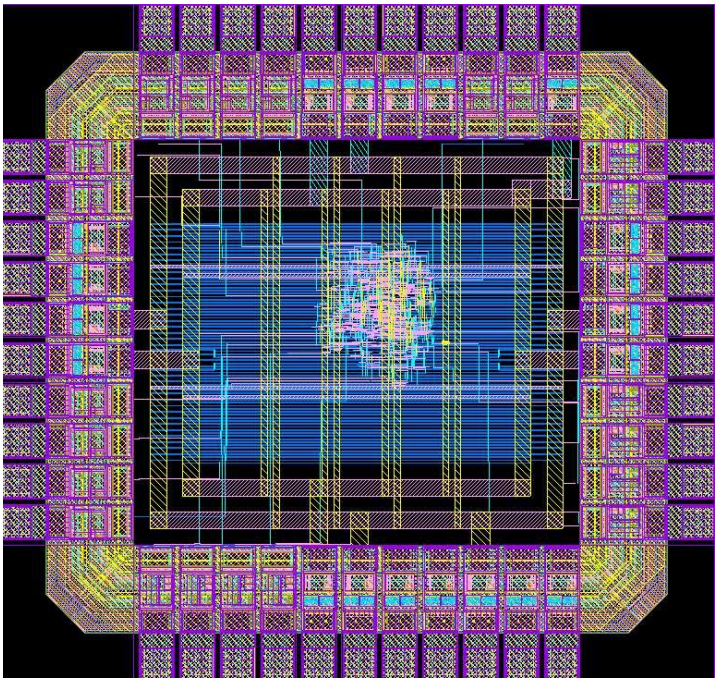
COMPLETE STUDY NOTES

Full-Flow ASIC Design RTL → GDSII

An 8-Bit Synchronous FIFO on SCL 180 nm Technology



Final `chip_top` GDSII layout — core, IO pad ring and seal ring

Technology Node	SCL 180 nm CMOS (TS18SL)
Metal Stack	4 Metal Layers (4LM): M1, M2, M3, TOP_M
Design	8-bit synchronous FIFO, depth 16, 100 MHz
Tool Chain	Cadence Genus · Innovus · Virtuoso · Siemens Calibre · HSPICE
Coverage	RTL → Verification → SDC → Synthesis → Floorplan → PDN → Placement → CTS → Routing → Sign-off → Tape-out

How This Note Is Organised

- Every main heading and sub-heading of the source material is retained; nothing is dropped.
- Each topic is written in **point form** — the concept first, then the **why**, then the **code**, then a **line-by-line explanation**.
- Every script, constraint file and verification deck is reproduced in full and then decoded command by command.
- All formulas, numeric targets and library cell names are collected in boxes so they can be revised quickly.
- Four kinds of boxes are used throughout:
  - **Blue — Why:** the physics or engineering reason behind a step.
  - **Green — Formula / Key numbers:** equations and target values.
  - **Red — If skipped:** the silicon failure that results if the step is omitted.
  - **Amber — Note / Warning:** tool-specific traps and best practice.
- All original tool screenshots are included at the point where the topic is explained. Additional hand-drawn diagrams have been created wherever a picture makes the idea clearer (clearly marked as explanatory diagrams).

> **The one-line story of the whole flow:**
> Verilog describes *behaviour* → synthesis converts behaviour into *gates* → place-and-route converts gates into *geometry* → sign-off proves the geometry is *manufacturable and electrically identical* to the gates → GDSII is the file the foundry turns into *masks*.

Contents

How This Note Is Organised **1**

1 Flow Overview and Abstract **6**
1.1 Abstract of the Design . . . 6
1.2 The Complete RTL-to-GDSII Flow at a Glance . . . 6
1.3 Design Specification Summary . . . 7

2 RTL Design — 8-Bit Synchronous FIFO **8**
2.1 Theoretical Background . . . 8
2.1.1 Synchronous Operation . . . 8
2.1.2 Circular Buffer Architecture . . . 8
2.1.3 The N+1 Pointer Method for Full/Empty Detection . . . 8
2.2 RTL Code — `fifo_8bit.v` . . . 9
2.2.1 Line-by-Line Explanation of the RTL . . . 10
2.3 Code Functionality Analysis . . . 11

3 Chip-Level Integration — Pad Ring Wrapper **12**
3.1 Theoretical Background . . . 12
3.1.1 Three Reasons the Pad Ring Exists . . . 12
3.2 IO Pad Library Cells (SCL 180 nm — `tsl18cio150`) . . . 13
3.3 RTL Code — `chip_top.v` . . . 13
3.3.1 Explanation of the Wrapper Code . . . 14

4 Functional Verification — RTL Simulation Testbench **15**
4.1 Verification Strategy . . . 15
4.1.1 Test Scenarios Covered . . . 15
4.1.2 Testbench Skeleton . . . 15
4.2 Simulation Results . . . 17

5 Synopsys Design Constraints (SDC) **18**
5.1 Overview of the SDC Constraint Set . . . 18
5.2 Constraint Section 1 — Clock Definition . . . 18
5.3 Constraint Section 2 — Clock Source Latency . . . 19
5.4 Constraint Section 3 — Clock Network Latency . . . 19
5.5 Constraint Section 4 — Clock Uncertainty (Jitter + Skew) . . . 19
5.6 Constraint Section 5 — Input Timing Constraints . . . 19
5.7 Constraint Section 6 — Output Timing Constraints . . . 20
5.8 Constraint Section 7 — Driving Cell Model . . . 20
5.9 Constraint Section 8 — Output Load . . . 20
5.10 Constraint Section 9 — Clock Transition . . . 20
5.11 Constraint Section 10 — Design Rule Constraints . . . 20

6 Logic Synthesis with Cadence Genus **22**
6.1 What Synthesis Does . . . 22
6.2 Stage 0 — Environment and Library Setup . . . 23
6.2.1 Key Parameters Explained . . . 23
6.3 Stage 1 — Design Elaboration . . . 24
6.4 Stage 2 — Generic Synthesis . . . 24
6.5 Stage 3 — Technology Mapping . . . 25

6.6 Stage 4 — Incremental Optimization . . . 25
6.7 Stage 5 — Netlist Export . . . 25
6.7.1 The Three Export Files . . . 26

7 Physical Design Setup **27**
7.1 Power Pad Insertion into the Synthesised Netlist . . . 27
7.1.1 Dual Voltage Domain Architecture . . . 27
7.1.2 Why Five Distributed Sets? . . . 27
7.2 Multi-Mode Multi-Corner (MMMC) Analysis Configuration . . . 28
7.2.1 Complete MMMC View Definition File . . . 28
7.2.2 Section-by-Section Explanation . . . 29
7.3 Cadence Innovus Global Configuration . . . 29
7.4 IO Pad Placement (`chip_top.io`) . . . 29
7.4.1 Pad Ring Architecture . . . 30
7.4.2 Pad Ring Assignment per Edge . . . 30
7.4.3 Complete `chip_top.io` File . . . 30
7.4.4 Syntax Decoded . . . 31

8 Floorplanning **32**
8.1 Overview and Objectives . . . 32
8.1.1 The Three Key Floorplan Parameters . . . 32
8.2 Floorplan Script (`floorplan.tcl`) . . . 33
8.2.1 Command-by-Command Explanation . . . 33

9 Power Planning — Constructing the Power Delivery Network **35**
9.1 Power Delivery Network (PDN) Architecture . . . 35
9.2 Power Plan Script (`power_plan.tcl`) . . . 35
9.2.1 Part 1 — Power Net Definitions and Global Connections . . . 35
9.2.2 Part 2 — Core Power Ring . . . 36
9.2.3 Part 3 — Power Stripes . . . 36
9.2.4 Part 4 — Special Routing (SRoute) . . . 37
9.2.5 Part 5 — PDN Verification and Save . . . 38

10 Physical Cell Placement — End Caps and IO Fillers **39**
10.1 End Cap Cells (Well Termination) . . . 39
10.2 IO Filler Cells (Pad Ring Continuity) . . . 39
10.2.1 IO Filler Cell Widths . . . 40
10.3 Script . . . 40

11 Standard Cell Placement **42**
11.1 Placement Theory . . . 42
11.2 Placement Configuration . . . 42
11.2.1 `setPlaceMode` Parameters . . . 42
11.2.2 `setOptMode` Parameters . . . 42
11.3 Placement Script (`placement.tcl`) . . . 43
11.3.1 What Each Analysis Command Checks . . . 43

12 Clock Tree Synthesis (CTS) **45**
12.1 Theory of Clock Distribution . . . 45
12.2 CTS Script (`cts.tcl`) . . . 46
12.2.1 Command-by-Command Explanation . . . 46
12.3 Library Cell Selection Rationale . . . 47

13 Signal Routing **49**
13.1 NanoRoute Overview . . . 49
13.2 Key Routing Challenges . . . 49
13.2.1 Antenna Effect . . . 49
13.2.2 Signal Integrity (Crosstalk) . . . 49
13.2.3 OCV and CPPR Analysis . . . 50
13.3 Routing Script (`routing.tcl`) . . . 50
13.3.1 Option-by-Option Explanation . . . 51

14 Innovus Sign-off and GDSII Export **53**
14.1 What Sign-off Means . . . 53
14.2 Standard Cell Filler Insertion . . . 53
14.3 GDSII Stream Out . . . 53
14.4 Sign-off Script (`signoff.tcl`) . . . 54
14.4.1 Explanation of Each Sign-off Step . . . 54

15 Tape-out Preparation — Cadence Virtuoso & Calibre **56**
15.1 Sign-off Philosophy . . . 56
15.2 GDSII Stream-In (Merging Layouts in Virtuoso) . . . 56
15.3 Chip Finishing Operations . . . 57
15.3.1 Power and Ground Labeling . . . 57
15.3.2 Silicon Number (Part ID) Insertion . . . 57
15.3.3 Seal Ring Customisation and Insertion . . . 58

16 Density Compliance — Dummy Fill Generation **59**
16.1 CMP Planarity and Dummy Fill . . . 59
16.2 Dummy Generation Script (`run_dummy.svrf`) . . . 59
16.2.1 Line-by-Line Explanation . . . 60
16.3 Final Database Merge . . . 60

17 Final Sign-off — DRC and LVS with Calibre **61**
17.1 Final Design Rule Check (DRC) . . . 61
17.1.1 Explanation . . . 61
17.2 Final Layout-Versus-Schematic (LVS) . . . 61
17.2.1 Explanation . . . 62

18 Post-Layout Simulation **64**
18.1 Parasitic Extraction (PEX) . . . 64
18.1.1 Explanation . . . 64
18.2 Post-Layout SPICE Simulation Testbench . . . 65
18.2.1 Line-by-Line Explanation . . . 65

19 Foundry Deliverables Checklist **67**
19.1 Why Each Deliverable Exists . . . 67

20 Chapter Summary **69**
20.1 Generality of the Flow . . . 69

21 Bibliography **71**
Textbooks & Monographs . . . 71
EDA Tool Documentation . . . 71
Process Technology & PDK . . . 71
IEEE & Industry Standards . . . 71
Additional References . . . 72

A Quick Revision Sheet — All Numbers and Formulas **73**
A.1 Every Numeric Target in One Place . . . 73
A.2 Every Formula in One Place . . . 74

B Master Command Reference **75**
B.1 Cadence Genus (Synthesis) . . . 75
B.2 Cadence Innovus (Physical Implementation) . . . 75
B.3 Siemens Calibre (Verification) . . . 76

C SCL 180 nm Library Cell Reference **77**
C.1 Library and Technology Files . . . 77

D Index of Key Terms **79**

PART 1

Flow Overview and Abstract

1.1 Abstract of the Design

- Complete, end-to-end **ASIC** (Application-Specific Integrated Circuit) design walkthrough on the **SCL 180 nm** process technology.
- Design under implementation: an **8-bit synchronous First-In First-Out (FIFO) buffer**, depth 16, running at **100 MHz**.
- Stages covered, in order:
  - RTL description of the FIFO in Verilog.
  - Functional verification with a directed testbench.
  - Logic synthesis with **Cadence Genus** against SCL standard-cell libraries.
  - Full physical implementation in **Cadence Innovus**: floorplanning, power-network construction, standard-cell placement, clock-tree synthesis, signal routing, GDSII stream-out.
  - Tape-out in **Cadence Virtuoso**: seal-ring customisation, dummy-fill generation, DRC/LVS sign-off with **Siemens (Mentor) Calibre**, and post-layout SPICE simulation.
- Every script, command, constraint file and verification deck is given in full so the flow is directly reproducible in a lab environment.

1.2 The Complete RTL-to-GDSII Flow at a Glance

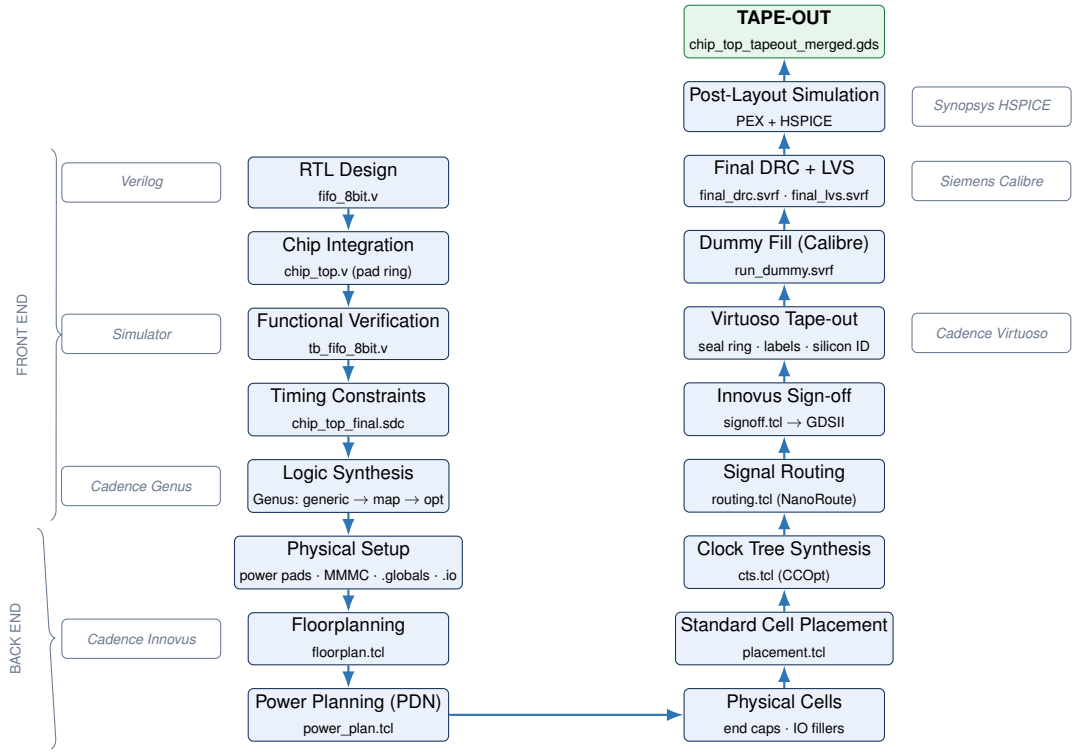


Figure 1.1: *(Explanatory diagram) The complete RTL-to-GDSII flow used in this design, with the tool responsible for each stage. Read down the left column, then up the right column.*

1.3 Design Specification Summary

Parameter	Value / Choice
Function	8-bit synchronous FIFO buffer
Data width	8 bits (`DATA_WIDTH = 8`)
Depth	16 words (`DEPTH = 16`)
Address width	4 bits (`ADDR_WIDTH = log2(DEPTH) = 4`)
Pointer width	5 bits (4 address bits + 1 phase/wrap bit)
Clock	100 MHz, 10 ns period, 50 % duty cycle
Reset	Active-low asynchronous (`rst_n`)
Storage	Flip-flop based register array, *not* a compiled SRAM macro
Technology	SCL 180 nm CMOS (TS18SL)
Std-cell library	`tsl18fs120_scl` (fs120)
IO library	`tsl18cio150` (cio150)
Metal stack	4 layers: M1, M2, M3, TOP_M
Core voltage	1.8 V (VDD_CORE / VSS_CORE)
IO voltage	3.3 V (VDDO_CORE / VSSO_CORE)
Die style	Pad-limited (area set by pad count, not logic)

Core numeric targets to memorise:

- Clock period $T = 10\,\text{ns} \Rightarrow f = 1/T = 100\,\text{MHz}$.
- Core utilisation $= 0.7$ (70 %); aspect ratio $= 1.0$ (square core).
- Core-to-die margin $= 150\,\mu\text{m}$ on all four sides.
- Power ring width $= 30\,\mu\text{m}$; vertical stripe pitch $= 112\,\mu\text{m} \Rightarrow$ worst distance to a stripe $= 112/2 = 56\,\mu\text{m}$.
- CTS: skew target < 0.3 ns; max clock transition < 0.5 ns.
- DRV limits: `max_transition` $= 1.50$ ns; `max_fanout` $= 16$.
- Metal density window for CMP: 25 %–75 % in every $100\,\mu\text{m} \times 100\,\mu\text{m}$ area.

PART 2

RTL Design — 8-Bit Synchronous FIFO

File: `fifo_8bit.v`

2.1 Theoretical Background

- The foundational building block of this design is a **Synchronous First-In First-Out (FIFO) memory buffer**.
- A FIFO decouples a **producer** (module that writes data) from a **consumer** (module that reads data) when the two work at **different rates** or in **bursts**, even if both share one clock.
- Three architectural decisions govern this implementation: synchronous operation, circular buffer addressing, and the N+1 pointer method.

2.1.1 Synchronous Operation

- Both write and read operations are triggered on the **rising edge of a single system clock** (`clk`).
- This deliberately avoids **Clock Domain Crossing (CDC)** synchronisers, which are mandatory whenever two asynchronous clock domains interact.
- Result: deterministic, simulation-friendly behaviour that is straightforward to verify and synthesise.

> **Why:** An asynchronous FIFO needs Gray-coded pointers and 2-flop synchronisers because a binary pointer crossing a clock boundary can be sampled mid-transition and produce a value that was never valid (metastability → multi-bit corruption). A single clock removes that entire failure class, so static timing analysis alone is sufficient to prove correctness.

2.1.2 Circular Buffer Architecture

- The physical memory is modelled as a **ring**.
- Write and read pointers advance sequentially; when either pointer reaches the end of the allocated depth it **wraps back to location zero**.
- Benefit: memory utilisation is maximised — previously read locations become immediately reusable for new writes.
- No data movement and no pointer-reset overhead are required (unlike a shift-register FIFO, where every word physically moves on every read).

2.1.3 The N+1 Pointer Method for Full/Empty Detection

- **The classic problem:** in both the *full* and the *empty* state, the **lower address bits** of the write pointer and the read pointer are **equal**. A 4-bit comparison alone cannot distinguish the two states.
- **The solution:** extend each pointer with one extra bit — the **phase bit** or **wrap bit**.
- For a FIFO of depth 16 ($2^4 = 16$, so 4 address bits are needed), the implementation uses **5-bit pointers**. The most significant bit, bit `[4]`, is the phase indicator.
- **Empty condition:** the entire 5-bit write pointer equals the entire 5-bit read pointer (`wr_ptr == rd_ptr`). Both pointers are in the same phase, pointing at the same location.
- **Full condition:** the lower 4 bits of both pointers are identical, but their MSBs differ. This means the write pointer has lapped the read pointer exactly once.

Formulas — FIFO sizing and flags

$$\begin{aligned}
\text{ADDR_WIDTH} &= \log_2(\text{DEPTH}) = \log_2 16 = 4 \\
\text{Pointer width} &= \text{ADDR_WIDTH} + 1 = 5 \text{ bits} \\
\texttt{empty} &= (\texttt{wr_ptr}[4{:}0] = \texttt{rd_ptr}[4{:}0]) \\
\texttt{full} &= \big(\texttt{wr_ptr}[4] \neq \texttt{rd_ptr}[4]\big) \wedge \big(\texttt{wr_ptr}[3{:}0] = \texttt{rd_ptr}[3{:}0]\big) \\
\text{Occupancy} &= \texttt{wr_ptr} - \texttt{rd_ptr} \pmod{2^5}
\end{aligned}$$

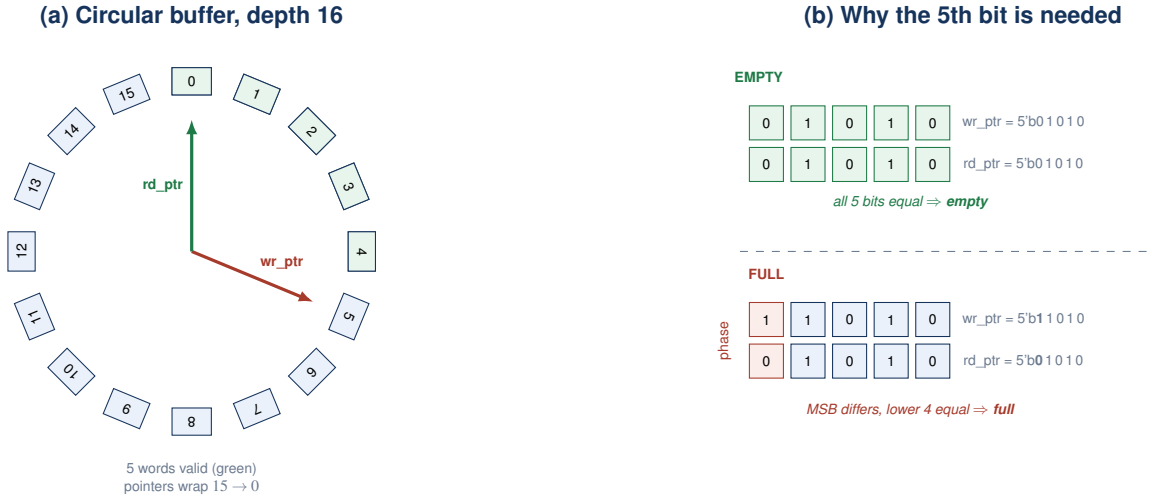


Figure 2.1: *(Explanatory diagram) (a) The memory is addressed as a ring; the pointers wrap from 15 back to 0. (b) With only 4 bits,* empty *and* full *look identical. The 5th phase bit separates them: equal means empty, differing MSB with equal address means full.*

2.2 RTL Code — `fifo_8bit.v`

```
//============================================================
// 8-Bit Synchronous FIFO
// Technology: SCL 180nm | Data Width: 8-bit | Depth: 16
//============================================================
`timescale 1ns/1ps

module fifo_8bit #(
    parameter DATA_WIDTH = 8,
    parameter DEPTH      = 16,
    parameter ADDR_WIDTH = 4        // log2(DEPTH)
)(
    input  wire                  clk,
    input  wire                  rst_n,
    input  wire                  wr_en,
    input  wire                  rd_en,
    input  wire [DATA_WIDTH-1:0] din,
    output reg  [DATA_WIDTH-1:0] dout,
    output wire                  full,
    output wire                  empty
);

    //------------------------------------------------------------
    // Memory Array: 16 words x 8 bits
    //------------------------------------------------------------
    reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];

    //------------------------------------------------------------
    // Pointers: 5-bit (N+1 method)
    //------------------------------------------------------------
    reg [ADDR_WIDTH:0] wr_ptr;
    reg [ADDR_WIDTH:0] rd_ptr;

    //------------------------------------------------------------
    // Status Flags (Combinational)
    //------------------------------------------------------------
    assign empty = (wr_ptr == rd_ptr);
```

```
    assign full  = (wr_ptr[ADDR_WIDTH]     != rd_ptr[ADDR_WIDTH]) &&
                   (wr_ptr[ADDR_WIDTH-1:0] == rd_ptr[ADDR_WIDTH-1:0]);

    //------------------------------------------------------------
    // Write Logic
    //------------------------------------------------------------
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            wr_ptr <= 0;
        else if (wr_en && !full) begin
            mem[wr_ptr[ADDR_WIDTH-1:0]] <= din;
            wr_ptr <= wr_ptr + 1;
        end
    end

    //------------------------------------------------------------
    // Read Logic
    //------------------------------------------------------------
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            rd_ptr <= 0;
            dout   <= 0;
        end else if (rd_en && !empty) begin
            dout   <= mem[rd_ptr[ADDR_WIDTH-1:0]];
            rd_ptr <= rd_ptr + 1;
        end
    end

endmodule
```

2.2.1 Line-by-Line Explanation of the RTL

- `` `timescale 1ns/1ps `` — simulation time unit is 1 ns and resolution is 1 ps. All delays in the testbench are interpreted in these units.
- `module fifo_8bit #( ... )` — the `#()` block declares **parameters**, making the module reusable: changing `DEPTH` and `ADDR_WIDTH` re-targets the same code to any FIFO size.
- `input wire clk, rst_n, wr_en, rd_en` — control inputs. `rst_n` is **active-low**: logic 0 means "reset is asserted".
- `output reg [DATA_WIDTH-1:0] dout` — declared `reg` because it is assigned inside an `always` block; it therefore infers a **register** (flip-flops), not wires.
- `reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];` — a 2-D array: 16 entries of 8 bits each. Synthesis maps this to **128 flip-flops** plus the address decode logic, because no SRAM macro is instantiated.
- `reg [ADDR_WIDTH:0] wr_ptr;` — note the range is `[4:0]`, i.e. **5 bits**, not `[3:0]`. This is exactly the N+1 pointer.
- `assign empty = (wr_ptr == rd_ptr);` — continuous assignment, so `empty` is **combinational** and updates immediately when either pointer changes.
- `assign full = (MSBs differ) && (lower bits equal);` — the two-part full test described above.
- `always @(posedge clk or negedge rst_n)` — the sensitivity list contains `negedge rst_n`, which infers an **asynchronous reset** flip-flop (reset takes effect without waiting for a clock edge).
- `if (!rst_n) wr_ptr <= 0;` — reset branch must come first; it has priority over all other logic.
- `else if (wr_en && !full)` — the `!full` guard is the **overflow protection**: writes are silently ignored when the FIFO is full, so stored data is never corrupted.
- `mem[wr_ptr[ADDR_WIDTH-1:0]] <= din;` — only the **lower 4 bits** index the memory. The phase bit is used purely for flag computation and must never reach the address decoder.
- `wr_ptr <= wr_ptr + 1;` — a 5-bit increment that naturally rolls over 31 → 0, giving the ring behaviour for free.
- `else if (rd_en && !empty)` — the `!empty` guard is the **underflow protection**.
- `<=` (non-blocking assignment) is used throughout sequential blocks so that all flip-flops sample their *old* values simultaneously, matching real hardware behaviour and avoiding simulation race conditions.

2.3 Code Functionality Analysis

Signal / Line	Direction	Description
`mem[0:DEPTH-1]`	Internal	Register-based storage array (16 words × 8 bits). Uses flip-flop resources rather than SRAM.
`wr_ptr[4:0]`	Internal	5-bit write pointer. Bits `[3:0]` index into `mem[]`; bit `[4]` is the phase/wrap indicator.
`rd_ptr[4:0]`	Internal	5-bit read pointer. Same structure as `wr_ptr`.
`empty`	Output	Asserted high when `wr_ptr == rd_ptr` (combinational).
`full`	Output	Asserted high when MSBs differ but lower bits match (combinational).
`din[7:0]`	Input	8-bit data input; written to `mem[wr_ptr]` on the rising clock edge if `wr_en && !full`.
`dout[7:0]`	Output	Registered output. Loaded from `mem[rd_ptr]` when `rd_en && !empty`.

Design Note — Registered vs. Combinational Read
The read data output `dout` is **registered** (clocked). This trades **one clock cycle of read latency** for **improved timing closure** and eliminates any combinational glitch on the data output bus — an important consideration when driving downstream synchronous logic.

Why: A combinational read path would be `rd_ptr → 16:1 address mux → chip output pad`, all inside one clock period. At 180 nm that mux alone can consume several nanoseconds of the 10 ns budget, and any pointer-bit skew makes the mux output glitch. Registering `dout` breaks the path at the flip-flop, so the only remaining requirement is $T_{cq} + T_{pad} + T_{output_delay} \leq T_{clk}$.

PART 3

Chip-Level Integration — Pad Ring Wrapper

File: `chip_top.v`

3.1 Theoretical Background

- In real silicon the core logic **cannot interface directly with the outside world**.
- Internal logic signals operate at low voltage and are too fragile to withstand the electrostatic and electrical stresses of a printed-circuit-board environment.
- The solution is an **IO Pad Ring** — a dedicated set of interface cells that surround the core and mediate all electrical transactions between the silicon die and the external world.

3.1.1 Three Reasons the Pad Ring Exists

1. **ESD Protection.** Pad cells contain Electro-Static Discharge (ESD) clamp structures. Without them, a charged human hand or manufacturing tool can inject **thousands of volts** through a bonding wire and permanently destroy thin-oxide transistors.
2. **Voltage Interfacing and Drive Strength.** Output pads buffer internal **1.8 V** core signals to drive the capacitive load of PCB traces and off-chip receivers. Input pads provide a Schmitt-trigger or level-sensitive input stage to clean up slow, noisy external signals before presenting them to core logic.
3. **Pad-Limited Design.** Because the FIFO core logic is extremely compact, the physical die area is dominated by the **number of bonding pads** rather than by logic density. This is called a **Pad-Limited** design.

Why: At 180 nm the gate oxide is roughly 4 nm thick. Its breakdown field is about 10 MV/cm, so the oxide fails at around 4 V applied directly. A human-body-model ESD event delivers 2 kV through ∼1.5 kΩ, i.e. an ampere-class current spike. The ESD clamp must therefore shunt that current to the supply rails in nanoseconds, before the voltage at the gate can rise past breakdown.

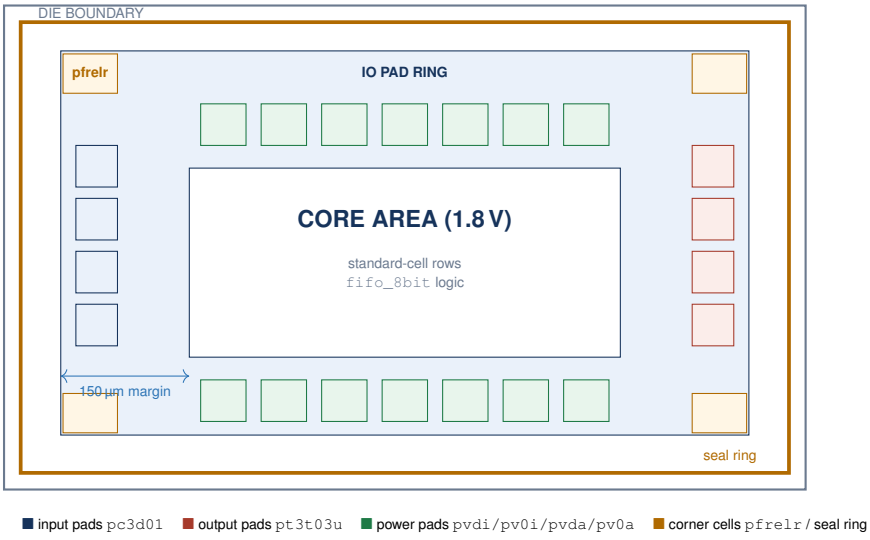


Figure 3.1: *(Explanatory diagram) Anatomy of a pad-limited die: the core sits in the middle, the IO pad ring surrounds it, corner cells turn the power rails through 90°, and the seal ring protects the die edge during dicing.*

3.2 IO Pad Library Cells (SCL 180 nm — `tsl18cio150`)

Cell Name	Type	Application in `chip_top`
`pc3d01`	Input Pad	CLK, RST_N, WR_EN, RD_EN, DIN[7:0]. Buffers external signal into core.
`pt3t03u`	Output Pad (3-state)	DOUT[7:0], FULL, EMPTY. Drives core signal to chip pin. `OEN = 0` permanently enables.
`pvdi`	Power Pad (VDD Core)	Supplies 1.8 V core voltage.
`pv0i`	Power Pad (VSS Core)	Provides 0 V core ground.
`pvda`	Power Pad (VDDO IO)	Supplies 3.3 V IO ring voltage.
`pv0a`	Power Pad (VSSO IO)	Provides 0 V IO ring ground.
`pfrelr`	Corner Cell	Turns the power ring 90 degrees at each chip corner.

3.3 RTL Code — `chip_top.v`

```
//============================================================
// CHIP TOP WITH PAD WRAPPER
// SCL 180nm IO Library (tsl18cio150)
//============================================================
`timescale 1ns/1ps

module chip_top (
    // External PAD pins (inout to allow bidirectional pad cells)
    inout       PAD_CLK,
    inout       PAD_RSTN,
    inout       PAD_WR_EN,
    inout       PAD_RD_EN,
    inout [7:0] PAD_DIN,
    inout [7:0] PAD_DOUT,
    inout       PAD_FULL,
    inout       PAD_EMPTY
);

    //------------------------------------------------------------
    // Internal core signals (post-pad, pre-core)
    //------------------------------------------------------------
    wire       clk, rst_n, wr_en, rd_en;
    wire [7:0] din, dout;
    wire       full, empty;

    //------------------------------------------------------------
    // INPUT PADS (pc3d01)
    //------------------------------------------------------------
    pc3d01 u_pad_clk (.PAD(PAD_CLK),   .CIN(clk));
    pc3d01 u_pad_rst (.PAD(PAD_RSTN),  .CIN(rst_n));
    pc3d01 u_pad_wr  (.PAD(PAD_WR_EN), .CIN(wr_en));
    pc3d01 u_pad_rd  (.PAD(PAD_RD_EN), .CIN(rd_en));

    genvar i;
    generate
        for (i = 0; i < 8; i = i+1) begin : DIN_PADS
            pc3d01 u_pad_din (.PAD(PAD_DIN[i]), .CIN(din[i]));
        end
    endgenerate

    //------------------------------------------------------------
    // OUTPUT PADS (pt3t03u) -- OEN = 1'b0 permanently enables
    //------------------------------------------------------------
    pt3t03u u_pad_full  (.PAD(PAD_FULL),  .I(full),  .OEN(1'b0));
    pt3t03u u_pad_empty (.PAD(PAD_EMPTY), .I(empty), .OEN(1'b0));

    generate
        for (i = 0; i < 8; i = i+1) begin : DOUT_PADS
            pt3t03u u_pad_dout (.PAD(PAD_DOUT[i]), .I(dout[i]), .OEN(1'b0));
        end
    endgenerate

    //------------------------------------------------------------
```

```
    // FIFO CORE INSTANTIATION
    //------------------------------------------------------------
    fifo_8bit u_fifo (
        .clk   (clk),   .rst_n (rst_n),
        .wr_en (wr_en), .rd_en (rd_en),
        .din   (din),   .dout  (dout),
        .full  (full),  .empty (empty)
    );

endmodule
```

3.3.1 Explanation of the Wrapper Code

- `inout PAD_CLK, ...` — every external port is declared `inout`, not `input`/`output`. Pad cells are physically **bidirectional** structures (they contain both a driver and a receiver plus ESD diodes), so the port direction must permit either role.
- **Two naming layers** exist deliberately:
  - `PAD_*` names = the **outside** of the pad, i.e. the bond pad / package pin.
  - plain names (`clk`, `din`) = the **inside** of the pad, i.e. core-side nets.
- `wire clk, rst_n, wr_en, rd_en;` — the post-pad internal nets. Nothing in the core ever connects directly to a `PAD_*` net.
- `pc3d01 u_pad_clk (.PAD(PAD_CLK), .CIN(clk));` — an input pad: `PAD` is the external side, `CIN` ("core in") is the buffered signal delivered to the core.
- `genvar i; generate for (i = 0; i < 8; i = i+1) begin : DIN_PADS` — a **generate loop** instantiates eight identical input pads for the 8-bit data bus. The loop label `DIN_PADS` becomes part of the hierarchical instance name (`DIN_PADS[3].u_pad_din`), which is exactly the name later used in the `chip_top.io` pad-placement file.
- `pt3t03u u_pad_full (.PAD(...), .I(full), .OEN(1'b0));` — a 3-state output pad. `.I` is the core data driving outward; `.OEN` is the active-low output enable.
- `.OEN(1'b0)` — hard-tying the enable low makes the pad a **permanently driving output**. A real bidirectional port would instead route a control signal here.
- `fifo_8bit u_fifo (...)` — the core is instantiated using **named port connection** (`.port(net)`), which is mandatory practice because positional connection silently mis-wires a design when the port list changes.

If skipped: If the pad ring is omitted and the core is bonded directly: (i) the first ESD event destroys the chip; (ii) 1.8 V core drivers cannot swing a PCB trace, so the board sees an invalid logic level; (iii) Innovus has no legal place to attach power bondwires, so the PDN cannot be built.

Note — the pad instance names matter later. The exact instance names created here (`u_pad_clk`, `DIN_PADS[i].u_pad_din`, `DOUT_PADS[i].u_pad_dout`, ...) are referenced verbatim in the `chip_top.io` file in Part 7. A typo there produces an "instance not found" error during floorplanning.

PART 4

Functional Verification — RTL Simulation Testbench

File: `tb_fifo_8bit.v`

4.1 Verification Strategy

- Before any synthesis is attempted, the RTL must be **proved functionally correct** through simulation.
- The testbench implements a **Direct Test** strategy, targeting:
  - the **Happy Path** — normal write-then-read operation;
  - **boundary conditions** — `empty` flag at start-up, `full` flag after 16 consecutive writes.
- Stimulus is applied to `chip_top` (the **pad-ring wrapper**), not to the bare FIFO, so that **IO pad buffering is also exercised**.

4.1.1 Test Scenarios Covered

- **Clock Generation:** a 100 MHz system clock (period = 10 ns) is generated by the statement `always #5 clk = ~clk;`
- **Reset Sequence:** active-low reset (`rst_n = 0`) is held for **20 ns**, ensuring all write and read pointers initialise to zero.
- **Burst Write Sequence:** a loop writes values 0 through 9 into the FIFO. During this sequence the `empty` flag transitions low and the write pointer advances.
- **Idle State:** a short pause after the burst write confirms data stability.
- **Burst Read Sequence:** a loop reads ten locations back. `dout` is monitored to confirm values arrive in correct FIFO order $(0, 1, 2, \ldots, 9)$, proving First-In First-Out behaviour.
- **Monitor Block:** the `$monitor` system task prints pointer values and data to the console transcript on every signal change, providing a complete textual audit log.

> **Clock arithmetic.** $T_{clk} = 10\,\text{ns} \Rightarrow$ half period $= 5\,\text{ns} \Rightarrow$ `#5` toggle. Frequency $f = 1/T_{clk} = 1/(10 \times 10^{-9}) = 100\,\text{MHz}$. Reset of 20 ns = 2 full clock cycles, which comfortably covers the asynchronous-reset recovery/removal window.

4.1.2 Testbench Skeleton

Reconstructed from the stimulus described above — this is the structure the listed scenarios imply.

```
`timescale 1ns/1ps

module tb_fifo_8bit;

    reg        clk  = 0;
    reg        rst_n;
    reg        wr_en, rd_en;
    reg  [7:0] din;
    wire [7:0] dout;
    wire       full, empty;
    integer    i;

    // ---- Device Under Test : the PAD-RING WRAPPER, not the bare core ----
    chip_top DUT (
        .PAD_CLK (clk),   .PAD_RSTN (rst_n),
        .PAD_WR_EN(wr_en),.PAD_RD_EN(rd_en),
        .PAD_DIN (din),   .PAD_DOUT (dout),
        .PAD_FULL(full),  .PAD_EMPTY(empty)
    );

    // ---- 1. Clock generation : 100 MHz ----
    always #5 clk = ~clk;
```

```
    // ---- 2. Monitor : textual audit log on every change ----
    initial $monitor("T=%0t rst_n=%b wr_en=%b rd_en=%b din=%0d dout=%0d full=%b empty=%b",
                     $time, rst_n, wr_en, rd_en, din, dout, full, empty);

    initial begin
        // ---- 3. Reset sequence : hold 20 ns ----
        rst_n = 0;  wr_en = 0;  rd_en = 0;  din = 8'd0;
        #20  rst_n = 1;

        // ---- 4. Burst write : values 0 .. 9 ----
        @(posedge clk);
        for (i = 0; i < 10; i = i + 1) begin
            @(negedge clk);
            wr_en = 1;  din = i[7:0];
        end
        @(negedge clk);  wr_en = 0;

        // ---- 5. Idle : confirm data stability ----
        #50;

        // ---- 6. Burst read : expect 0,1,2, ... ,9 in order ----
        for (i = 0; i < 10; i = i + 1) begin
            @(negedge clk);
            rd_en = 1;
        end
        @(negedge clk);  rd_en = 0;

        #50  $finish;
    end

endmodule
```

- `always #5 clk = ~clk;` — free-running 10 ns clock. It is placed outside the main `initial` block so it never stops.
- Stimulus is driven on the **negative edge** (`@(negedge clk)`) while the design samples on the **positive edge**. This guarantees half a clock period of setup and removes simulation races.
- `$monitor` prints only when a listed signal *changes*, so the log is compact yet complete.
- Driving `chip_top` rather than `fifo_8bit` means the simulation also validates that the pad instantiations and port polarities in the wrapper are correct.

What to look for in the waveform (checklist):

- `empty` is high immediately after reset and falls on the *first* write.
- `wr_ptr` increments by exactly one per asserted `wr_en` cycle, and never while `full` is high.
- `dout` lags `rd_en` by **one clock cycle** — this is the registered-read latency, not a bug.
- The read-back sequence is $0, 1, 2, \ldots, 9$: same order as written $\Rightarrow$ FIFO behaviour proven.
- `full` asserts only when exactly 16 words are resident, confirming the N+1 pointer logic.

4.2 Simulation Results

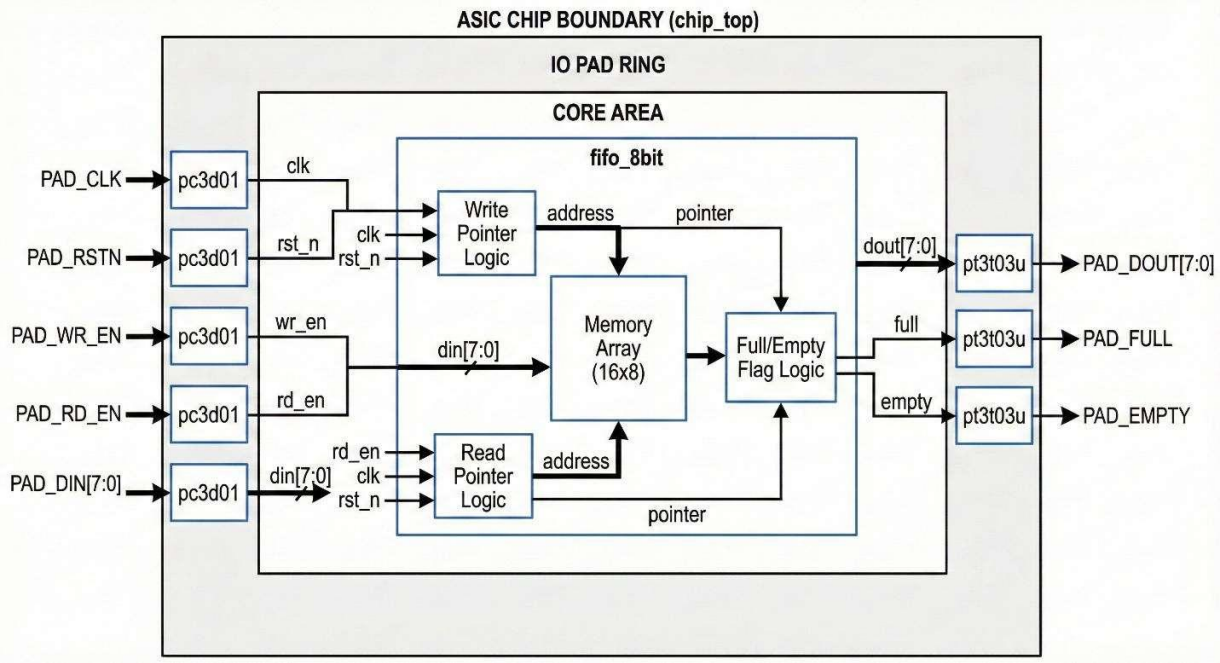


Figure 4.1: *Chip-level architecture of `chip_top`: input pads (`pc3d01`) feed the core; the core contains write-pointer logic, the 16×8 memory array, read-pointer logic and the full/empty flag logic; output pads (`pt3t03u`) drive `PAD_DOUT[7:0]`, `PAD_FULL` and `PAD_EMPTY`. This is the block the testbench stimulates.*

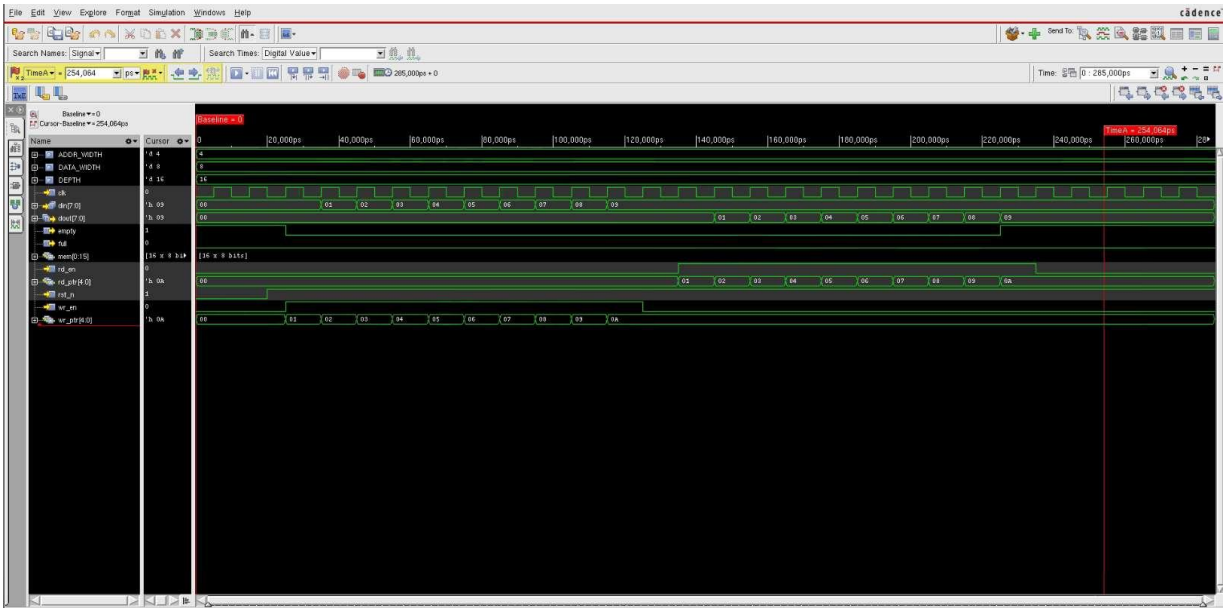


Figure 4.2: ***Figure 5.2 (original)*** *— RTL simulation waveform: 100 MHz system clock, the burst write followed by the burst read, and the resulting status-flag behaviour. The pointer buses and the `full`/`empty` flags are the signals to inspect when debugging.*

PART 5

Synopsys Design Constraints (SDC)

File: `chip_top_final.sdc`

5.1 Overview of the SDC Constraint Set

- The SDC file is the **authoritative specification** that communicates design intent to both the logic synthesis tool (Cadence Genus) and the physical design tool (Cadence Innovus).
- It defines three things:
  - the **operating frequency**;
  - the **electrical environment at the chip boundary**;
  - the **design-rule limits** that must be satisfied across the timing graph.
- In an industrial flow the SDC becomes the **contract** between the design team and the sign-off tools.

If skipped: Any path **not covered** by an SDC constraint is treated as an **unconstrained don't-care path**. The tool will not optimise it and will not report it, which can lead to **silently incorrect timing closure** — the design reports "zero violations" and still fails on silicon.

The two equations every SDC constraint ultimately serves

$$\textbf{Setup (max delay):} \quad T_{cq} + T_{comb} + T_{setup} + T_{uncert} \le T_{clk} + T_{skew}$$

$$\textbf{Hold (min delay):} \quad T_{cq} + T_{comb} \ge T_{hold} + T_{skew} + T_{uncert,hold}$$

$$\text{Slack}_{setup} = T_{required} - T_{arrival} \quad (\text{must be} \ge 0)$$

Setup failures are fixed by *slowing the data down or speeding the logic up*; hold failures are fixed by *adding delay*. Setup scales with frequency; **hold does not** — a hold violation is broken at *every* frequency.

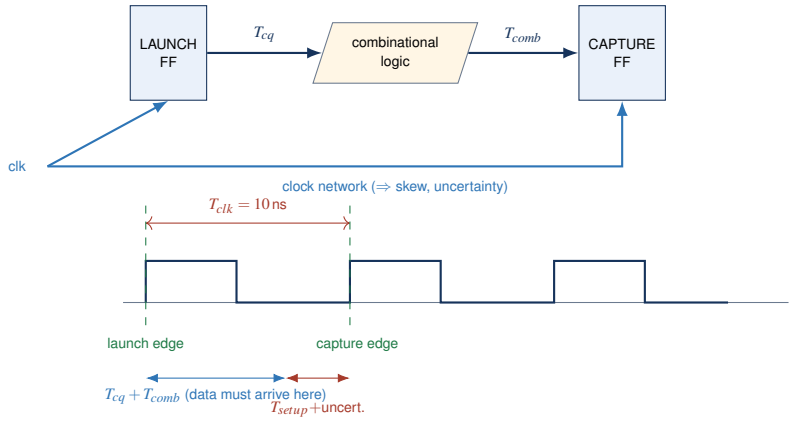


Figure 5.1: *(Explanatory diagram) The single timing budget that every SDC line in this chapter either consumes or protects. The 10 ns period must contain the clock-to-Q delay, the combinational delay, the setup time of the capture flip-flop and all clock uncertainty.*

5.2 Constraint Section 1 — Clock Definition

- A 100 MHz clock (10 ns period, 50 % duty cycle) is defined at the **external pad boundary** (`PAD_CLK`), which is the true entry point of the clock into the chip.
- Defining the clock **at the pad** rather than at an internal port is **mandatory** when a pad wrapper is used, because it accurately models the latency introduced by the input pad cell.

```
# 100 MHz clock defined at the PAD boundary
create_clock -name CLK \
             -period 10 \
             -waveform {0 5} \
             [get_ports PAD_CLK]
```

- `-period 10` — the clock period in the SDC time unit (ns here).
- `-waveform {0 5}` — rising edge at 0 ns, falling edge at 5 ns ⇒ 50 % duty cycle.
- `[get_ports PAD_CLK]` — anchors the clock to the physical port object, not to a net name.

5.3 Constraint Section 2 — Clock Source Latency

- Clock **source** latency models the delay between the **real clock source** (an on-board oscillator or PLL) and the chip pad.
- Early/Late values bound the **arrival-time uncertainty** at the source.

```
set_clock_latency -source -early 0.30 [get_clocks CLK]
set_clock_latency -source -late  0.50 [get_clocks CLK]
```

Why: Source latency is *outside* the chip, so CTS cannot change it. Declaring an early/late window (0.30–0.50 ns) forces the timing engine to assume the worst combination of launch and capture source delay, which is the only safe assumption for a board-level clock.

5.4 Constraint Section 3 — Clock Network Latency

- Before Clock Tree Synthesis, the physical clock buffer tree **does not yet exist**.
- The tool uses an **estimated insertion delay** so that meaningful pre-CTS timing analysis and placement optimisation can still be performed.

```
# Pre-CTS estimated insertion delay
set_clock_latency -early 0.80 [get_clocks CLK]
set_clock_latency -late  1.20 [get_clocks CLK]
```

This is a **placeholder**. After CTS the real tree exists, the design is analysed in **propagated-clock** mode, and these estimates are ignored. If the estimate is wildly optimistic, the pre-CTS design will look clean and then explode into violations after CTS.

5.5 Constraint Section 4 — Clock Uncertainty (Jitter + Skew)

- Clock uncertainty **derates the timing budget** to account for clock jitter and skew that cannot be modelled deterministically.

```
set_clock_uncertainty -setup 0.20 [get_clocks CLK]
set_clock_uncertainty -hold  0.10 [get_clocks CLK]
```

- Setup uncertainty (0.20 ns) is **subtracted** from the available period: the effective period becomes $10 - 0.20 = 9.80$ ns.
- Hold uncertainty (0.10 ns) is **added** to the required hold time, making hold checks stricter.
- Separate values are used because jitter hurts setup and skew hurts hold through different mechanisms.

5.6 Constraint Section 5 — Input Timing Constraints

- Input delay constraints model the **logic delay in the upstream device** that drives the chip.
- They tell synthesis **how much of the clock period is already consumed** before the signal arrives at the chip input pads.

```
set_input_delay -min 0.50 -clock CLK [get_ports {PAD_DIN[*] PAD_WR_EN PAD_RD_EN}]
set_input_delay -max 2.50 -clock CLK [get_ports {PAD_DIN[*] PAD_WR_EN PAD_RD_EN}]
```

Input budget: $T_{internal,max} \leq T_{clk} - T_{input_delay,max} - T_{setup} - T_{uncert} = 10 - 2.50 - T_{setup} - 0.20 \approx 7.1$ ns of internal logic allowed.
`-min 0.50` protects the **hold** check: data must not arrive sooner than 0.5 ns after the edge.

5.7 Constraint Section 6 — Output Timing Constraints

- Output delay constraints model the **setup requirement of the downstream receiving device**.

```
set_output_delay -min 0.40 -clock CLK [get_ports {PAD_DOUT[*] PAD_FULL PAD_EMPTY}]
set_output_delay -max 2.00 -clock CLK [get_ports {PAD_DOUT[*] PAD_FULL PAD_EMPTY}]
```

Output budget: $T_{cq} + T_{internal} + T_{pad} \leq T_{clk} - T_{output_delay,max} = 10 - 2.00 = 8.0$ ns.

5.8 Constraint Section 7 — Driving Cell Model

- The driving-cell constraint tells the synthesiser **what cell drives the chip's input pads from outside**, which establishes the **transition time** at the pad input.

```
set_driving_cell -lib_cell inv0d2 [get_ports {PAD_DIN[*] PAD_WR_EN PAD_RD_EN PAD_RSTN}]
```

Why: Without a driving cell the tool assumes an **ideal zero-transition** source. Real slew is finite, and cell delay in a Liberty library is a 2-D function of *input slew* and *output load*. An ideal slew makes every input path look faster than it really is, producing optimistic and therefore unsafe timing.

5.9 Constraint Section 8 — Output Load

- Output load models the **capacitance of the PCB trace** plus the **input capacitance of downstream devices** seen by each output pad.

```
set_load 5.0 [get_ports {PAD_DOUT[*] PAD_FULL PAD_EMPTY}]
```

- 5.0 is in the library capacitance unit (typically pF). A 5 pF board load is orders of magnitude larger than an on-chip gate load (∼fF), which is precisely why output pads need large drivers.

5.10 Constraint Section 9 — Clock Transition

- Specifies the **slew rate of the incoming clock** at the chip pad, which directly impacts **jitter** and **hold-time margins**.

```
set_clock_transition 0.10 [get_clocks CLK]
```

Why: A slow clock edge spends longer in the transistor's high-gain region, where supply and substrate noise translate directly into timing displacement — i.e. it converts voltage noise into jitter. 0.10 ns is an aggressively clean edge, which is why CTS later enforces `target_max_trans 0.5` ns internally.

5.11 Constraint Section 10 — Design Rule Constraints

- These set **global limits on transition time and fanout**, tuned for the SCL 180 nm process.
- Violations are treated as **Design Rule Violations (DRVs)** that must be repaired during synthesis and optimisation.

```
set_max_transition 1.50 [current_design]
set_max_fanout     16   [current_design]
# END OF SDC
```

- `max_transition 1.50 ns` — no net may have a slew slower than 1.5 ns. Slow edges cause **crowbar current** (both PMOS and NMOS conducting simultaneously), extra dynamic power, and delay-model inaccuracy.
- `max_fanout 16` — no cell may drive more than 16 loads. Exceeding this forces the optimiser to **insert buffers** and build a fanout tree.

SDC quick-revision table

Command	Value	Purpose
`create_clock`	10 ns / {0 5}	Define 100 MHz clock at `PAD_CLK`
`set_clock_latency -source`	0.30 / 0.50	Off-chip oscillator-to-pad delay window
`set_clock_latency`	0.80 / 1.20	Pre-CTS estimated insertion delay
`set_clock_uncertainty`	0.20 / 0.10	Jitter + skew margin (setup / hold)
`set_input_delay`	0.50 / 2.50	Upstream device delay before the pad
`set_output_delay`	0.40 / 2.00	Downstream device setup requirement
`set_driving_cell`	`inv0d2`	External driver ⇒ realistic input slew
`set_load`	5.0	PCB + receiver capacitance on each output
`set_clock_transition`	0.10	Clock edge slew at the pad
`set_max_transition`	1.50	Global slew DRV limit
`set_max_fanout`	16	Global fanout DRV limit

PART 6

Logic Synthesis with Cadence Genus

6.1 What Synthesis Does

- Logic synthesis translates the RTL description into a **structural netlist** composed of SCL 180 nm **standard cells**.
- The synthesiser uses the **SDC constraints as a target** and selects cells from the **Liberty timing library** to meet timing, area and power objectives.
- The Genus flow used here is divided into **six stages**, each corresponding to one script section.

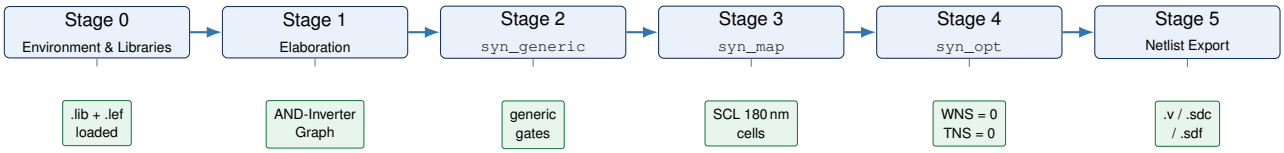


Figure 6.1: *(Explanatory diagram) The six Genus stages and the deliverable each one produces. Technology independence is lost at Stage 3 — before that the netlist is pure Boolean logic.*

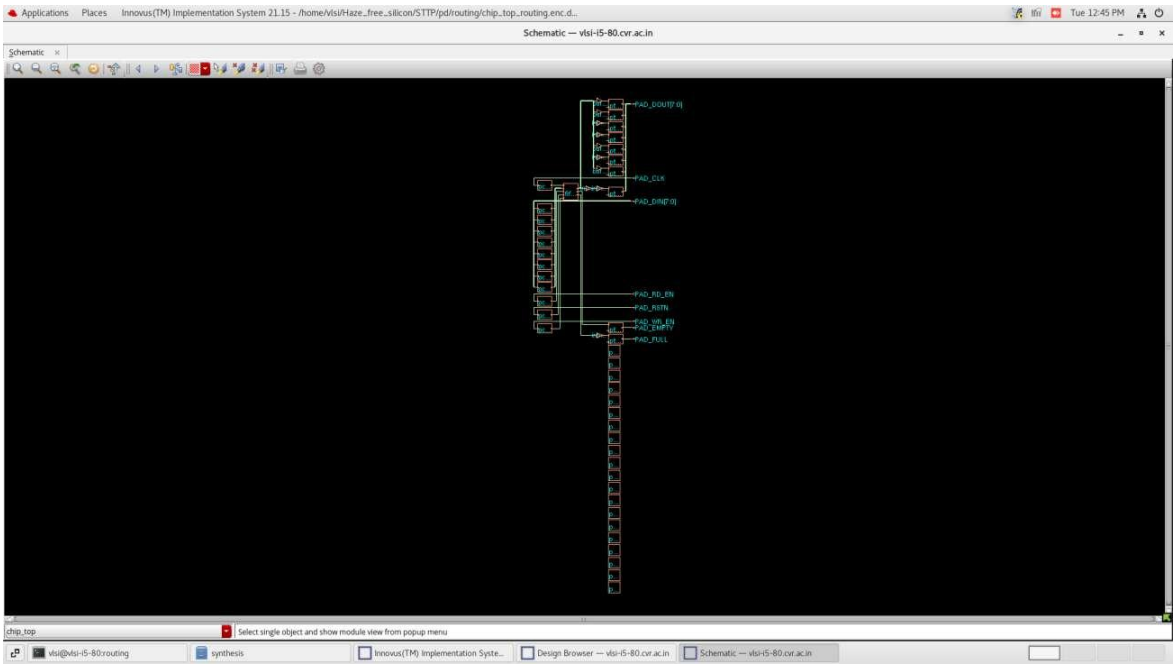


Figure 6.2: ***Figure 5.3 (original)*** *— Cadence Genus post-synthesis schematic:* `chip_top` *top-level view showing the IO pad ring and the core instance.*

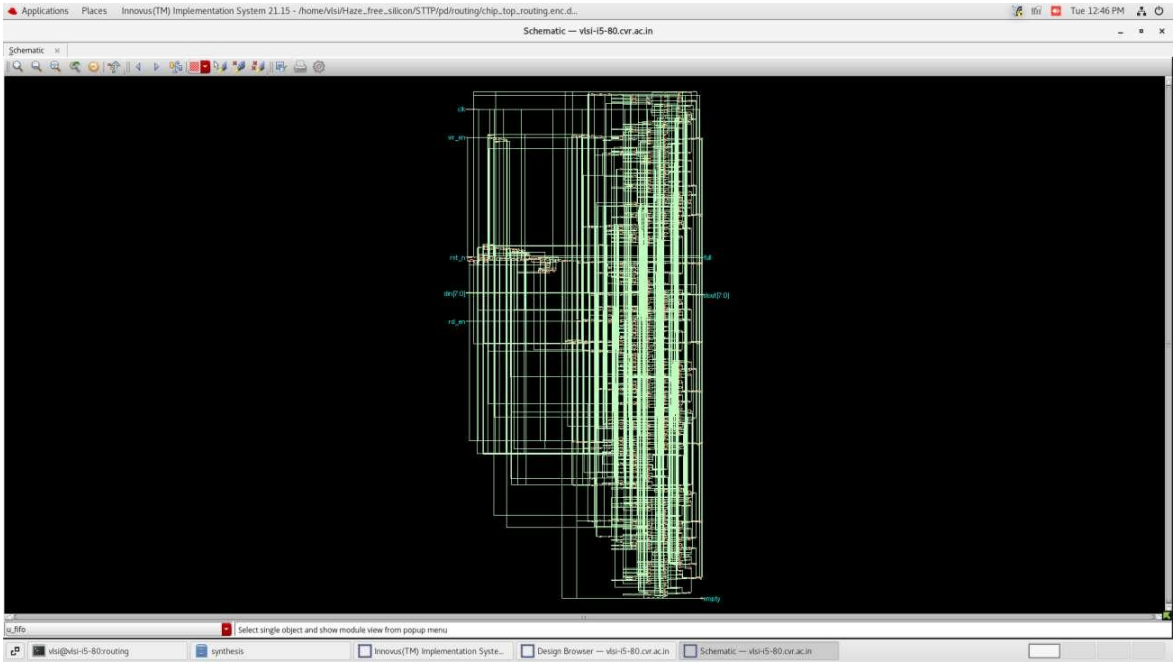


Figure 6.3: ***Figure 5.4 (original)*** *— Cadence Genus post-synthesis schematic: internal FIFO datapath logic with pointer arithmetic, after technology mapping to SCL 180 nm cells.*

6.2 Stage 0 — Environment and Library Setup

- Prepares the synthesis workspace by loading the **logical (Liberty)** and **physical (LEF)** libraries that define the SCL 180 nm technology.

```
#==========================================================================
# 1. ENVIRONMENT & LIBRARY SETUP
#==========================================================================

# Increase tool memory limit -- prevents elaboration failure on large arrays
set_db hdl_max_memory_address_range 2000000

# Standard Cell Liberty Library (Slow-Slow corner for max timing)
set std_cell_lib \
  "/home/install/scl_pdk_v2/stdlib/fs120/liberty/lib_flow_ss/tsl18fs120_scl_ss.lib"

# IO Pad Liberty Library
set iopad_lib \
  "/home/install/scl_pdk_v2/iolib/cio150/synopsys/2002.05/models/tsl18cio150_min.lib"

# Load both libraries
set_db library [list $std_cell_lib $iopad_lib]

# Physical LEF Libraries (enables Physical Layout Estimation - PLE)
set_db lef_library [list \
  /home/install/scl_pdk_v2/stdlib/fs120/tech_data/lef/tsl180l4.lef \
  /home/install/scl_pdk_v2/stdlib/fs120/lef/tsl18fs120_scl.lef   \
  /home/install/scl_pdk_v2/iolib/cio150/cds/lef/tsl18cio150_4lm.lef \
  /home/install/scl_pdk_v2/iolib/cio150/cds/lef/tsl180l4.lef     \
]

# Maximum synthesis effort (area, power, and timing optimization)
set_db syn_generic_effort high
set_db syn_map_effort     high
set_db syn_opt_effort     high
```

6.2.1 Key Parameters Explained

- `set_db hdl_max_memory_address_range 2000000` — without this setting, Genus may **crash during elaboration** when it encounters large register arrays. The default limit is insufficient for a 16-element FIFO array run alongside the pad-ring wrapper.
- **Liberty files (.lib)** — contain the **timing** (setup, hold, propagation delay), **power** (dynamic and leakage) and **area** characterisation of every standard cell and IO pad cell at a specific PVT corner. The **SS (Slow-Slow)**

library represents the **worst-case corner for setup** timing analysis.

- **LEF files** — Library Exchange Format files define the **physical geometry** (height, width, pin locations, obstruction layers) of each cell. Loading LEF enables **Physical Layout Estimation (PLE)**, where the synthesiser uses statistical wire-length models based on cell size and aspect ratio, producing timing estimates **significantly more accurate** than pure logic-level analysis.
- `syn_*_effort high` — instructs Genus to use the most computationally intensive optimisation algorithms. Runtime increases, but **Quality of Results (QoR)** in timing margin, power and area improves.

> **Why:** The .lib file is the *electrical* model and the .lef file is the *geometric* model of the same cell. Timing needs both: delay $= f(\text{input slew}, \text{output load})$, and output load $= C_{pin} + C_{wire}$, where C_{wire} can only be estimated once the tool knows how big the cells are and how far apart they will sit. That is exactly what PLE provides.

6.3 Stage 1 — Design Elaboration

```
#================================================================================
# 2. DESIGN ELABORATION
#================================================================================

# Load both RTL source files
read_hdl -language sv {
    /home/vlsi/Haze_free_silicon/STTP/synthesis/fifo_8bit.v
    /home/vlsi/Haze_free_silicon/STTP/synthesis/chip_top.v
}

# Set chip_top as the absolute top-level module
elaborate chip_top

# Sanity check -- detect undriven inputs, multiply-driven nets, etc.
check_design -all > ./reports/check_design_elaborated.rpt
```

- The `elaborate` command **parses the Verilog text** and converts it into an internal Boolean network representation — an **AND-Inverter Graph (AIG)**.
- Setting `chip_top` as top level instructs the tool to **trace connectivity through the pad-ring wrapper into the core FIFO logic**, ensuring IO pad models are included in all timing paths.
- `check_design -all` catches **undriven inputs**, **multiply-driven nets**, **unresolved module references** and **unconnected ports** before any optimisation effort is wasted.

6.4 Stage 2 — Generic Synthesis

```
#================================================================================
# 3. STAGE 1: GENERIC SYNTHESIS (RTL to Technology-Independent Gates)
#================================================================================

# Load timing constraints
read_sdc /home/vlsi/Haze_free_silicon/STTP/synthesis/fifo_input_sdc.sdc

# First pass: optimize Boolean logic, technology-independent
syn_generic

report_area   > ./reports/generic_area.rpt
report_timing > ./reports/generic_timing.rpt
```

- `syn_generic` performs **Boolean optimisation and retiming without consulting the SCL standard-cell library**.
- It simplifies the logic cone by applying:
  - algebraic transformations;
  - technology-independent NAND/NOR tree restructuring;
  - register merging.
- The SDC file is loaded **at this stage** so the tool already knows which paths are timing-critical before it starts restructuring.

6.5 Stage 3 — Technology Mapping

```
#================================================================================
# 4. STAGE 2: MAPPED SYNTHESIS (Technology Mapping to SCL 180 nm)
#================================================================================

syn_map

report_area   > ./reports/mapped_area.rpt
report_timing > ./reports/mapped_timing.rpt
```

- During technology mapping, each **generic gate** in the unmapped netlist is replaced with the **optimal physical cell** from the SCL 180 nm Liberty library.
- Example: a 2-input AND function may be implemented as an `and2d1` (weak drive) or an `and2d4` (strong drive) cell, depending on the **fanout** and **timing slack** on that net.
- The mapper uses a **dynamic programming algorithm** to minimise cost (area, power or slack) subject to the timing constraints.

Cell-name convention in SCL libraries: `<function><inputs>d<drive>`. So `and2d4` = 2-input AND, drive strength 4×. Higher drive = larger transistors = faster into big loads, but more area and more input capacitance presented to the *previous* stage.

6.6 Stage 4 — Incremental Optimization

```
#================================================================================
# 5. STAGE 3: OPTIMIZED SYNTHESIS (Incremental QoR Improvement)
#================================================================================

syn_opt

# Final Quality of Results reports
report_area   > ./reports/final_area.rpt
report_power  > ./reports/final_power.rpt
report_timing > ./reports/final_timing.rpt
report_qor    > ./reports/final_qor.rpt
```

- `syn_opt` performs a **post-mapping optimisation pass** that applies:
  - **gate sizing** — upsizing slow cells on critical paths;
  - **buffer insertion** — splitting long nets;
  - **logic restructuring** — removing remaining timing violations and **Design Rule Violations (DRVs)**.
- The **QoR report is the primary acceptance criterion**: it must show **zero Worst Negative Slack (WNS)** and **zero Total Negative Slack (TNS)** before the netlist is exported.

WNS vs TNS
$\text{WNS} = \min_{\text{all paths}}(\text{slack})$ — the single worst path; tells you *how badly* the design fails.
$\text{TNS} = \sum_{\text{paths with slack}<0} \text{slack}$ — tells you *how many* paths fail. A design with WNS = −0.05 ns and TNS = −50 ns has a systemic problem, not a single bad path.

6.7 Stage 5 — Netlist Export

```
#================================================================================
# 6. FINAL EXPORTS FOR INNOVUS & LEC
#================================================================================

# Structural Verilog netlist (SCL cells, ready for place-and-route)
write_hdl > ./outputs/chip_top_final.v

# Updated SDC (annotated with synthesis-level timing updates)
write_sdc > ./outputs/chip_top_final.sdc

# Standard Delay Format (gate + wire delays for post-synth simulation)
write_sdf -timescale ns/ps > ./outputs/chip_top_final.sdf
```

6.7.1 The Three Export Files

- **`chip_top_final.v`** — a flat structural Verilog netlist containing **only instantiated SCL 180 nm standard-cell primitives**. This is the input to Cadence Innovus for physical implementation. The file explicitly includes all **power pad instances** (`pvdi`, `pv0i`, `pvda`, `pv0a`) because these must appear in the netlist for the physical tool to know where to route power straps.
- **`chip_top_final.sdc`** — the timing constraints, **updated** to reflect changes made by the synthesis optimiser (e.g. adjusted clock latency estimates, computed false paths).
- **`chip_top_final.sdf`** — annotated **delay values** for every cell and net in the synthesised netlist. This file is back-annotated into a gate-level Verilog simulation to verify the design still functions correctly when real propagation delays are accounted for — a process known as **Post-Synthesis Gate-Level Simulation**.

> **Critical Requirement — Timescale in SDF**
> The `-timescale ns/ps` flag in `write_sdf` is **mandatory**. Without it, simulation tools may misinterpret the delay units, causing the post-synthesis simulation to see either **zero delay** (optimistic) or **absurdly large delay** (pessimistic) — both of which produce incorrect results.

PART 7

Physical Design Setup

7.1 Power Pad Insertion into the Synthesised Netlist

File: `chip_top_final.v` (post-synthesis, modified for physical implementation)

- A synthesised netlist produced by Genus contains **only logical connectivity**: standard cells and **signal** IO pads.
- Before importing the netlist into Innovus, **power pads must be explicitly instantiated in Verilog** so the physical tool can model the power delivery network correctly.

7.1.1 Dual Voltage Domain Architecture

- **Core Voltage Domain (VDD_CORE / VSS_CORE):** supplies **1.8 V** to all SCL standard cells in the FIFO logic core.
- **IO Voltage Domain (VDDO_CORE / VSSO_CORE):** supplies **3.3 V** to the pad-ring driver circuits, which require a higher voltage to swing the output signal to PCB-compatible levels.
- **Five complete power pad sets** are instantiated — one per die edge (left, top, right, bottom) plus one partial set — creating a **balanced power ring**.

```
//------------------------------------------------------------
// POWER SUPPLY NETWORK DEFINITIONS
//------------------------------------------------------------
wire VDD_CORE;     // Core Logic Power   (1.8 V)
wire VSS_CORE;     // Core Logic Ground  (0 V)
wire VDDO_CORE;    // I/O Ring Power     (3.3 V)
wire VSSO_CORE;    // I/O Ring Ground    (0 V)

//------------------------------------------------------------
// POWER PADS -- 5 complete sets for balanced ring
//------------------------------------------------------------
// Set 1 (Left Edge)
pvdi pvdi_VDD_CORE_1 (.VDD(VDD_CORE),.VSS(VSS_CORE),.VDDO(VDDO_CORE),.VSSO(VSSO_CORE));
pv0i pv0i_VSS_CORE_1 (.VDD(VDD_CORE),.VSS(VSS_CORE),.VDDO(VDDO_CORE),.VSSO(VSSO_CORE));
pvda pvda_VDDO_CORE_1(.VDD(VDD_CORE),.VSS(VSS_CORE),.VDDO(VDDO_CORE),.VSSO(VSSO_CORE));
pv0a pv0a_VSSO_CORE_1(.VDD(VDD_CORE),.VSS(VSS_CORE),.VDDO(VDDO_CORE),.VSSO(VSSO_CORE));

// Sets 2-5 : identical instantiations on Top, Right, Bottom, and partial edges
// (pvdi / pv0i / pvda / pv0a instances with suffixes _2 through _5)
```

7.1.2 Why Five Distributed Sets?

- **Electromigration Prevention:** multiple parallel current paths reduce the current density in any individual power wire below the electromigration threshold.
- **IR Drop Reduction:** distributed power entry points reduce the resistive voltage drop across the power mesh, ensuring all standard cells see adequate supply voltage.
- **SSO Noise Reduction:** placing power pads adjacent to signal pads (**Signal-Power Interleaving**) minimises the **inductive loop area** of the transient current path, reducing **Simultaneous Switching Output (SSO) noise** and ground bounce.

The three equations behind power-pad count

IR drop: $\Delta V = I \cdot R \quad \Rightarrow \quad N$ parallel pads $\Rightarrow R_{eff} = R/N \Rightarrow \Delta V$ falls as $1/N$

Ground bounce: $V_{gb} = L\dfrac{di}{dt} \quad \Rightarrow \quad$ more, shorter return paths $\Rightarrow$ lower L

Electromigration (Black): $\mathrm{MTTF} = A\,J^{-n}\exp\left(\dfrac{E_a}{kT}\right),\ n \approx 2$

Because MTTF depends on J^{-2}, **halving the current density gives roughly four times the lifetime**. That single

fact is why wide rings and many power pads are not optional.

7.2 Multi-Mode Multi-Corner (MMMC) Analysis Configuration

File: `chip_top.view`

- Modern VLSI designs must close timing across a range of **Process, Voltage and Temperature (PVT) corners**.
- MMMC analysis allows the place-and-route tool to **simultaneously** analyse **setup timing at the worst-case corner** and **hold timing at the best-case corner**.
- This prevents the classic error of **fixing setup violations and inadvertently introducing hold violations**.

Analysis Type	Corner	Temperature	Voltage	Purpose
Setup (Max Delay)	Slow-Slow (SS)	125 °C (Worst-RC) / −40 °C	1.68 V	Data must arrive before the clock edge — slowest cells, highest R and C
Hold (Min Delay)	Fast-Fast (FF)	−40 °C (Best-RC) / 125 °C	1.98 V	Data must not race ahead — fastest cells, lowest R and C

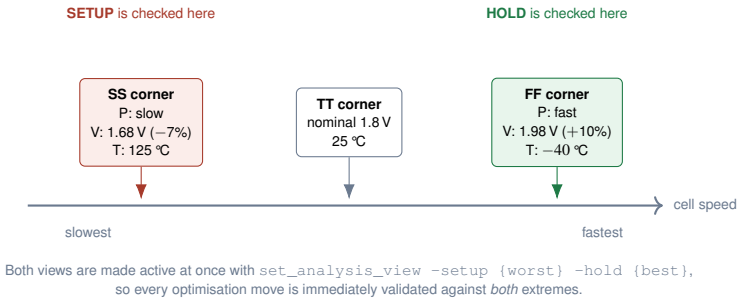


Figure 7.1: *(Explanatory diagram) Why two corners are analysed simultaneously. Fixing a setup violation by making a path faster can push the same path into a hold violation at the fast corner — MMMC catches that in the same optimisation step.*

7.2.1 Complete MMMC View Definition File

```
# Version:1.0 MMMC View Definition File
# Do Not Remove Above Line

# SECTION 1: RC CORNER DEFINITIONS
create_rc_corner -name RC_Corner_worst \
  -cap_table {.../SCL_TLUPLUS_4M1L_TYP.capTbl} \
  -T {125} -preRoute_res {1.0} -preRoute_cap {1.0} \
  -preRoute_clkres {0.0} -preRoute_clkcap {0.0} \
  -postRoute_res {1.0} -postRoute_cap {1.0} -postRoute_xcap {1.0} \
  -postRoute_clkres {0.0} -postRoute_clkcap {0.0}

create_rc_corner -name RC_Corner_best \
  -cap_table {.../SCL_TLUPLUS_4M1L_TYP.capTbl} \
  -T {-40} -preRoute_res {1.0} -preRoute_cap {1.0} \
  -preRoute_clkres {0.0} -preRoute_clkcap {0.0} \
  -postRoute_res {1.0} -postRoute_cap {1.0} -postRoute_xcap {1.0} \
  -postRoute_clkres {0.0} -postRoute_clkcap {0.0}

# SECTION 2: OPERATING CONDITIONS (PVT)
create_op_cond -name ss -library_file {tsl18fs120_scl_ss.lib} -P {1} -V {1.68} -T {-40}
create_op_cond -name ff -library_file {tsl18fs120_scl_ff.lib} -P {1} -V {1.98} -T {125}

# SECTION 3: LIBRARY SETS
create_library_set -name max_timing -timing {tsl18fs120_scl_ss.lib tsl18cio150_max.lib}
create_library_set -name min_timing -timing {tsl18fs120_scl_ff.lib tsl18cio150_min.lib}

# SECTION 4: CONSTRAINT MODES & DELAY CORNERS
create_constraint_mode -name sdc_constraints \
  -sdc_files {/home/vlsi/Haze_free_silicon/STTP/pd/inputs/chip_top_final.sdc}
```

```
create_delay_corner -name min_delay -library_set {min_timing} -rc_corner {RC_Corner_best}
create_delay_corner -name max_delay -library_set {max_timing} -rc_corner {RC_Corner_worst}

# SECTION 5: ANALYSIS VIEWS
create_analysis_view -name best  -constraint_mode {sdc_constraints} -delay_corner {min_delay}
create_analysis_view -name worst -constraint_mode {sdc_constraints} -delay_corner {max_delay}

# Activate both views simultaneously in Innovus
set_analysis_view -setup {worst} -hold {best}
```

7.2.2 Section-by-Section Explanation

- **Section 1 — RC corners.** `-cap_table` points to the **TLU+ capacitance table**, which converts routed metal geometry into resistance and capacitance. `-T` sets the temperature at which the *interconnect* (not the transistors) is evaluated: metal resistivity *rises* with temperature, so 125 °C is the **worst-RC** condition and −40 °C is the **best-RC** condition. The `preRoute_*` and `postRoute_*` scale factors (all 1.0 here) let a team derate R and C without editing the cap table.
- **Section 2 — operating conditions.** Defines the **transistor** PVT point: `-P` process, `-V` voltage, `-T` temperature. SS uses the −7 % supply (1.68 V); FF uses the +10 % supply (1.98 V).
- **Section 3 — library sets.** Bundles the standard-cell `.lib` with the matching IO `.lib`; `max_timing` is used for setup and `min_timing` for hold.
- **Section 4 — constraint mode + delay corners.** A **delay corner** = library set + RC corner. A **constraint mode** = one SDC file (in a multi-mode chip there would be several: functional mode, scan/test mode, low-power mode).
- **Section 5 — analysis views.** An **analysis view** = constraint mode + delay corner. `set_analysis_view -setup {worst} -hold {best}` activates both at once so every optimisation is validated against both extremes.

> **Temperature inversion caution.** At 180 nm, cells are slowest at *high* temperature (mobility degradation dominates). At deeply scaled nodes the opposite can happen below ∼0.8 V — cells become slowest at *low* temperature. That is why both temperature extremes appear in the corner table and why the corner set must always be taken from the PDK, never assumed.

7.3 Cadence Innovus Global Configuration

File: `chip_top.globals`

- The `.globals` file is the **configuration manifest** that initialises the Innovus workspace.
- It **links the physical and logical design data** before any implementation command is issued.
- Key settings:
  - `init_verilog` — the Verilog netlist;
  - `init_lef_file` — the LEF physical libraries;
  - `init_pwr_bus_name` and `init_gnd_bus_name` — explicit power net name definitions.
- Innovus **requires explicit power definitions** because the tool uses these names to identify which nets drive the power rails of every standard cell and IO pad.

7.4 IO Pad Placement (`chip_top.io`)

- IO pad placement is specified in a **technology-independent text file** that maps each pad instance to a physical location (left, right, top or bottom edge).
- Unlike standard cells, **IO pads cannot be automatically placed**: their positions are constrained by **PCB connector layout**, **signal integrity** and **power distribution** requirements.

7.4.1 Pad Ring Architecture

- **Corner Cells (`pfrelr`):** non-functional metal cells placed at each of the four corners of the IO ring. They are **structural connectors** that turn the continuous power and ground metal rails by 90 degrees, preventing an electrical open in the ring at each corner.
- **Pad Spacing (`space = 10 µm`):** a 10 µm gap is enforced between adjacent pad cells. This spacing prevents **mechanical interference during wire bonding**, in which a gold capillary presses wire loops onto the pad metal surface.
- **Signal-Power Interleaving:** power pad sets are distributed on every edge. This reduces the **loop inductance** of the transient supply-current path, minimising SSO noise.

Location	Instance Name	Cell Type	Orientation	Purpose
Top-Left	`cornercell1`	`pfrelr`	R180	Power Rail Turn (90°)
Top-Right	`cornercell2`	`pfrelr`	R90	Power Rail Turn (90°)
Bottom-Right	`cornercell3`	`pfrelr`	R0	Power Rail Turn (90°)
Bottom-Left	`cornercell4`	`pfrelr`	R270	Power Rail Turn (90°)

7.4.2 Pad Ring Assignment per Edge

- **Left Edge:** `DIN[3:0]` (input data), Power Set 1 (`pvdi`, `pv0i`, `pvda`, `pv0a`), `CLK`, `RST_N`.
- **Top Edge:** `DIN[7:4]` (input data), Power Set 2, `WR_EN`, `RD_EN`, and partial Power Set 5 (`pvdi VDD_CORE` only).
- **Right Edge:** `DOUT[3:0]` (output data), Power Set 3, `FULL` status flag, `EMPTY` status flag.
- **Bottom Edge:** `DOUT[7:4]` (output data), Power Set 4, and the remainder of Power Set 5 (`pv0i`, `pvda`, `pv0a`).

7.4.3 Complete `chip_top.io` File

```
(globals
    version    = 3
    io_order   = clockwise
    space      = 10
    total_edge = 18
)
(iopad
(topleft  (inst name="cornercell1" cell=pfrelr offset=0 orientation=R180 place_status=fixed))
(left
    (inst name="DIN_PADS[0].u_pad_din" cell=pc3d01 place_status=fixed)
    (inst name="DIN_PADS[1].u_pad_din" cell=pc3d01 place_status=fixed)
    (inst name="DIN_PADS[2].u_pad_din" cell=pc3d01 place_status=fixed)
    (inst name="DIN_PADS[3].u_pad_din" cell=pc3d01 place_status=fixed)
    (inst name="pvdi_VDD_CORE_1"   cell=pvdi   place_status=fixed)
    (inst name="pv0i_VSS_CORE_1"   cell=pv0i   place_status=fixed)
    (inst name="pvda_VDDO_CORE_1"  cell=pvda   place_status=fixed)
    (inst name="pv0a_VSSO_CORE_1"  cell=pv0a   place_status=fixed)
    (inst name="u_pad_clk"         cell=pc3d01 place_status=fixed)
    (inst name="u_pad_rst"         cell=pc3d01 place_status=fixed)
)
(topright (inst name="cornercell2" cell=pfrelr offset=0 orientation=R90 place_status=fixed))
(top
    (inst name="DIN_PADS[4].u_pad_din" cell=pc3d01 place_status=fixed)
    (inst name="DIN_PADS[5].u_pad_din" cell=pc3d01 place_status=fixed)
    (inst name="DIN_PADS[6].u_pad_din" cell=pc3d01 place_status=fixed)
    (inst name="DIN_PADS[7].u_pad_din" cell=pc3d01 place_status=fixed)
    (inst name="pvdi_VDD_CORE_2"   cell=pvdi   place_status=fixed)
    (inst name="pv0i_VSS_CORE_2"   cell=pv0i   place_status=fixed)
    (inst name="pvda_VDDO_CORE_2"  cell=pvda   place_status=fixed)
    (inst name="pv0a_VSSO_CORE_2"  cell=pv0a   place_status=fixed)
    (inst name="u_pad_wr"          cell=pc3d01 place_status=fixed)
    (inst name="u_pad_rd"          cell=pc3d01 place_status=fixed)
    (inst name="pvdi_VDD_CORE_5"   cell=pvdi   place_status=fixed)
)
(bottomright (inst name="cornercell3" cell=pfrelr offset=0 orientation=R0 place_status=fixed))
(right
    (inst name="DOUT_PADS[0].u_pad_dout" cell=pt3t03u place_status=fixed)
    (inst name="DOUT_PADS[1].u_pad_dout" cell=pt3t03u place_status=fixed)
    (inst name="DOUT_PADS[2].u_pad_dout" cell=pt3t03u place_status=fixed)
```

```
    (inst name="DOUT_PADS[3].u_pad_dout" cell=pt3t03u place_status=fixed)
    (inst name="pvdi_VDD_CORE_3"   cell=pvdi   place_status=fixed)
    (inst name="pv0i_VSS_CORE_3"   cell=pv0i   place_status=fixed)
    (inst name="pvda_VDDO_CORE_3"  cell=pvda   place_status=fixed)
    (inst name="pv0a_VSSO_CORE_3"  cell=pv0a   place_status=fixed)
    (inst name="u_pad_full"        cell=pt3t03u place_status=fixed)
    (inst name="u_pad_empty"       cell=pt3t03u place_status=fixed)
)
(bottomleft (inst name="cornercell4" cell=pfrelr offset=0 orientation=R270 place_status=fixed))
(bottom
    (inst name="DOUT_PADS[4].u_pad_dout" cell=pt3t03u place_status=fixed)
    (inst name="DOUT_PADS[5].u_pad_dout" cell=pt3t03u place_status=fixed)
    (inst name="DOUT_PADS[6].u_pad_dout" cell=pt3t03u place_status=fixed)
    (inst name="DOUT_PADS[7].u_pad_dout" cell=pt3t03u place_status=fixed)
    (inst name="pvdi_VDD_CORE_4"   cell=pvdi   place_status=fixed)
    (inst name="pv0i_VSS_CORE_4"   cell=pv0i   place_status=fixed)
    (inst name="pvda_VDDO_CORE_4"  cell=pvda   place_status=fixed)
    (inst name="pv0a_VSSO_CORE_4"  cell=pv0a   place_status=fixed)
    (inst name="pv0i_VSS_CORE_5"   cell=pv0i   place_status=fixed)
    (inst name="pvda_VDDO_CORE_5"  cell=pvda   place_status=fixed)
    (inst name="pv0a_VSSO_CORE_5"  cell=pv0a   place_status=fixed)
)
)
```

7.4.4 Syntax Decoded

- `version = 3` — file format version expected by Innovus.
- `io_order = clockwise` — pads listed within each edge block are placed in **clockwise** order around the die.
- `space = 10` — 10 µm gap between adjacent pads (the gap later filled by `pfeed` IO fillers).
- `total_edge = 18` — maximum number of pad slots per edge.
- `place_status=fixed` — the pad must **not** be moved by any later optimisation step.
- `orientation=R0/R90/R180/R270` — rotation in degrees; each corner cell needs a different rotation so its internal metal rails line up with the two edges it joins.
- Instance names must **exactly match** the names created by the generate loops in `chip_top.v` (e.g. `DIN_PADS[3].u_pad_din`).

> **If skipped:** Omit a corner cell and the power ring is **electrically open** at that corner — current cannot circulate, half the pad ring loses its supply, and the ESD network is disabled on that side. The error usually shows up only at `verifyConnectivity`, long after the floorplan has been committed.

PART 8

Floorplanning

File: `floorplan.tcl`

8.1 Overview and Objectives

- Floorplanning is the **first major step in physical implementation**.
- It translates the **logical netlist into physical dimensions**, establishing:
  - the **die size**;
  - the **core area** available for standard-cell placement;
  - the **routing infrastructure** (metal tracks) needed by the routing engine.

8.1.1 The Three Key Floorplan Parameters

- **Aspect Ratio (1.0):** a ratio of 1.0 means the core is **square** (height equals width). A square core is preferable for **short, balanced power stripes** and **minimal clock-tree insertion delay**.
- **Core Utilization (0.7):** 70 % of the core area is pre-allocated for standard cells. The remaining **30 %** is reserved as **white space** to accommodate:
  - routing channels;
  - clock buffer insertion during CTS;
  - timing-closure buffers inserted by the optimiser.
- **Core Margins (150 µm):** a 150 µm boundary is maintained on all four sides between the core edge and the die edge. This space is occupied by the **IO pad ring** (which sits between the core and the die boundary) and provides room for **power ring and power-stripe routing**.

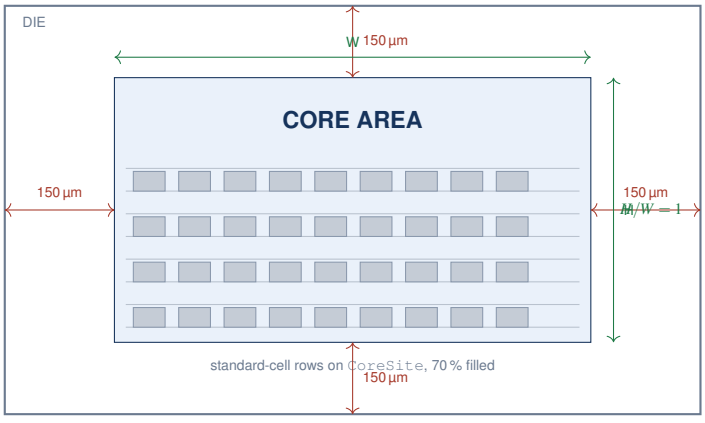
Floorplan formulas

$$\text{Utilization} = \frac{\sum \text{standard-cell area}}{\text{core area}} = 0.7$$

$$\text{Core area} = \frac{\sum \text{cell area}}{0.7}$$

$$\text{Aspect ratio} = \frac{\text{core height}}{\text{core width}} = 1.0 \;\Rightarrow\; W = H = \sqrt{\text{core area}}$$

$$\text{Die width} = \text{core width} + 2 \times 150\,\mu\text{m}\ (+\text{pad ring} + \text{seal ring})$$



`add_tracks -honor_pitch` lays the routing grid so every track lands exactly on a standard-cell pin — no off-grid DRC.

Figure 8.1: *(Explanatory diagram) Floorplan anatomy: die, 150 µm margin ring for the pads and power ring, square core filled to 70 % with standard-cell rows.*

8.2 Floorplan Script (`floorplan.tcl`)

```
# --- Configuration ---
set design      fifo_8bit
set design_node 180

# Configure Innovus for SCL 180nm process rules
setDesignMode -process $design_node

# Create routing tracks aligned to standard-cell pin pitch
# -honor_pitch prevents DRC violations at cell pin interfaces
add_tracks -honor_pitch

# Design Integrity Checks BEFORE committing to floorplan
checkUnique  -verbose      ;# Check for duplicate instance names
check_timing -verbose      ;# Verify SDC completeness

# Floorplan Definition:
# -site CoreSite : placement row type from LEF
# -r             : relative sizing mode
# 1              : aspect ratio H/W = 1.0 (square)
# 0.7            : core utilization = 70%
# 150.00 x4      : left / bottom / right / top margin (microns)
floorPlan -site CoreSite -r 1 0.7 150.00 150.00 150.00 150.00

# Comprehensive post-floorplan design check
checkDesign -all > ./${design}.rpt

# Gate Count Report (10-level hierarchy breakdown)
reportGateCount -hinst $design -level 10 > ./${design}_gatecount.rpt

# Net Statistics (total nets, fanout distribution, net length)
reportNetStat > ./${design}_netcount.rpt

puts "--- Floorplan and Track Generation Completed for $design ---"
```

8.2.1 Command-by-Command Explanation

- `setDesignMode -process 180` — loads the **process-specific rule set** so that all later commands (routing, extraction, DRC) apply 180 nm assumptions.
- `add_tracks -honor_pitch` — generates the routing grid. `-honor_pitch` forces the track pitch to **match the standard-cell pin pitch from the LEF**, so a wire landing on a track lands exactly on a pin. Without it, off-grid pin access produces thousands of DRC violations at the very end of routing.
- `checkUnique -verbose` — detects **duplicate instance names**, which would make later instance-based commands ambiguous.
- `check_timing -verbose` — verifies **SDC completeness**: unclocked registers, unconstrained endpoints, missing input/output delays.
- `floorPlan -site CoreSite -r 1 0.7 150 150 150 150` — the single command that creates the die. Argument order after `-r` is: **aspect ratio**, **utilisation**, then the four margins **left, bottom, right, top**.
- `reportGateCount -level 10` — gate count broken down through 10 levels of hierarchy; used to sanity-check that synthesis produced a sensible cell count.
- `reportNetStat` — total nets, fanout distribution and estimated net lengths; an early warning of routing congestion.

> **Why run the checks *before* `floorPlan`?** The floorplan is the first **irreversible** commitment in the back end. Every later step — power planning, placement, CTS, routing — is built on the die size chosen here. Discovering a missing clock constraint after CTS means redoing all of it.

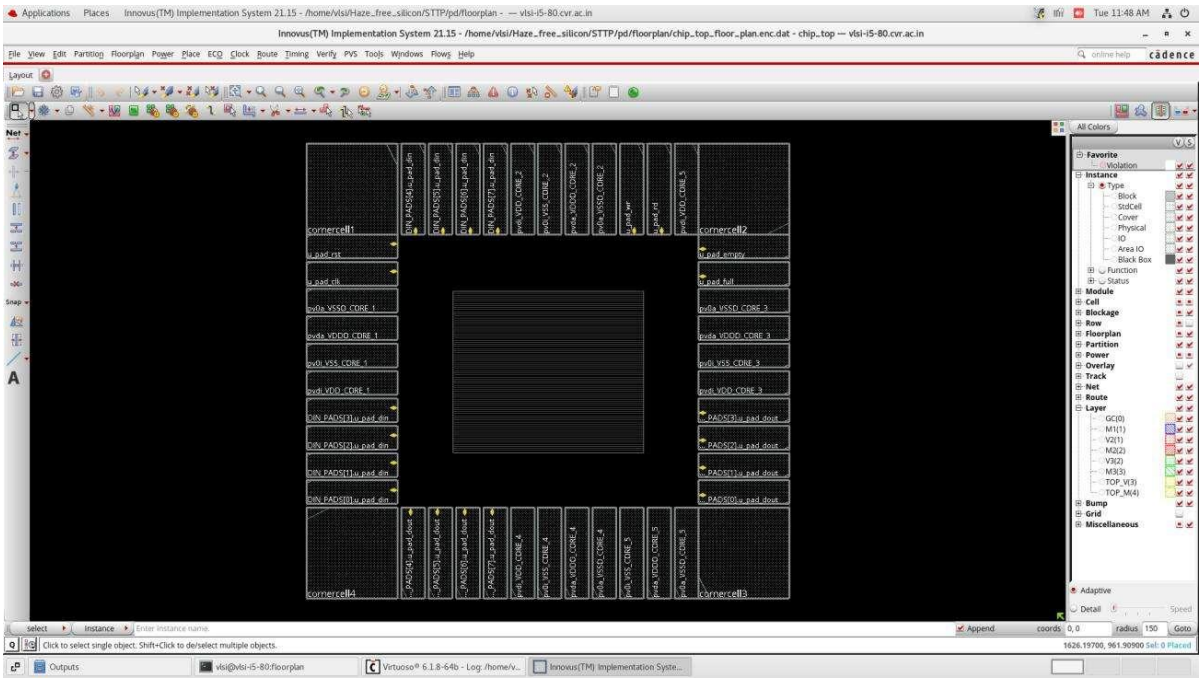


Figure 8.2: ***Figure 5.5 (original)*** *— Cadence Innovus floorplan view: the empty core region (centre), the IO pad ring placed around it according to* `chip_top.io`*, and the 150 µm boundary margins. At this point no standard cell has been placed and no power metal exists.*

PART 9

Power Planning — Constructing the Power Delivery Network

File: `power_plan.tcl`

9.1 Power Delivery Network (PDN) Architecture

- A robust PDN is essential to prevent **two critical failure mechanisms**:
  - **IR Drop** — resistive voltage sag under high current demand;
  - **Electromigration** — gradual metal void formation under sustained high current density.
- The PDN is constructed in **three hierarchical tiers**:

1. **Power Rings:** wide metal loops (**30 µm width** in this design) that encircle the core boundary. They act as **primary current arteries**, collecting current from the power-pad bondwires and distributing it around the perimeter of the core.
2. **Power Stripes:** a regular grid of metal lines running both **horizontally (Metal 3)** and **vertically (Top Metal)** across the full core area. They tap current from the rings and carry it into the interior of the chip, ensuring **no cell is farther than approximately 56 µm from a vertical supply stripe**.
3. **Special Routing (SRoute):** the final micro-scale connection that physically ties the power and ground pins of **every individual standard cell (M1 rails)** and **every IO pad** to the overlying stripe and ring network.

Where the 56 µm number comes from Vertical TOP_M stripes are placed with `set_to_set_distance 112` µm. A cell sitting exactly between two stripes is therefore $112/2 = \mathbf{56}$ µm from the nearest one.

IR drop along that path: $\Delta V = I \cdot R_{M1} = I \cdot R_{sheet} \dfrac{L}{W}$.

Keeping $L \leq 56$ µm bounds the worst-case ΔV. Typical budget: total IR drop $< 5\,\%$ of V_{DD}, i.e. < 90 mV on a 1.8 V supply.

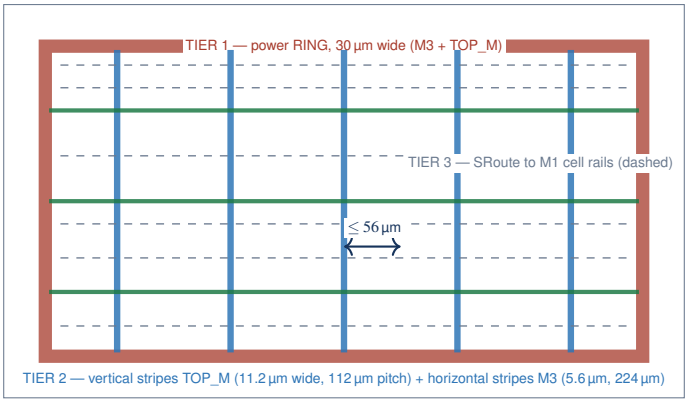


Figure 9.1: *(Explanatory diagram) The three PDN tiers. Current flows bondwire → ring → stripes → M1 cell rails. Each tier gets narrower and more numerous, exactly like an arterial system.*

9.2 Power Plan Script (`power_plan.tcl`)

9.2.1 Part 1 — Power Net Definitions and Global Connections

```
#===============================================================================
# 1. POWER NET DEFINITIONS
#===============================================================================
set PWR_NET_CORE VDD_CORE   ;# Core Power Net     (1.8 V)
```

```
set GND_NET_CORE VSS_CORE   ;# Core Ground Net   (0 V)
set PWR_NET_ALOG VDDO_CORE  ;# IO Ring Power Net (3.3 V)
set GND_NET_ALOG VSSO_CORE  ;# IO Ring Ground Net (0 V)

#===============================================================================
# 2. GLOBAL NET CONNECTIONS
#===============================================================================
clearGlobalNets
globalNetConnect $PWR_NET_CORE -type pgpin -pin VDD  -all
globalNetConnect $GND_NET_CORE -type pgpin -pin VSS  -all
globalNetConnect $PWR_NET_ALOG -type pgpin -pin VDDO -all
globalNetConnect $GND_NET_ALOG -type pgpin -pin VSSO -all
globalNetConnect $PWR_NET_CORE -type tiehi        ;# Logic '1' tie-offs
globalNetConnect $GND_NET_CORE -type tielo        ;# Logic '0' tie-offs
```

- `clearGlobalNets` — wipes any previous connection rules so the script is idempotent.
- `globalNetConnect ... -type pgpin -pin VDD -all` — connects the **VDD power/ground pin of every instance in the design** to the net `VDD_CORE`. This is a **logical** connection; the physical metal is created later by `sroute`.
- `-type tiehi` / `-type tielo` — connects constant logic-1 and logic-0 inputs.
- **Four separate connects** are needed because this is a **dual-domain** chip: core pins (`VDD`/`VSS`) go to the 1.8 V domain; pad pins (`VDDO`/`VSSO`) go to the 3.3 V domain.

If skipped: A gate input left floating because a tie-off was not connected will **oscillate or sit at an intermediate voltage**, turning on both PMOS and NMOS and burning static current. LVS will also flag it as an open.

9.2.2 Part 2 — Core Power Ring

```
#===============================================================================
# 3. CORE POWER RING (30 um width, M3 horizontal, TOP_M vertical)
#===============================================================================
setAddRingMode -stacked_via_top_layer TOP_M \
               -stacked_via_bottom_layer M1 \
               -via_using_exact_crossover_size 1 \
               -orthogonal_only true \
               -skip_via_on_pin { standardcell } \
               -skip_via_on_wire_shape { noshape }

addRing -nets {VDD_CORE VSS_CORE} \
        -type core_rings \
        -follow core \
        -center 1 \
        -layer  {top M3 bottom M3 left TOP_M right TOP_M} \
        -width  {top 30 bottom 30 left 30 right 30} \
        -spacing {top 30 bottom 30 left 30 right 30} \
        -offset {top 10 bottom 10 left 10 right 10} \
        -snap_wire_center_to_grid None
```

- `-stacked_via_top_layer TOP_M` / `-stacked_via_bottom_layer M1` — vias are stacked through **all four metal layers**, giving the lowest possible resistance between ring and cell rails.
- `-orthogonal_only true` — vias are only dropped where two **perpendicular** shapes cross; prevents malformed via stacks.
- `-follow core` — the ring traces the **core boundary**, not the die boundary.
- `-layer {top M3 ... left TOP_M ...}` — the ring uses **M3 for the horizontal segments** and **TOP_M for the vertical segments**, respecting each layer's preferred routing direction.
- `-width 30` — 30 µm of metal. Width directly sets both resistance ($R \propto 1/W$) and electromigration current-carrying capacity ($I_{max} \propto W$).
- `-spacing 30` — 30 µm between the VDD and VSS loops.
- `-offset 10` — the ring starts 10 µm outside the core boundary.

9.2.3 Part 3 — Power Stripes

```
#===============================================================================
# 4. POWER STRIPES
```

```
#===============================================================================
setAddStripeMode -spacing_type edge_to_edge \
                 -stacked_via_top_layer TOP_M \
                 -stacked_via_bottom_layer M1 \
                 -via_using_exact_crossover_size false \
                 -split_vias false \
                 -orthogonal_only true \
                 -allow_jog { padcore_ring block_ring } \
                 -skip_via_on_wire_shape { noshape }

# Vertical Stripes on TOP_M (11.2 um wide, set-to-set 112 um)
addStripe -nets {VDD_CORE VSS_CORE} \
          -layer TOP_M -direction vertical \
          -width 11.2 -spacing 11.2 \
          -set_to_set_distance 112 \
          -start_from left -start_offset 84.28 \
          -switch_layer_over_obs true \
          -max_same_layer_jog_length 2 \
          -padcore_ring_top_layer_limit TOP_M \
          -padcore_ring_bottom_layer_limit M1 \
          -block_ring_top_layer_limit TOP_M \
          -block_ring_bottom_layer_limit M1 \
          -use_wire_group 0 \
          -snap_wire_center_to_grid None \
          -skip_via_on_pin { standardcell }

# Horizontal Stripes on M3 (5.6 um wide, set-to-set 224 um)
addStripe -nets {VDD_CORE VSS_CORE} \
          -layer M3 -direction horizontal \
          -width 5.6 -spacing 11.2 \
          -set_to_set_distance 224 \
          -start_from bottom -start_offset 120.4 \
          -switch_layer_over_obs true \
          -max_same_layer_jog_length 2 \
          -padcore_ring_top_layer_limit TOP_M \
          -padcore_ring_bottom_layer_limit M1 \
          -block_ring_top_layer_limit TOP_M \
          -block_ring_bottom_layer_limit M1 \
          -use_wire_group 0 \
          -snap_wire_center_to_grid None \
          -skip_via_on_pin { standardcell }
```

- **Vertical stripes on TOP_M** are **wider (11.2 µm)** and **denser (112 µm pitch)** than the horizontal ones. TOP_M is the thickest metal, so it has the lowest sheet resistance and is the natural choice for carrying the bulk of the supply current.
- **Horizontal stripes on M3** are **5.6 µm wide** at **224 µm pitch** — half the width, double the pitch. Their job is to **stitch the vertical stripes together** into a mesh, not to carry current end-to-end.
- `-start_offset 84.28` / `120.4` — offsets chosen so the stripes land on the routing grid and avoid colliding with the ring.
- `-switch_layer_over_obs true` — a stripe may hop to another layer to get over an obstruction rather than stopping.
- `-skip_via_on_pin {standardcell}` — prevents vias being dropped directly on a standard-cell signal pin, which would block the signal router.

> A mesh (vertical × horizontal) rather than parallel stripes is used because a mesh provides **multiple redundant current paths** to every point. If one path is blocked by a macro or an obstruction, current simply redistributes — the same reason a city grid is more robust than a single highway.

9.2.4 Part 4 — Special Routing (SRoute)

```
#===============================================================================
# 5. SPECIAL ROUTING (Pad Pins + Core Standard Cell Rails)
#===============================================================================
setSrouteMode -viaConnectToShape { ring stripe followpin }

sroute -connect { padPin corePin } \
       -layerChangeRange { M1(1) TOP_M(4) } \
       -padPinPortConnect { allPort oneGeom } \
       -padPinTarget { nearestTarget } \
       -corePinTarget { firstAfterRowEnd } \
       -allowJogging 1 \
```

```
        -crossoverViaLayerRange { M1(1) TOP_M(4) } \
        -nets {VDD_CORE VSS_CORE} \
        -allowLayerChange 1 \
        -targetViaLayerRange { M1(1) TOP_M(4) }
```

- `-connect {padPin corePin}` — two jobs in one command:
  - **padPin** — connect every IO power-pad pin to the ring;
  - **corePin** — connect every standard-cell VDD/VSS pin to the overlying stripes (also called **follow-pin routing**, because the metal follows the M1 rail along each cell row).
- `-layerChangeRange {M1(1) TOP_M(4)}` — the router may use any layer from M1 up to TOP_M.
- `-corePinTarget firstAfterRowEnd` — each cell row is tied to the first available stripe past the end of the row.
- `-allowJogging 1` — small detours are permitted to dodge obstacles.

9.2.5 Part 5 — PDN Verification and Save

```
#==========================================================================
# 6. PDN VERIFICATION & SAVE
#==========================================================================
set init_top_cell "chip_top"
verifyConnectivity -type all -connLoop -error 10000 -warning 5000 \
                   > ./${init_top_cell}.connectivity.rpt
checkDesign -all   > ./${init_top_cell}_PP_design.rpt
timeDesign -prePlace > ./timing_prePlace.rpt
saveDesign ./${init_top_cell}_fp_powerplan.enc
```

- `verifyConnectivity -type all -connLoop` — the **acceptance test for the PDN**. It searches for **opens** (a cell not reached by power) and **shorts** (VDD touching VSS). `-connLoop` additionally verifies the ring forms a **closed loop** — which is exactly what the corner cells were inserted to guarantee.
- `timeDesign -prePlace` — a first timing snapshot before any cell has been placed.
- `saveDesign ...enc` — writes the Innovus database checkpoint. **Every major milestone is saved** so the flow can be restarted from the last good state rather than from scratch.

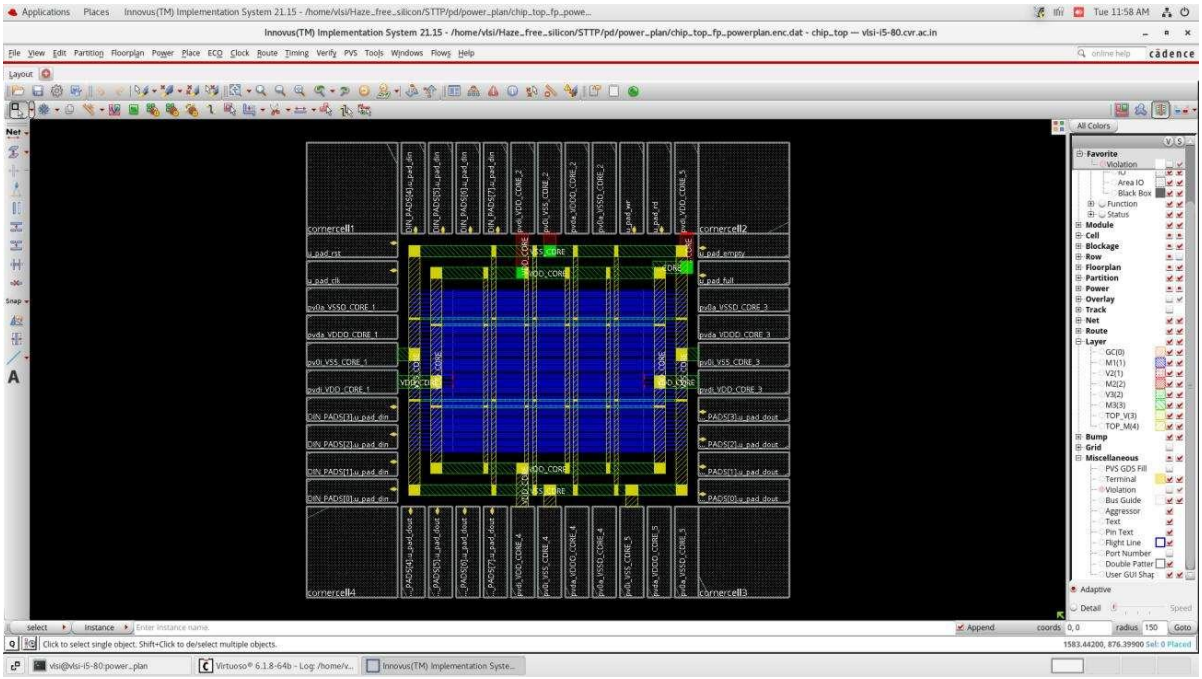


Figure 9.2: ***Figure 5.6 (original)*** *— Cadence Innovus power delivery network: the 30 µm power ring around the core, the vertical and horizontal power stripes forming the mesh, and the special routing that ties the mesh down to the standard-cell M1 rails.*

PART 10

Physical Cell Placement — End Caps and IO Fillers

File: `physical_placement.tcl`

10.1 End Cap Cells (Well Termination)

- Standard cells are designed to be placed in **continuous rows**.
- The **N-Well and P-Well layers** that run through each row **cannot abruptly terminate** at the row boundary without:
  - violating **Oxide Diffusion (OD) density rules**;
  - potentially causing **latch-up**.
- **Latch-up** is a destructive condition in CMOS circuits where **parasitic npnp thyristor structures** inadvertently trigger, creating a **low-resistance path from VDD to VSS**.
- **End Cap cells** (specifically `feedth9` in the SCL 180 nm library) are placed as **physical bookends** at the beginning and end of every standard-cell row. They provide:
  - graceful N-Well and P-Well layer termination that satisfies **DRC enclosure rules**;
  - **VDD and VSS well-tie connections** that suppress latch-up by ensuring the well potentials are clamped at every row edge.

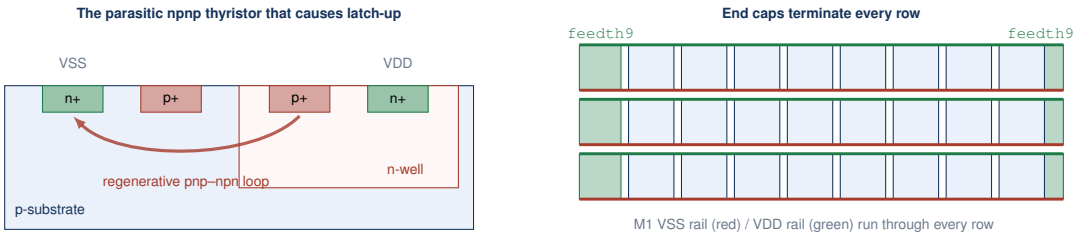


Figure 10.1: *(Explanatory diagram) Left: the parasitic pnp–npn pair that latches VDD to VSS once triggered. Right: `feedth9` end caps bookend every row, terminating the wells cleanly and tying them to the supply so the parasitic loop cannot turn on.*

Why: The parasitic bipolar pair has combined gain $\beta_{pnp} \cdot \beta_{npn}$. If that product exceeds 1 and a transient injects enough base current, the pair latches **on** and stays on until power is removed — frequently melting the metal first. Well ties pull the base resistances down so the loop gain never reaches unity.

10.2 IO Filler Cells (Pad Ring Continuity)

- When individual pad cells (`pc3d01`, `pt3t03u`, `pvdi`, etc.) are placed around the IO ring with the **10 µm gap** specified in the `.io` file, the metal rails (**VDDO** and **VSSO**) that run continuously through the ring body are **interrupted at every gap**.
- If these breaks are not repaired:
  - the **ESD protection network becomes electrically open**;
  - **power cannot circulate** around the ring.
- **IO filler cells** (the `pfeed_xxxx` family) are **metal-only cells with no logic function**. They bridge the gaps, ensuring a **continuous low-resistance power ring** around the full perimeter of the die.
- The `addIoFiller` command accepts a library of filler cells of varying widths and **automatically calculates the optimal combination** to fill each gap.

10.2.1 IO Filler Cell Widths

- `pfeed30000`: 300 µm wide — used for large gaps.
- `pfeed10000`: 100 µm wide.
- `pfeed02000`: 20 µm wide.
- `pfeed01000`: 10 µm wide.
- `pfeed00540`: 5.4 µm wide.
- `pfeed00120`: 1.2 µm wide.
- `pfeed00040`: 0.4 µm wide.
- `pfeed00010`: 0.1 µm wide — used for sub-micron residual gaps.

Why the widths form a decade-like series. The set $\{300, 100, 20, 10, 5.4, 1.2, 0.4, 0.1\}$ µm is a **greedy bin-packing basis**. Any gap can be decomposed as $G = \sum n_i w_i$ with a small number of pieces, leaving **zero residual gap**. The 0.1 µm cell exists purely to absorb the final rounding error.

10.3 Script

```
# End Cap Insertion -- placed at both ends of every standard-cell row
# feedth9: SCL 180nm library cell providing N/P-Well termination
addEndCap -preCap feedth9 -postCap feedth9 -prefix ENDCAP

# IO Filler Insertion -- fills gaps in the IO pad ring
# Tool uses a bin-packing algorithm to fill every gap exactly
addIoFiller -cell {pfeed30000 pfeed10000 pfeed02000 pfeed01000 \
                   pfeed00540 pfeed00120 pfeed00040 pfeed00010} \
            -prefix IO_FILLER
```

- `-preCap feedth9` — the cell placed at the **start** (left end) of each row.
- `-postCap feedth9` — the cell placed at the **end** (right end) of each row.
- `-prefix ENDCAP` / `-prefix IO_FILLER` — names every inserted instance with this prefix so the cells can later be identified, counted or deleted as a group. Good hygiene for any automatically inserted physical cell.
- The filler list is given in **descending width order**, which is what the bin-packing algorithm expects for a greedy fill.

If skipped: Skip `addEndCap` ⇒ OD-density DRC errors along every row end plus latch-up risk. Skip `addIoFiller` ⇒ the IO ring's VDDO/VSSO rails are segmented; `verifyConnectivity` reports opens and the ESD clamps on isolated segments are useless.

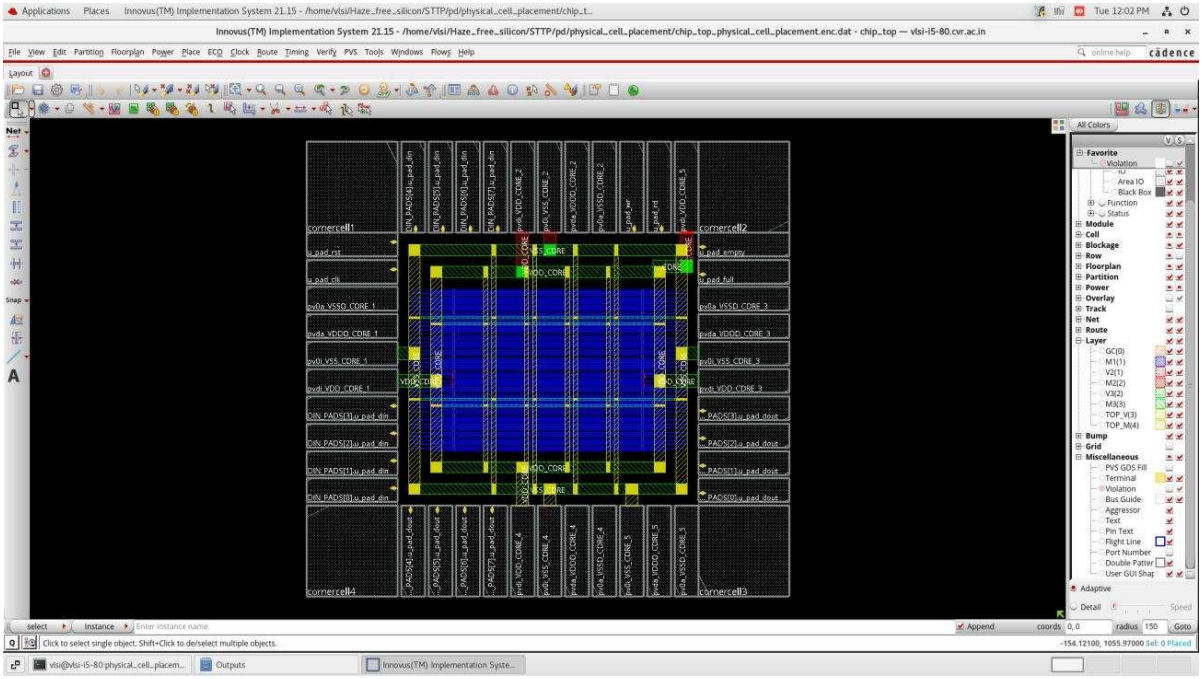


Figure 10.2: ***Figure 5.7 (original)*** *— Cadence Innovus physical placement: end caps and IO filler cells completing the standard-cell rows and the IO pad ring. The ring is now electrically continuous all the way round the die.*

PART 11

Standard Cell Placement

File: `placement.tcl`

11.1 Placement Theory

- Standard-cell placement assigns **precise X,Y coordinates** to every logic cell in the design.
- The process occurs in **two stages**:
  - **Global Placement** — the tool computes approximate locations that **minimise total wire length** while balancing congestion and timing.
  - **Detailed Placement (Legalization)** — each cell is **snapped to the nearest legal position** on the placement grid, ensuring **no overlaps** and **proper alignment with the horizontal power rails**.
- Immediately after legalization, a **Pre-Placement Optimization (PPO)** pass is executed that:
  - inserts buffers to fix high-fanout nets;
  - sizes up cells on critical paths;
  - restructures logic trees to reduce transition-time violations;
  - all **without moving cells off their legal positions**.

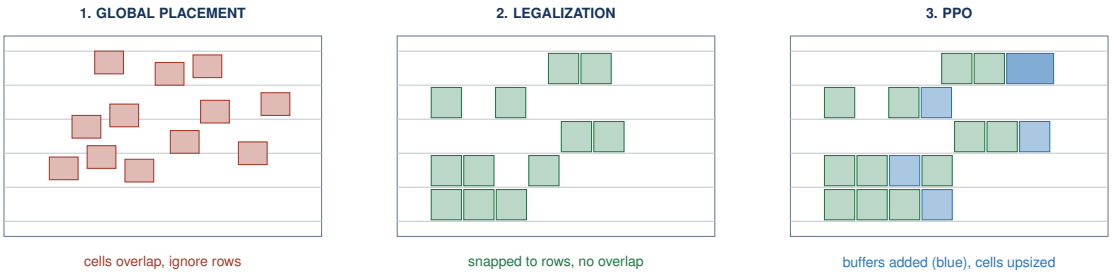


Figure 11.1: *(Explanatory diagram) The three placement stages executed by the single command `place_opt_design`.*

11.2 Placement Configuration

11.2.1 `setPlaceMode` Parameters

- `-place_global_cong_effort high`: forces the placer to **spread cells apart in congested regions**. Without this, densely connected clusters of cells are co-located, leaving insufficient routing tracks for the signal wires between them. **Routing-time congestion greater than 2 % overflow is a strong indicator of a subsequent routing failure**.
- `-place_global_timing_effort high`: prioritises cells on **timing-critical paths** (those with the smallest positive slack). These cells are placed as close to their fanin drivers as possible to minimise net lengths and propagation delays.
- `-place_global_uniform_density true`: prevents the tool from creating **local density hotspots** by distributing cells evenly across the available core area.
- `-place_global_max_density 0.7`: a **hard constraint** preventing any local region from exceeding 70 % cell utilisation. This reserves 30 % of every local area for **routing vias and optimisation buffers** inserted by later physical-synthesis steps.

11.2.2 `setOptMode` Parameters

- `-effort high`: enables complex optimisation transformations including **logic restructuring** and **multi-layer assignment**.

- `-setupTargetSlack 0.0`: the optimisation target is **zero negative slack**. Any path with negative slack is a timing violation that must be resolved before proceeding to CTS.
- `-holdTargetSlack 0.0`: hold violations are also tracked. Since the clock tree has **not yet been built**, hold analysis uses the **ideal clock assumption**.
- `-fixFanoutLoad true`: automatically inserts clock buffers or data buffers on high-fanout nets (e.g. a reset signal driving many flip-flops) to reduce the transition time below the `max_transition` constraint.

Congestion overflow

$$\text{Overflow} = \frac{\text{routing demand in a G-Cell}}{\text{routing supply (available tracks)}}$$

Overflow $> 2\,\%$ at placement almost always becomes a **routing failure or DRC explosion** later. It is far cheaper to fix congestion by spreading cells now than to fix it after CTS.

11.3 Placement Script (`placement.tcl`)

```
# Reset and configure placement engine
setPlaceMode -reset
setPlaceMode -place_global_cong_effort high \
             -place_global_timing_effort high \
             -place_global_uniform_density true \
             -place_design_refine_place true \
             -place_global_max_density 0.7

# Configure optimization engine for post-placement timing repair
setOptMode -reset
setOptMode -effort high \
           -setupTargetSlack 0.0 \
           -holdTargetSlack  0.0 \
           -fixFanoutLoad true

# Execute global placement + legalization + PPO in a single call
place_opt_design

#================================================================================
# POST-PLACEMENT ANALYSIS
#================================================================================

# 1. PHYSICAL & CONGESTION ANALYSIS
checkPlace ./placement_legality.rpt
reportDensityMap  > ./placement_density.rpt
reportCongestion -hotspot > ./placement_congestion.rpt
reportGateCount           > ./placement_gate_count.rpt

# 2. TIMING ANALYSIS (Pre-CTS, Ideal Clock)
timeDesign -preCTS -idealClock -pathReports -drvReports -slackReports \
           -numPaths 50 -prefix placement_preCTS > ./placement_timing.rpt
reportFanoutViolation > ./placement_fanout.rpt
reportCapViolation    > ./placement_cap.rpt

# 3. AREA & POWER ANALYSIS
report_area  > ./placement_area.rpt
report_power > ./placement_power.rpt

# 4. CONNECTIVITY INTEGRITY
verifyConnectivity -type all -error 1000 -warning 50 > ./placement_connectivity.rpt
```

11.3.1 What Each Analysis Command Checks

- `checkPlace` — legality: no overlapping cells, no cell off-row, no cell outside the core.
- `reportDensityMap` — cell density per region; reveals hotspots that `max_density` failed to prevent.
- `reportCongestion -hotspot` — routing demand vs. supply per G-Cell; the **single most important pre-routing report**.
- `timeDesign -preCTS -idealClock` — **ideal clock** means zero insertion delay and zero skew are assumed, because the tree does not exist yet. Numbers here are optimistic by design.
- `-numPaths 50` — reports the 50 worst paths rather than just one, so systemic problems are visible.
- `reportFanoutViolation` / `reportCapViolation` — the DRV checks against `set_max_fanout 16`

and the cell load limits.

- `verifyConnectivity -error 1000 -warning 50` — aborts the run if more than 1000 connectivity errors are found, preventing a hopeless database from being carried forward.

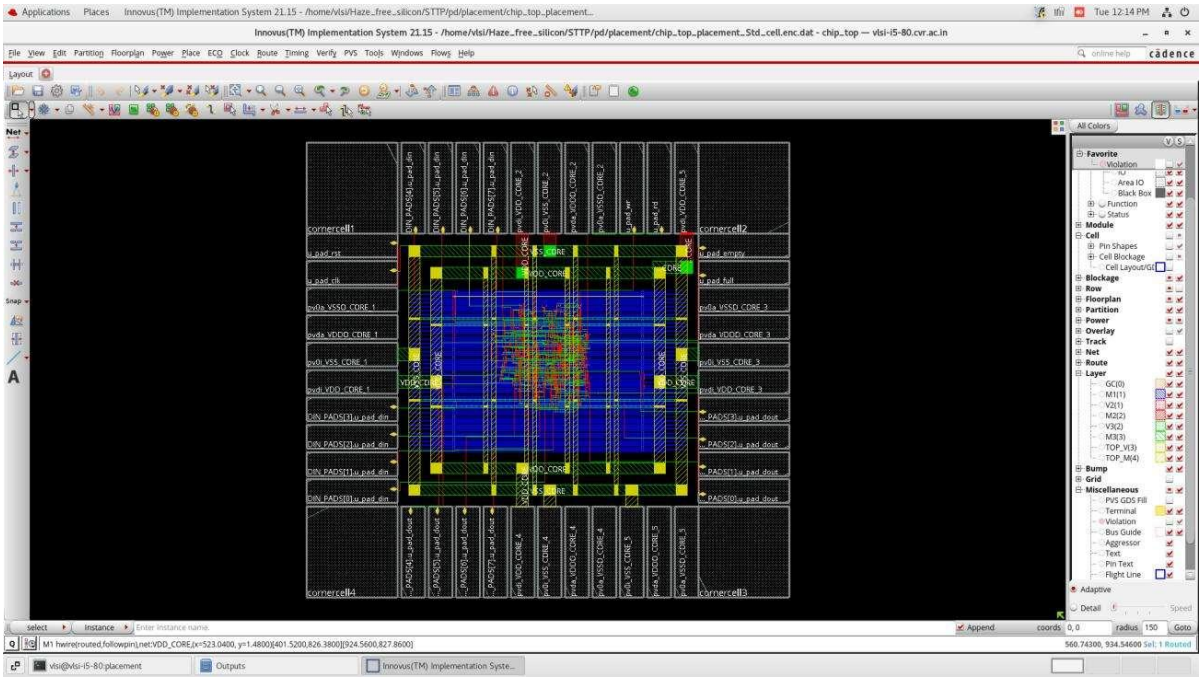


Figure 11.2: ***Figure 5.8 (original)*** *— Cadence Innovus post-placement view: standard cells placed and legalized inside the core, with the IO pad ring and the power mesh already in place.*

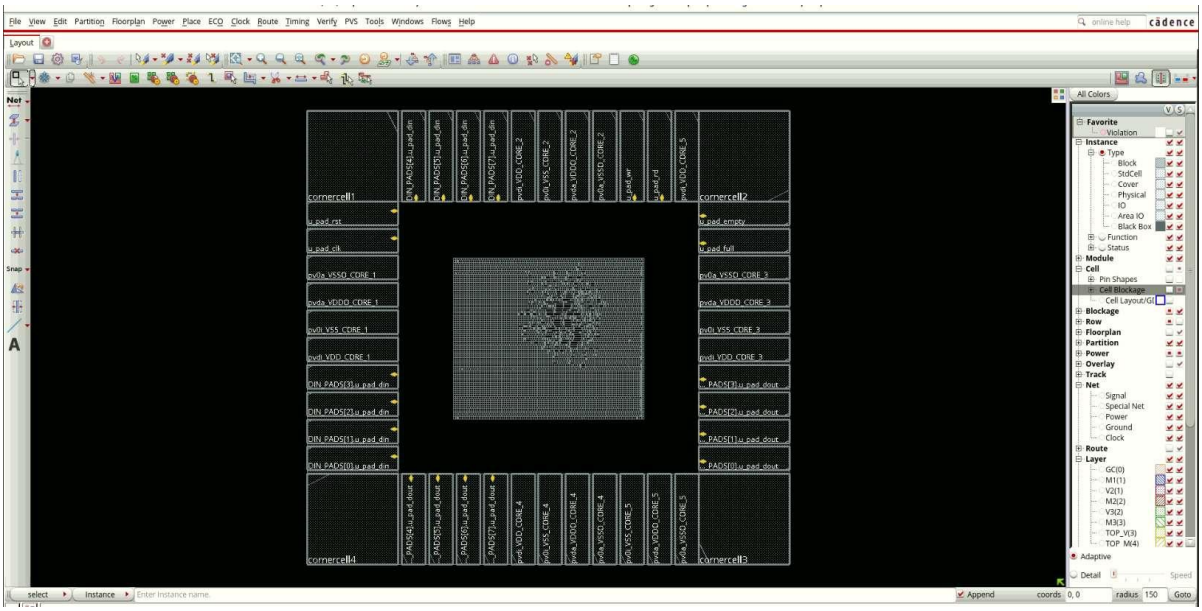


Figure 11.3: ***Figure 5.9 (original)*** *— post-placement congestion and density map: routing-overflow analysis before CTS. Dense regions here predict exactly where detailed routing will later struggle.*

PART 12

Clock Tree Synthesis (CTS)

File: `cts.tcl`

12.1 Theory of Clock Distribution

- CTS transforms the clock signal from a **logical definition** (a `create_clock` SDC statement) into a **physical buffer network** that distributes the clock from its source at the pad pin to **every flip-flop (FF) sink** in the design.
- Two primary quality metrics describe a clock tree:
  - **Clock Skew:** the difference in clock arrival time between the earliest and the latest flip-flop. Skew **directly reduces the effective timing budget** for setup analysis and **introduces hold violations**. **Target:** < 0.3 **ns.**
  - **Clock Insertion Delay (Latency):** the total propagation time from the clock source to the leaf flip-flops. This delay **affects the accuracy of setup and hold timing after CTS**.
- This flow uses the **CCOpt (Clock Concurrent Optimization)** engine in Cadence Innovus.
- Unlike classical CTS methods, CCOpt **simultaneously builds the clock tree and optimises the logic datapath**, enabling faster timing convergence.
- The clock tree is routed preferentially on **upper metal layers (M3, M4, TOP_M)**, which have **lower resistance** and are **less susceptible to coupling noise** from adjacent data signals.

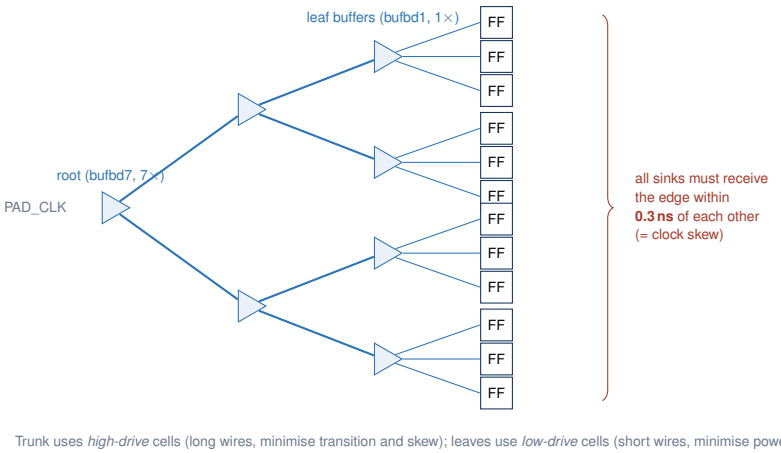
How skew enters the timing equations

$$\textbf{Setup:}\quad T_{cq} + T_{comb} + T_{setup} \leq T_{clk} + \underbrace{(T_{capture} - T_{launch})}_{\text{skew}}$$

$$\textbf{Hold:}\quad T_{cq} + T_{comb} \geq T_{hold} + \underbrace{(T_{capture} - T_{launch})}_{\text{skew}}$$

Positive skew (capture clock arrives late) *helps* setup but *hurts* hold. **Negative skew** does the opposite. Because the sign flips between the two checks, skew must be **minimised**, not merely made positive.

With a 10 ns period, the 0.3 ns skew target consumes **3 %** of the budget.



Trunk uses *high-drive* cells (long wires, minimise transition and skew); leaves use *low-drive* cells (short wires, minimise power).

Figure 12.1: *(Explanatory diagram) Structure of the CCOpt clock tree and the meaning of skew. Buffer drive strength deliberately decreases from trunk to leaf.*

12.2 CTS Script (`cts.tcl`)

```
############################################################
# CTS FOR chip_top -- SCL 180nm -- CCOpt Engine
############################################################

# 1. CLEANUP -- remove any previous failed CTS specifications
delete_ccopt_clock_tree_spec

# 2. SPECIFICATION -- auto-define tree from SDC create_clock
create_ccopt_clock_tree_spec

# 3. LIBRARY CELL SELECTION
# Allow a range of drive strengths (1x ... 7x)
# bufbd = balanced buffer; inv0d/invbd = inverter variants
set_ccopt_property buffer_cells   {bufbd1 bufbd2 bufbd4 bufbd7 \
                                   buffd1 buffd2 buffd4 buffd7}
set_ccopt_property inverter_cells {inv0d1 inv0d2 inv0d4 inv0d7 \
                                   invbd2 invbd4 invbd7}

# 4. OPTIMIZATION TARGETS
# Target clock skew: 300 ps across all sink flip-flops
set_ccopt_property target_skew 0.3
# Maximum allowed transition on clock nets: 500 ps
set_ccopt_property target_max_trans 0.5

# 5. EXECUTE CTS -- builds tree, routes clock nets, optimizes logic
ccopt_design -outDir ./cts

############################################################
# 6. POST-CTS VERIFICATION (Propagated Clock mode)
############################################################

# Setup timing with real clock insertion delay
timeDesign -postCTS -prefix postCTS_setup > ./cts_setup.rpt

# Hold timing -- CTS skew often introduces hold violations here
timeDesign -postCTS -hold -prefix postCTS_hold > ./cts_hold.rpt

# Clock tree structure report: skew, latency, buffer count
report_ccopt_clock_trees > ./cts_summary.rpt
```

12.2.1 Command-by-Command Explanation

- `delete_ccopt_clock_tree_spec` — clears any stale specification so a re-run does not inherit a broken tree definition.
- `create_ccopt_clock_tree_spec` — auto-generates the tree specification by **tracing from the `create_clock` definition in the SDC** to every sink pin. This is why the clock had to be defined at `PAD_CLK` rather than at an internal net.
- `set_ccopt_property buffer_cells {...}` — the **only** cells CCOpt is permitted to insert. Restricting the list prevents the tool from choosing cells with poor rise/fall balance.
- `set_ccopt_property target_skew 0.3` — the skew budget in nanoseconds.
- `set_ccopt_property target_max_trans 0.5` — the slew limit on clock nets. Note this is **three times tighter** than the global `set_max_transition 1.5` for data nets.
- `ccopt_design -outDir ./cts` — the single command that builds the tree, routes the clock nets and concurrently optimises the datapath.
- `timeDesign -postCTS` — now analysed in **propagated-clock** mode: the real insertion delay replaces the SDC estimates from Constraint Section 3.
- `timeDesign -postCTS -hold` — hold is checked **separately and explicitly** because **CTS skew is the number-one source of new hold violations**.
- `report_ccopt_clock_trees` — reports achieved skew, insertion delay and buffer count — the CTS acceptance report.

12.3 Library Cell Selection Rationale

- The list of permissible buffer and inverter cells covers a range of drive strengths from **1×** (`bd1, 0d1`) to **7×** (`bd7, 0d7`).
- The CCOpt engine uses:
  - **smaller, weaker cells** for **short local branches** close to the flip-flop sinks — minimising power;
  - **larger, higher-drive cells** on the **trunk** of the tree, to drive long wires that span large portions of the die — minimising transition time and skew.

> **Why:** Clock power dominates dynamic power in a synchronous chip because the clock is the only net that switches *every* cycle: $P = \alpha C V^2 f$ with $\alpha = 1$ (actually 2 transitions per cycle) for the clock, versus $\alpha \approx 0.1$–0.2 for data. Using 7× buffers everywhere would multiply C and therefore clock power several-fold for no timing benefit on short branches.

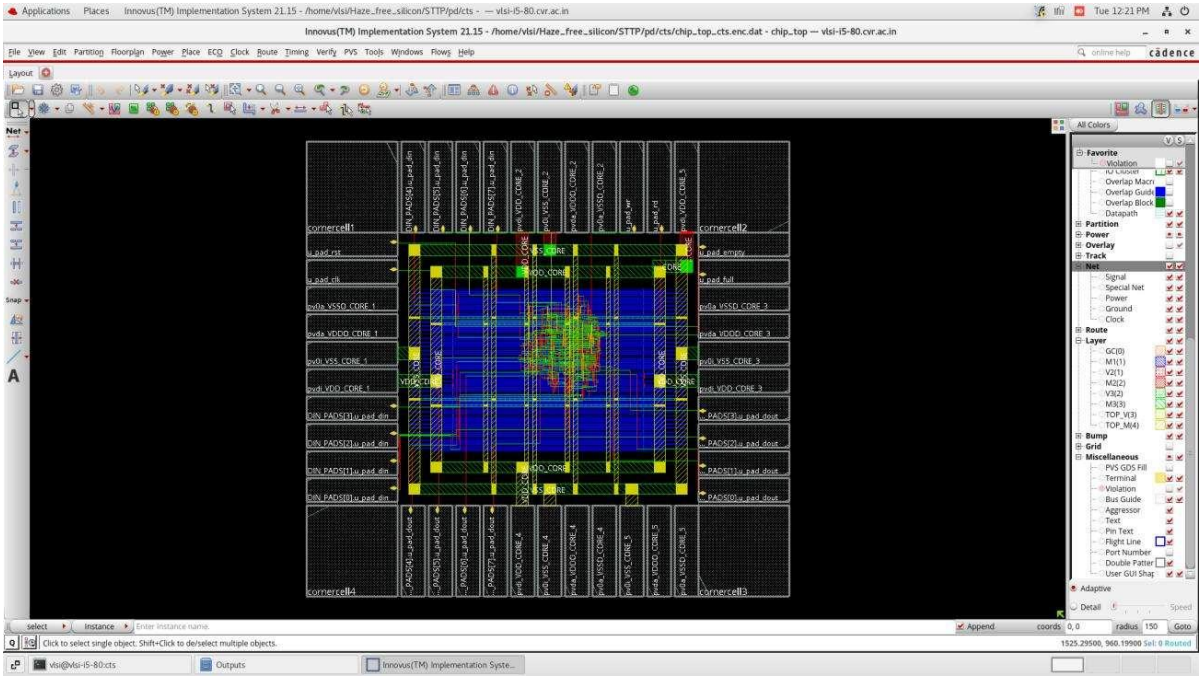


Figure 12.2: *Figure 5.10 (original) — Cadence Innovus post-CTS view: the clock buffer tree with M3/TOP_M routing. The clock network is visible as the dense green structure spreading from the clock pad into the core.*

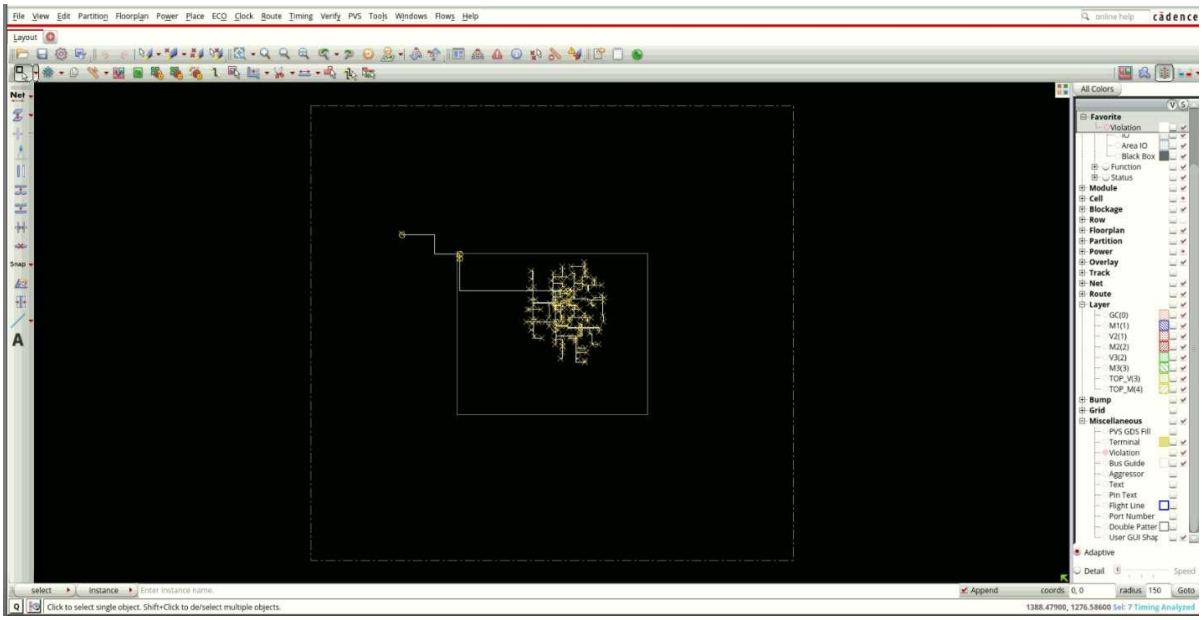


Figure 12.3: ***Figure 5.11 (original)*** *— post-CTS timing analysis showing the clock tree in the design. This display isolates the clock network so skew and latency can be inspected branch by branch.*

Hold-fix ordering matters. Hold violations created by CTS are repaired by **inserting delay buffers**, which adds cells and area. Fixing hold *before* routing is preferred, because post-route hold fixes require ripping up and re-routing nets.

PART 13

Signal Routing

File: `routing.tcl`

13.1 NanoRoute Overview

- Signal routing creates the **physical metal wire connections** between all standard cells, macros and IO pads using the **four available metal layers (Metal 1 through Metal 4)** of the SCL 180 nm technology.
- The **NanoRoute** engine in Cadence Innovus performs routing in **two sequential phases**:

1. **Global Routing:** the die area is divided into a **coarse mesh of G-Cells**. NanoRoute calculates the approximate routing path for every net through this mesh, solving a **multi-commodity flow problem** to balance routing-resource utilisation across the chip. The global router identifies potential **congestion hot-spots** and applies **detour routing** to prevent resource exhaustion in critical areas.
2. **Detailed Routing:** within each G-Cell, NanoRoute places the **actual metal polygons and via cuts**, resolving all **Design Rule Check (DRC)** violations concerning minimum metal width, spacing between adjacent wires, via enclosure and metal-to-metal overlap. The detailed router must simultaneously satisfy **thousands of geometric design rules** while completing every electrical connection.

13.2 Key Routing Challenges

13.2.1 Antenna Effect

- During **plasma etching** in fabrication, long metal wires that are **not yet connected to a transistor gate** (floating wires) **act as antennas**.
- They **collect free electrons from the plasma** and can build up a charge large enough to **tunnel through and permanently damage the gate oxide** of connected transistors.
- The NanoRoute **antenna-diode insertion** feature (`-routeInsertAntennaDiode true`) automatically places **diode cells adjacent to vulnerable gate pins** to provide a **safe discharge path**.

Antenna ratio

$$\text{Antenna Ratio} = \frac{A_{metal}\ (\text{area of connected floating metal})}{A_{gate}\ (\text{area of the gate oxide it reaches})}$$

Foundries specify a maximum (commonly 100–400 at 180 nm). Fixes: (1) **antenna diode** — a reverse-biased junction to substrate that leaks the charge away; (2) **layer hopping** — break the long wire with a via up to a higher layer so the segment attached to the gate during that etch step is short.

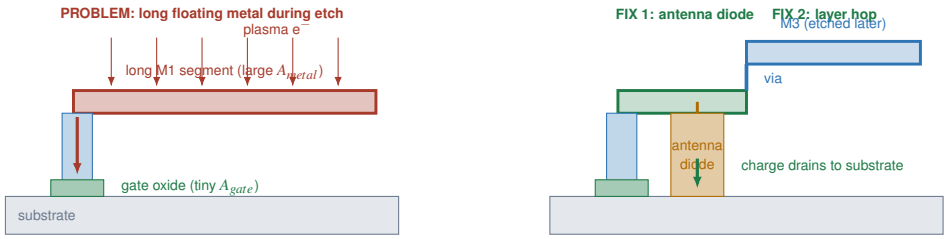


Figure 13.1: *(Explanatory diagram) Left: charge collected by a long floating wire has nowhere to go but through the thin gate oxide. Right: a diode gives it a safe path, and layer hopping shortens the exposed segment.*

13.2.2 Signal Integrity (Crosstalk)

- When **two long parallel wires** run adjacent to each other, they form a **capacitive coupling structure**.
- If both aggressors switch simultaneously, the resulting **displacement current** induces a **voltage spike** or a **timing perturbation** on the victim net.

- **Two modes of crosstalk** are relevant:
  - **Crosstalk Noise** — a glitch that causes a logic-0 to momentarily appear as a logic-1 (or vice versa).
  - **Crosstalk Delay** — a timing shift that **advances or retards** signal transitions on the victim.
- The `-routeWithSiDriven true` option instructs NanoRoute to **spread critical nets apart** and to **prefer routing on different metal layers** to reduce coupling capacitance.

Crosstalk glitch amplitude (capacitive divider)

$$\Delta V_{victim} = V_{DD} \cdot \frac{C_{coupling}}{C_{coupling} + C_{ground}}$$

Same-direction aggressor switching ⇒ victim speeds up (*negative* crosstalk delay). **Opposite-direction** switching ⇒ victim slows down (*positive* crosstalk delay) — effective coupling capacitance can appear up to 2× larger (the Miller effect).

13.2.3 OCV and CPPR Analysis

- **On-Chip Variation (OCV)** accounts for the **statistical spread** of transistor threshold voltage, channel length and gate-oxide thickness **across a manufactured wafer**.
- In OCV analysis the timing engine applies **derating factors**:
  - it assumes the **launch clock path experiences worst-case delay** while the **capture clock path experiences best-case delay** (for setup analysis);
  - and **vice versa** for hold analysis.
- **Common Path Pessimism Removal (CPPR)** corrects an inherent over-pessimism: because **the same buffer cannot simultaneously be slow and fast**, any clock buffer **shared by both the launch and the capture paths** is **credited back** to the timing budget.

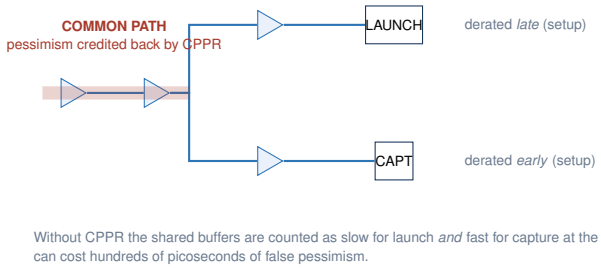


Figure 13.2: *(Explanatory diagram) Common Path Pessimism Removal: only the* divergent *part of the two clock paths is legitimately derated.*

13.3 Routing Script (`routing.tcl`)

```
#===============================================================================
# 1. ROUTING CONFIGURATION (NanoRoute)
#===============================================================================

# Insert antenna diodes on nets with high antenna ratio
setNanoRouteMode -quiet -routeInsertAntennaDiode true

# Prioritize critical-path nets during routing (timing-driven)
setNanoRouteMode -quiet -routeWithTimingDriven true

# Space out high-activity nets to reduce crosstalk (SI-driven)
setNanoRouteMode -quiet -routeWithSiDriven true

# Execute global and detailed routing in a single pass
routeDesign -globalDetail

#===============================================================================
# 2. ANALYSIS MODE: OCV & SIGNAL INTEGRITY
#===============================================================================

# Enable OCV with CPPR for both setup and hold analysis
```

```
setAnalysisMode -reset
setAnalysisMode -analysisType onChipVariation -cppr both

# Enable SI-aware delay calculation (Advanced Analysis Engine)
setDelayCalMode -engine aae -SIAware true

#=============================================================================
# 3. POST-ROUTE OPTIMIZATION
#=============================================================================
setOptMode -reset
setOptMode -effort high \
           -fixCap true \
           -fixTran true \
           -fixFanoutLoad true \
           -setupTargetSlack 0.0 \
           -holdTargetSlack 0.05   ;# 50 ps margin for hold robustness

# Final optimization pass: fix residual setup/hold violations
optDesign -postRoute -setup -hold

#=============================================================================
# 4. FINAL SIGNOFF VERIFICATION
#=============================================================================

# Geometric DRC: metal width, spacing, enclosure
set_verify_drc_mode -check_implant true -check_separation true -limit 5000
verify_drc -report ./signoff_geometry.rpt

# Connectivity: opens and shorts
verifyConnectivity -type all -error 5000 > ./signoff_connectivity.rpt

# Signoff timing: setup and hold with OCV/CPPR
timeDesign -signoff -prefix signoff_setup > ./signoff_setup.rpt
timeDesign -signoff -hold -prefix signoff_hold > ./signoff_hold.rpt
```

13.3.1 Option-by-Option Explanation

- `-routeInsertAntennaDiode true` — antenna fix, as described above.
- `-routeWithTimingDriven true` — critical nets get the **shortest, least-detoured** paths and the **lowest-resistance layers**.
- `-routeWithSiDriven true` — crosstalk-aware spacing and layer assignment.
- `routeDesign -globalDetail` — runs **global then detailed** routing in one command.
- `setAnalysisMode -analysisType onChipVariation -cppr both` — switches the timer from a single fixed corner to **statistical OCV derating** and enables CPPR for **both** setup and hold.
- `setDelayCalMode -engine aae -SIAware true` — the **Advanced Analysis Engine** computes delay including the crosstalk-induced delta delays, rather than using a lumped-C approximation.
- `-fixCap true` / `-fixTran true` — repair remaining capacitance and transition DRVs using the now-**real** extracted parasitics.
- `-holdTargetSlack 0.05` — hold is targeted to **+50 ps**, not 0. This deliberate margin makes the design robust against extraction error and OCV effects that the model may underestimate. **Setup** is left at 0.0 because extra setup margin costs area and power for no reliability gain.
- `optDesign -postRoute -setup -hold` — the final cleanup pass, now operating on real routed parasitics rather than estimates.
- `verify_drc -limit 5000` — caps the number of reported violations so the log file stays usable if something goes badly wrong.
- `timeDesign -signoff` — the strictest timing mode; its reports are what the project signs off on.

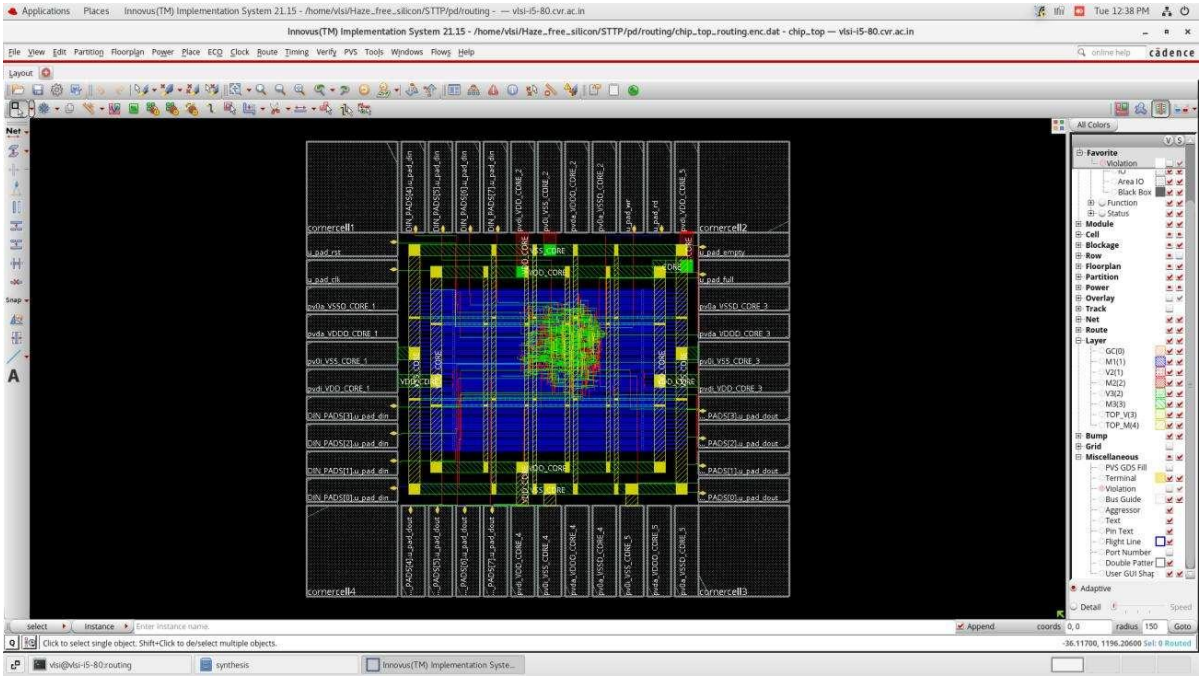


Figure 13.3: ***Figure 5.12 (original)*** *— Cadence Innovus fully routed* `chip_top`*: Metal 1 through Top Metal, signal and power routing complete.*

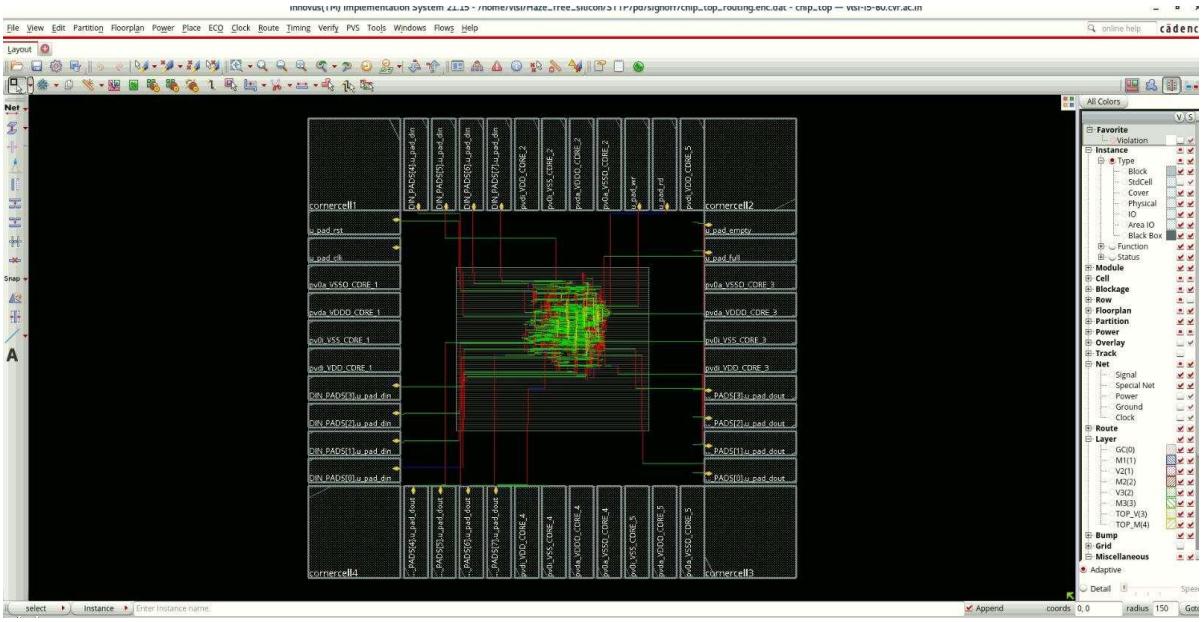


Figure 13.4: ***Figure 5.13 (original)*** *— post-route view showing* only *the routing in the design. Hiding the cells makes it easy to spot detours, long nets and layer usage.*

PART 14

Innovus Sign-off and GDSII Export

File: `signoff.tcl`

14.1 What Sign-off Means

- This is the **Point of No Return** in the physical design flow.
- The sign-off script validates that the routed database is:
  - **physically manufacturable** — DRC-clean;
  - **electrically correct** — no opens or shorts;
  - **timing-safe** — zero negative slack at sign-off corners.
- It then generates the **GDSII file** that will be delivered to the foundry.

14.2 Standard Cell Filler Insertion

- After routing, **small gaps typically remain between logic cells** in the standard-cell rows.
- These gaps break the continuity of:
  - the **horizontal M1 power rails (VDD and VSS)** that run through each row, disconnecting some cells from their power supply;
  - the **N-Well and P-Well layers**, causing **OD density DRC violations**.
- **Standard-cell filler cells** (`feedth`, `feedth3`, `feedth9`) are inserted to bridge these gaps. They contain **no logic** but carry the necessary **well and M1 metal geometry** to restore continuity.
- The `-doDRC` flag ensures **no new DRC violations are introduced** by the filler insertion.

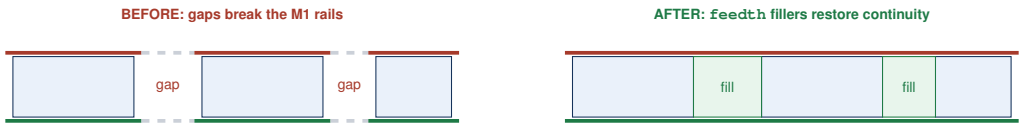


Figure 14.1: *(Explanatory diagram) Filler cells restore the broken M1 VDD/VSS rails and the N-Well/P-Well continuity along every standard-cell row.*

> Three filler widths are used (`feedth`, `feedth3`, `feedth9`) for exactly the same bin-packing reason as the `pfeed` IO fillers: a small set of widths can fill any residual gap with no leftover. Note `feedth9` appears here *and* as the end-cap cell — it is the widest member of the family and carries the well termination geometry.

14.3 GDSII Stream Out

- The Innovus internal database uses **LEF abstract views** for all standard cells and IO pads. These are **lightweight placeholders** that contain **only pin locations and routing blockages**.
- **The foundry cannot manufacture from abstracts**; it requires the **full transistor-level GDSII layout**.
- The `streamOut` command performs a **hierarchical merge**: it **replaces every LEF abstract** in the Innovus database with the corresponding **full GDSII layout** from the foundry-provided library files.
- A **layer map file** translates Innovus symbolic layer names (e.g. `Metal1`) into the **numeric GDSII layer/-datatype pairs** (e.g. Layer 46 Datatype 0) required by the foundry mask generation tools.



Figure 14.2: *(Explanatory diagram) What `streamOut` actually does: abstracts in, real polygons out.*

14.4 Sign-off Script (`signoff.tcl`)

```
#####################################################
# Innovus Sign-off Script
# Design: chip_top | Technology: SCL 180nm
#####################################################

## 1. RESTORE ROUTED DESIGN
restoreDesign /home/vlsi/Haze_free_silicon/STTP/pd/signoff/chip_top_routing.enc.dat chip_top

## 2. STANDARD CELL FILLER INSERTION
# feedth, feedth3, feedth9: SCL 180nm row-filling cells
# -doDRC: validate filler placement against DRC rules
addFiller -cell feedth feedth3 feedth9 -prefix FILLER -doDRC

## 3. PHYSICAL VERIFICATION

# A. Design Rule Check
set_verify_drc_mode -report chip_top.drc.rpt -limit 1000
verify_drc

# B. Connectivity (LVS-equivalent: Opens and Shorts)
verifyConnectivity -type all -error 1000

# C. Process Antenna Check
verifyProcessAntenna -report chip_top.antenna.rpt -error 1000

## 4. RC PARASITIC EXTRACTION
# effortLevel low: sufficient accuracy for 180nm, saves runtime
setExtractRCMode -effortLevel low
extractRC
rcOut -spef chip_top.spef

## 5. FINAL TIMING & POWER
timeDesign -postRoute
report_timing > timing_report.txt
summaryReport -outdir summaryReport

## 6. DESIGN EXPORTS (TAPE-OUT)

# A. Post-Layout Structural Netlist (for gate-level simulation)
saveNetlist chip_top_postlayout.v

# B. Standard Delay Format (annotated wire + gate delays)
write_sdf > chip_top_postlayout_sdf.sdf

# C. GDSII STREAM OUT
# -mapFile: translates layer names to GDS layer numbers
# -merge:   replaces LEF abstracts with full foundry GDS layouts
streamOut chip_final_gds.gds \
    -mapFile ../../../../../install/scl_pdk_v2/stdlib/fs120/tech_data/lef/gds2_fe_4l.map \
    -libName DesignLib \
    -merge {../../../../install/scl_pdk_v2/stdlib/fs120/gds/tsl18fs120.gds \
            ../../../../install/scl_pdk_v2/iolib/cio150/gds/tsl18cio150_4lm.gds} \
    -uniquifyCellNames -units 1000 -mode ALL

## 7. SAVE FINAL SESSION
saveDesign chip_top_signoff.enc

puts "#######################################"
puts " SIGN-OFF SCRIPT COMPLETED SUCCESSFULLY "
puts "#######################################"
```

14.4.1 Explanation of Each Sign-off Step

- `restoreDesign ...enc.dat chip_top` — loads the saved routed database. Sign-off always starts from a checkpoint, never from a live session, so the run is reproducible.
- `addFiller -doDRC` — row fill with built-in DRC validation (see above).

- `verify_drc` — geometric rule check inside Innovus. This is a **pre-check**; the **authoritative** DRC is run later in Calibre against the full merged GDS.
- `verifyConnectivity -type all` — opens and shorts; the LVS-equivalent check available inside the P&R tool.
- `verifyProcessAntenna` — confirms every net satisfies the foundry antenna ratio after the diodes were inserted during routing.
- `setExtractRCMode -effortLevel low` — at 180 nm the interconnect is simple enough that low effort gives sufficient accuracy; higher effort would only cost runtime.
- `extractRC` → `rcOut -spef` — writes the parasitics in **SPEF** format, which can be back-annotated into an external static timing tool (e.g. PrimeTime) for independent sign-off.
- `saveNetlist chip_top_postlayout.v` — the **post-layout** netlist. It differs from the synthesis netlist because it now contains all physical-only cells: fillers, end caps, antenna diodes, CTS buffers and hold-fix buffers.
- `write_sdf` — delays extracted from the *real* routed geometry, for the final gate-level simulation.
- `-uniquifyCellNames` — guarantees no two cells in the merged GDS share a name, which would silently corrupt the hierarchy.
- `-units 1000` — 1000 database units per micron, i.e. **1 nm resolution**.
- `-mode ALL` — stream out the complete hierarchy rather than the top level only.

Three files leave Innovus, and each has a different consumer:

- `chip_final_gds.gds` → Virtuoso / Calibre / the foundry (geometry).
- `chip_top_postlayout.v` → gate-level simulator and LVS (connectivity).
- `chip_top.spef` / `...sdf` → static timing and gate-level simulation (delay).

PART 15

Tape-out Preparation — Cadence Virtuoso & Calibre

- **Document type:** Standard Operating Procedure (SOP).
- **Process node:** SCL 180 nm (TS18SL). **Metal stack:** 4LM.
- **Applicable tools:** Cadence Virtuoso Layout GXL, Mentor Graphics Calibre DRC/LVS/PEX, Synopsys HSPICE.

15.1 Sign-off Philosophy

- **Physical Verification (PV)** is the bridge between **digital abstraction** and **physical silicon reality**.
- The foundry mask-generation tool processes a **database of geometric polygons**, not logic functions. Therefore, before tapeout, the GDSII database must be certified as:

1. **Manufacturable:** all polygons comply with the lithography design rules (verified by **DRC**).
2. **Electrically Connected:** the physical layout is electrically identical to the golden schematic (verified by **LVS**).
3. **Physically Complete:** mandatory structures have been inserted — **Seal Ring** (mechanical protection against dicing micro-cracks), **Silicon Number** (wafer traceability ID), and **Dummy Fill** (CMP planarity).

15.2 GDSII Stream-In (Merging Layouts in Virtuoso)

- The `streamOut` command from Innovus produced a GDSII file (`chip_final_gds.gds`) that **already contains merged standard-cell and IO-pad transistor-level layouts**.
- This file is imported into a Cadence Virtuoso library as the **starting point for chip finishing**.
- The Virtuoso stream-in configuration is:
  - **Target Library:** `SCL180_Tapeout_Lib` — a new Virtuoso library attached to the `tsl180` technology file, which ensures all layer definitions match the SCL 180 nm process.
  - **Technology File:** `tsl180.tf` — the master technology file that maps layer names to manufacturing layer numbers.
  - **Input GDS Files:** (1) Standard Cell GDS `tsl18fs120.gds`, (2) IO Pad GDS `tsl18cio150_4lm.gds`, (3) Digital Block GDS `chip_final_gds.gds` from Innovus.
  - **Layer Map File:** `gds2_fe_4l.map` — translates Innovus `Metal1`/`Metal2`/... names to GDS layer numbers.

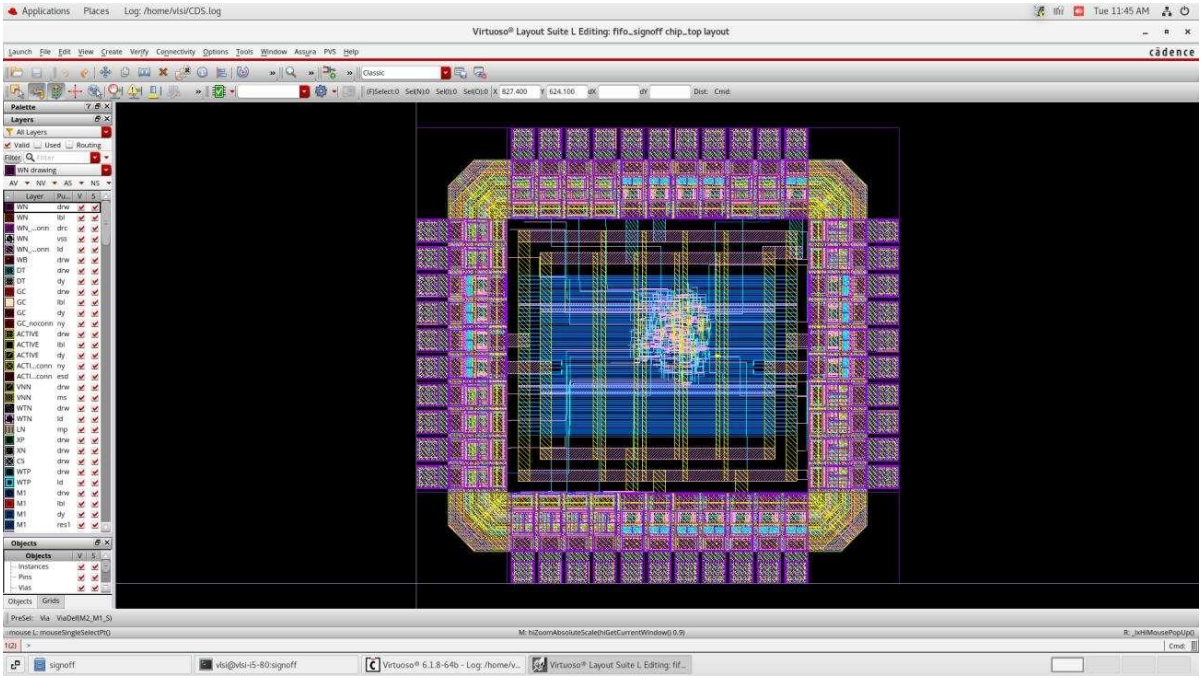


Figure 15.1: ***Figure 5.14 (original)*** *— Cadence Virtuoso post-stream-in layout: the full transistor-level view of* `chip_top`*. Every LEF abstract has been replaced by real polygons, so the diffusion, poly, contact and all four metal layers are visible.*

15.3 Chip Finishing Operations

15.3.1 Power and Ground Labeling

- LVS tools determine net connectivity by **matching geometry labels against the netlist**.
- Without explicit labels on the power pad metal, **LVS sees the pad metal as floating conductors** and reports **thousands of spurious open-circuit errors**.
- **Metal 4 (M4) labels must be placed on the *Pin* purpose layer** (not the *Drawing* layer), because Calibre's LVS deck recognises **only Pin-layer text** for connectivity extraction.

Procedure

1. In the Virtuoso **LSW (Layer Selection Window)**, select **Metal 4 — Pin**.
2. Press the `l` keybinding to invoke the **label creation dialog**.
3. Type the **exact net name used in the Verilog netlist**: `VDD_CORE`, `VSS_CORE`, `VDDO`, `VSSO`. **Case must match exactly.**
4. Click to place the **label origin inside the boundary** of the corresponding metal polygon. Label **all distinct power domains**.

> **Critical Warning**
> Do **NOT** place power labels on **M4 Drawing** purpose. Most Calibre LVS decks are configured to **ignore text on Drawing layers**, so the connection will appear **open** in the LVS report even though the label is **visually present** in the layout.

15.3.2 Silicon Number (Part ID) Insertion

- The silicon ID is a **hand-drawn polygon text in Metal 4** that identifies the chip on the wafer during post-fabrication inspection.

Procedure

1. Create a new layout cell named `silicon_logo` in Virtuoso.
2. Select **Metal 4 Drawing** and manually draw rectangular polygons to form the project identification text (e.g.

`SCL_V1`). The **stroke width of each letter must exceed the minimum M4 line width** of approximately **0.5 µm**.

3. Instantiate the `silicon_logo` cell into the `chip_top` top-level layout. Place it in an **empty corner of the die, away from active circuitry and dense routing** to prevent noise coupling.

> The logo is drawn on **Drawing** purpose (not Pin) precisely *because* it must **not** be seen by LVS — it is decoration, not connectivity. This is the exact opposite of the power-label rule above, and the two are easy to confuse.

15.3.3 Seal Ring Customisation and Insertion

- The **Seal Ring** is a **closed metal-and-via structure** surrounding the entire die perimeter.
- Its purpose is to **absorb micro-cracks generated during diamond-saw dicing** of the wafer, preventing crack propagation into the active circuit area.
- The foundry provides a **fixed-size Seal Ring GDS cell** (`SCL_SealRing_Fixed`) that must be **stretched to match the exact dimensions of the final die**.

Stretch Procedure in Virtuoso

1. **Import:** stream the `SCL_SealRing_Fixed` GDS into the library.
2. **Measure:** determine the exact X and Y die dimensions including the IO pad ring perimeter.
3. Open the Seal Ring cell in **Edit Mode** in Virtuoso.
4. Apply the **Stretch** command (keybinding: `s`). Drag a selection box covering the **entire Right Edge** of the ring, **capturing all layers**: M1, Via1, M2, Via2, M3, Via3, M4, Contacts, P-Diffusion. **If any via layer is missed, the ring becomes electrically open.**
5. Move the edge to the right until the width matches the target die X-dimension.
6. Repeat the stretch operation on the **Top Edge** to set the Y-dimension.
7. Instantiate the customised seal ring cell into the `chip_top` layout.
8. **Spacing Check:** use the **Ruler** (keybinding: `k`) to verify that the distance between the **inner wall of the Seal Ring** and the **outer edge of the IO pads** is **greater than 10 µm**, as required by SCL design guidelines.

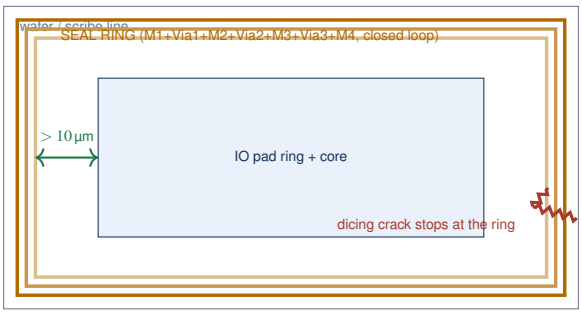


Figure 15.2: *(Explanatory diagram) The seal ring is a continuous metal/via wall around the die. A crack from the dicing saw propagates along the brittle dielectric until it meets the ductile metal stack, where it is arrested — which is why* every *via layer must be captured during the stretch.*

> **If skipped:** Miss one via layer during the stretch ⇒ the seal ring has an electrical and mechanical discontinuity at that point. LVS will flag it as a floating conductor, and more seriously, a dicing crack can walk straight through the gap into the active area, killing the die after packaging.

PART 16

Density Compliance — Dummy Fill Generation

Tool: Mentor Graphics Calibre

16.1 CMP Planarity and Dummy Fill

- **Chemical Mechanical Polishing (CMP)** is a fabrication step used **after each metal deposition layer** to **planarize the silicon wafer surface**.
- CMP removes metal from raised areas while leaving metal in recessed areas — but its effectiveness **depends on having a locally uniform density of metal features**.
- In regions where there is **very little metal (open areas)**, the CMP polishing pads **press down harder** and **remove too much metal** from the narrow features nearby. This phenomenon is called **dishing**.
- To prevent dishing and ensure uniform post-CMP thickness across the wafer, the foundry mandates that **metal density on every layer be kept within a specified range (typically 25 %–75 %) across every** $100\,\mu\text{m} \times 100\,\mu\text{m}$ **window**.
- **Dummy fill cells** are **small floating metal squares** (not connected to any signal) inserted into empty areas to bring the local density into compliance.
- They must be placed with **sufficient spacing from active routing** to avoid coupling capacitance that would degrade signal integrity.

Density rule

$$\text{Density} = \frac{\text{metal area inside the window}}{\text{window area}}, \qquad 0.25 \leq \text{Density} \leq 0.75 \;\text{ in every } 100\,\mu\text{m} \times 100\,\mu\text{m window}$$

The window **slides** — it is not a fixed grid — so a design can pass on average and still fail locally.

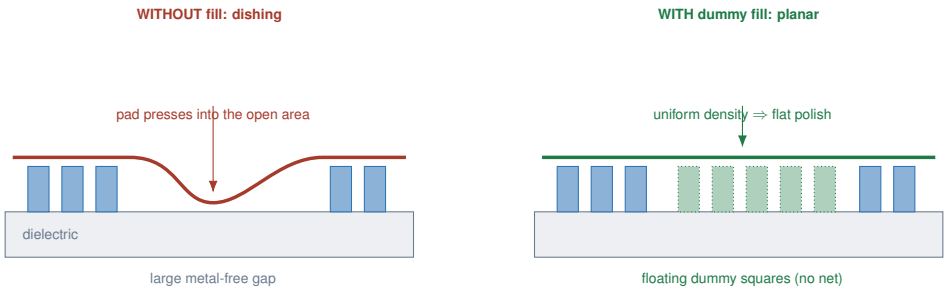


Figure 16.1: *(Explanatory diagram) Why dummy fill exists. A metal-free region lets the polishing pad flex into the gap and over-polish the real metal at the edges of that region.*

16.2 Dummy Generation Script (`run_dummy.svrf`)

```
// ============================================================
// SCL 180nm DUMMY GENERATION CONTROL FILE (Calibre)
// ============================================================

// 1. INPUT DEFINITIONS
LAYOUT SYSTEM GDSII
LAYOUT PATH    "chip_top_finished.gds"   // Layout with Seal Ring & Logo
LAYOUT PRIMARY "chip_top"
PRECISION  1000
RESOLUTION 5

// 2. OUTPUT DEFINITIONS
// Separate GDS containing ONLY dummy fill -- keeps original design clean
DRC RESULTS DATABASE "chip_top_dummies.gds" GDSII
```

```
DRC SUMMARY REPORT   "dummy_fill.rpt"

// 3. PROCESS CONFIGURATION
#define 4LM            // 4-Metal stack

// 4. GENERATION SWITCHES (enable layers requiring fill)
#define AA_gen         // Active Area Dummies
#define GC_gen         // Poly Gate Dummies
#define M1_gen         // Metal 1 Dummies
#define M2_gen         // Metal 2 Dummies
#define M3_gen         // Metal 3 Dummies
#define M4_gen         // Metal 4 Dummies
// #define MT_gen      // Top Metal (excluded if used for logo/bonding)

// 5. EXECUTION -- include foundry's mathematical fill algorithm
INCLUDE "/home/install/scl_pdk_v2/design_kit/drclvs_rsf/DUMMYFILL_TS18SL_SCL_CALIBRE"
```

16.2.1 Line-by-Line Explanation

- `LAYOUT PATH "chip_top_finished.gds"` — the **input** is the layout *after* the seal ring and silicon logo were added, so the fill algorithm sees the final geometry.
- `PRECISION 1000` — 1000 database units per micron (1 nm grid), matching the `streamOut -units 1000` setting.
- `RESOLUTION 5` — geometry smaller than 5 database units is ignored.
- `DRC RESULTS DATABASE "chip_top_dummies.gds"` — the fill is written to a **separate GDS file**, **not** merged into the design. This keeps the original design database clean and makes it trivial to regenerate the fill if the layout changes.
- `#define 4LM` — selects the 4-metal rule set inside the foundry deck.
- `#define AA_gen ... #define M4_gen` — enable fill generation layer by layer. **Active Area** and **Poly** also need fill, not just metal, because they are polished too.
- `// #define MT_gen` — **commented out**. Top Metal fill is **excluded** because TOP_M is used for the silicon logo and for bonding pads; dummy squares there would interfere with wire bonding and visual inspection.
- `INCLUDE ".../DUMMYFILL_TS18SL_SCL_CALIBRE"` — the **foundry's own fill algorithm**. The engineer supplies the switches; the foundry supplies the mathematics. Never hand-write fill rules.

16.3 Final Database Merge

- After dummy generation, **three separate GDSII files** are merged into a **single tape-out file** using `calibredrv`.

```
calibredrv -a layout filemerge \
    -in chip_top_finished.gds \
    -in chip_top_dummies.gds \
    -out chip_top_tapeout_merged.gds
```

- `-a layout filemerge` — invoke the layout file-merge application.
- Each `-in` adds one source database; the inputs are overlaid in the order given.
- `-out chip_top_tapeout_merged.gds` — this is the **single file that every subsequent check must run on**, and ultimately the file shipped to the foundry.

> **Keep design and fill in separate source files.** If the routing changes, only `chip_top_dummies.gds` needs regenerating — the design database is never polluted by thousands of floating squares, and version control stays readable.

PART 17

Final Sign-off — DRC and LVS with Calibre

17.1 Final Design Rule Check (DRC)

File: `final_drc.svrf`

- The final DRC run verifies the **merged database** — which now includes the **stretched Seal Ring**, the **Silicon ID**, and the **dummy fill patterns** — against the **complete SCL 180 nm design rule deck**.
- The **CHIP switch is mandatory** because it enables rules specifically applicable to the **full-chip context** (IO pad spacing from core, seal-ring clearances, etc.) that are **not checked during block-level DRC runs**.

```
// ============================================================
// SCL 180nm FINAL DRC SIGN-OFF (Calibre)
// ============================================================
LAYOUT SYSTEM GDSII
LAYOUT PATH    "chip_top_tapeout_merged.gds"
LAYOUT PRIMARY "chip_top"

DRC RESULTS DATABASE "chip_top_drc.db" ASCII
DRC SUMMARY REPORT   "chip_top_drc.rpt" REPLACE HIER
DRC MAXIMUM RESULTS  ALL
DRC CELL NAME YES CELL SPACE XFORM

#define 4LM
#define CHIP        // Enable full-chip rules (Pads + Seal Ring)

INCLUDE "/home/install/scl_pdk_v2/design_kit/drclvs_rsf/DRC_TS18SL_SCL_CALIBRE"
```

17.1.1 Explanation

- `LAYOUT PATH "chip_top_tapeout_merged.gds"` — the check runs on the **exact file being taped out**, including seal ring, logo and fill.
- `DRC RESULTS DATABASE ... ASCII` — results in a readable database that can be loaded back into Virtuoso to highlight each violation in the layout.
- `REPLACE HIER` — overwrite the previous report and preserve the **hierarchy** of violations, so one repeated cell-level error is reported once rather than thousands of times.
- `DRC MAXIMUM RESULTS ALL` — do **not** truncate. At final sign-off every violation must be visible; truncation can hide a systematic problem.
- `DRC CELL NAME YES CELL SPACE XFORM` — report the offending cell name and its placement transform, which is what makes a violation findable in a large hierarchy.
- `#define CHIP` — enables full-chip rules. **Forgetting this is a classic tape-out error**: the run reports zero errors simply because the chip-level rules were never executed.

> **Acceptance criterion:** **ZERO** DRC errors. A zero-error DRC is a **hard foundry requirement** before mask generation — there is no "waiver by default".

17.2 Final Layout-Versus-Schematic (LVS)

File: `final_lvs.svrf`

- LVS **extracts the netlist from the GDSII geometry** and **compares it against the source netlist** (`chip_top_lvs.sp`).
- The most critical checks at this stage are:
  - **Seal Ring grounding** — the seal ring must be connected to **VSS/VSSO**, not left floating;
  - the **correctness of the power label assignments** placed earlier in Virtuoso.
- **Virtual connections** are used to allow Calibre to **bridge equivalent net names** that differ between the

layout and the schematic due to **library naming conventions**.

```
// ============================================================
// SCL 180nm FINAL LVS SIGN-OFF (Calibre)
// ============================================================

// 1. LAYOUT INPUT
LAYOUT SYSTEM GDSII
LAYOUT PATH    "chip_top_tapeout_merged.gds"
LAYOUT PRIMARY "chip_top"
LAYOUT CASE YES

// 2. SOURCE NETLIST
SOURCE PATH    "chip_top_lvs.sp"
SOURCE PRIMARY "chip_top"
SOURCE SYSTEM SPICE
SOURCE CASE YES

// 3. REPORTING
LVS REPORT        "chip_top_lvs.rpt"
LVS REPORT OPTION S
MASK RESULTS DATABASE NONE
DRC ICSTATION YES

// 4. POWER MAPPING
LVS POWER NAME  VDD_CORE VDDO_CORE VDDO VDD VCC
LVS GROUND NAME VSS_CORE VSSO_CORE VSSO VSS GND

// 5. VIRTUAL CONNECTIONS (name aliasing between layout and schematic)
VIRTUAL CONNECT NAME VDD_CORE VDD
VIRTUAL CONNECT NAME VSS_CORE VSS
VIRTUAL CONNECT NAME VSS_CORE VSSO_CORE   // Seal ring substrate tie

// 6. SWITCHES
#define 4LM
// #define ANALOG  -- keep commented to avoid resistor errors in IO cells

INCLUDE "/home/install/scl_pdk_v2/design_kit/drclvs_rsf/LVS_TS18SL_SCL_CALIBRE"
```

17.2.1 Explanation

- `LAYOUT CASE YES` / `SOURCE CASE YES` — net names are **case-sensitive** on both sides. This is why the Virtuoso labelling step insisted that **case must match exactly**.
- `SOURCE SYSTEM SPICE` — the golden netlist is in SPICE format (`chip_top_lvs.sp`), the standard LVS source format.
- `LVS REPORT OPTION S` — include a **summary of the comparison statistics** (device counts, net counts, discrepancy list).
- `LVS POWER NAME` / `LVS GROUND NAME` — declares **every alias** by which a supply may be known. Listing all of them lets Calibre recognise a supply regardless of which naming convention a given cell library used.
- `VIRTUAL CONNECT NAME VSS_CORE VSSO_CORE` — ties the **core ground** and the **IO ground** together **for the purposes of the check**, because physically they share the substrate. This is the line that **grounds the seal ring** in the comparison.
- `// #define ANALOG` — deliberately **commented out**. In analog mode Calibre extracts parasitic resistors inside the IO cells, which then **do not exist in the digital source netlist** and are reported as mismatches.

> **Acceptance criterion:** the report must read **CORRECT** — netlist and layout match exactly. **No opens, no shorts, no parameter mismatches.**

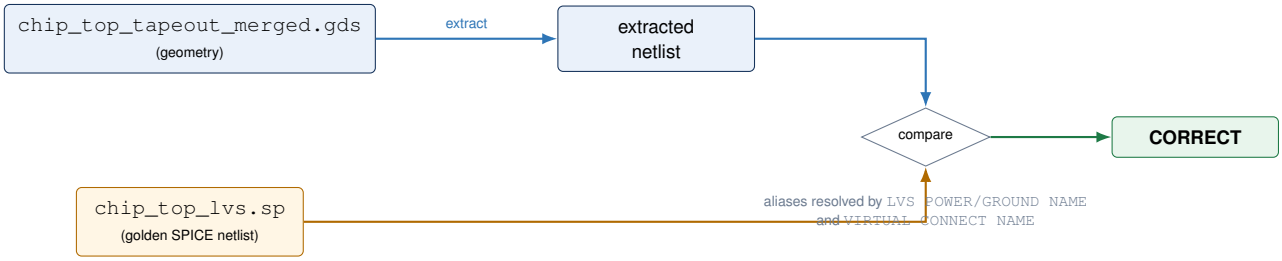


Figure 17.1: *(Explanatory diagram) LVS does not "check the layout" — it rebuilds a netlist from polygons and proves it is isomorphic to the golden netlist.*

PART 18

Post-Layout Simulation

18.1 Parasitic Extraction (PEX)

File: `run_pex.svrf`

- Post-layout simulation requires a **SPICE netlist that includes the parasitic resistance (R) and capacitance (C) of every metal wire and via** in the layout.
- **Mentor Graphics Calibre PEX** (Parasitic EXtraction) reads the **GDSII geometry** and the **LVS source netlist** and produces an **RC-annotated SPICE netlist** that can be simulated with HSPICE.
- `PEX EXTRACT RESISTOR YES` and `PEX EXTRACT CAPACITOR ALL` ensure **all parasitics are captured**.

```
// ============================================================
// SCL 180nm PARASITIC EXTRACTION (Calibre PEX)
// ============================================================
LAYOUT SYSTEM GDSII
LAYOUT PATH    "chip_top_tapeout_merged.gds"
LAYOUT PRIMARY "chip_top"

SOURCE PATH    "chip_top_lvs.sp"
SOURCE PRIMARY "chip_top"
SOURCE SYSTEM SPICE

// OUTPUT CONFIGURATION
PEX REDUCTION ANALOG NO
PEX EXTRACT RESISTOR YES
PEX EXTRACT CAPACITOR ALL
PEX NETLIST "chip_top_postlayout.pex.netlist" HSPICE 1

// POWER MAPPING
LVS POWER NAME  VDD_CORE VDDO_CORE VDD
LVS GROUND NAME VSS_CORE VSSO_CORE VSS
VIRTUAL CONNECT NAME VDD_CORE VDD
VIRTUAL CONNECT NAME VSS_CORE VSS
VIRTUAL CONNECT NAME VSS_CORE VSSO_CORE

INCLUDE "/home/install/scl_pdk_v2/design_kit/drclvs_rsf/xRC_TS18SL_SCL_CALIBRE"
```

Execution command:

```
calibre -xrc -fmt -all run_pex.svrf
```

18.1.1 Explanation

- **PEX reuses the LVS inputs.** It needs both the geometry (to measure the parasitics) and the source netlist (to know which device each extracted element belongs to). That is why the power mapping and virtual connect lines are repeated verbatim from the LVS deck.
- `PEX REDUCTION ANALOG NO` — disables the analog network reduction that would merge small RC elements. For a digital block, no reduction gives the most faithful netlist.
- `PEX EXTRACT RESISTOR YES` — extract **wire resistance**, not only capacitance. At 180 nm with long power rails, R is as important as C for IR drop and delay.
- `PEX EXTRACT CAPACITOR ALL` — extracts **ground capacitance and coupling capacitance** between neighbouring wires, which is what makes crosstalk visible in simulation.
- `PEX NETLIST "..." HSPICE 1` — output format is HSPICE, format version 1.
- `calibre -xrc -fmt -all` — `-xrc` runs the extraction engine; `-fmt -all` runs all formatting phases (PHDB → PDB → netlist) in one invocation.

Why post-layout differs from post-synthesis

$$T_{delay} = T_{cell} + T_{wire}, \qquad T_{wire} \approx 0.69\, R_{wire}\, C_{load} \quad \text{(Elmore)}$$

Synthesis *estimated* R and C from a wire-load model. PEX *measures* them from the real polygons. This is the first point in the whole flow where the delay numbers are physically true.

18.2 Post-Layout SPICE Simulation Testbench

File: `tb_simulation.sp`

- The post-layout testbench **includes the RC-annotated extracted netlist** and applies the **same stimulus used in the RTL simulation**.
- The key difference is that **power supply voltage levels now reflect the dual-domain architecture** (1.8 V core, 3.3 V IO).
- The `.MEASURE` statement captures **average power consumption**, which can be compared against the power budgets established during synthesis.

```
*************************************************************
* SCL 180nm Post-Layout Simulation Testbench (HSPICE)
*************************************************************
.TITLE Final Sign-off Simulation
.OPTION POST=2 PROBE

*** 1. Technology Model Libraries (Typical Corner)
.LIB "/home/install/scl_pdk_v2/models/hspice/scl180_mos.lib" tt
.LIB "/home/install/scl_pdk_v2/models/hspice/scl180_res.lib" tt
.LIB "/home/install/scl_pdk_v2/models/hspice/scl180_cap.lib" tt

*** 2. Include Extracted RC Netlist
.INCLUDE "chip_top_postlayout.pex.netlist"

*** 3. Power Supplies
V_VDD_CORE VDD_CORE 0 DC 1.8V
V_VSS_CORE VSS_CORE 0 DC 0V
V_VDDO     VDDO     0 DC 3.3V
V_VSSO     VSSO     0 DC 0V

*** 4. Device Under Test (DUT)
X_DUT
+ clk rst_n din[0] dout[0]
+ VDD_CORE VSS_CORE VDDO VSSO
+ chip_top

*** 5. Stimulus
V_clk clk 0 PULSE(0 1.8 0 0.1n 0.1n 10n 20n)    ;* 100 MHz clock
V_rst rst_n 0 PWL(0 0 50n 0 55n 1.8)            ;* Reset deassert at 55ns

*** 6. Analysis
.TRAN 0.1n 1000n
.MEASURE TRAN avg_power AVG P(V_VDD_CORE)
.END
```

Execution command:

```
hspice tb_simulation.sp > sim.log
```

18.2.1 Line-by-Line Explanation

- `.OPTION POST=2 PROBE` — write waveform output in binary format (`POST=2`) and only save the signals explicitly probed, keeping the output file manageable.
- **Three model libraries** are included because the extracted netlist contains three kinds of device: **MOS transistors**, **parasitic resistors** and **parasitic capacitors**. The `tt` argument selects the **typical-typical** corner.
- `.INCLUDE "chip_top_postlayout.pex.netlist"` — pulls in the extracted netlist produced by PEX.
- **Four supplies** are declared, matching the **dual voltage domain**: 1.8 V core and 3.3 V IO. This is the point at which the dual-domain design is actually validated electrically.

- `X_DUT ... chip_top` — a subcircuit instantiation; the continuation lines (+) list the port connections, ending with the subcircuit name.
- `PULSE(0 1.8 0 0.1n 0.1n 10n 20n)` — the clock source. Arguments in order: **V1 = 0 V**, **V2 = 1.8 V**, **delay = 0**, **rise = 0.1 ns**, **fall = 0.1 ns**, **pulse width = 10 ns**, **period = 20 ns**. **Note:** a 20 ns period with a 10 ns high time gives a 50 % duty cycle — consistent with the SDC `-waveform {0 5}` if the SDC time unit is taken as the half-period reference.
- `PWL(0 0 50n 0 55n 1.8)` — piecewise-linear reset: held at 0 V until 50 ns, then ramped to 1.8 V by 55 ns, i.e. **reset deasserts at 55 ns**.
- `.TRAN 0.1n 1000n` — transient analysis, 0.1 ns step, 1000 ns total.
- `.MEASURE TRAN avg_power AVG P(V_VDD_CORE)` — the measured average power drawn from the core supply. This number is compared against the `report_power` estimate from synthesis; a large discrepancy means either the switching-activity assumption or the parasitic model was wrong.

Power check

$$P_{total} = \underbrace{\alpha C_L V_{DD}^2 f}_{\text{dynamic}} + \underbrace{I_{SC} V_{DD}}_{\text{short-circuit}} + \underbrace{I_{leak} V_{DD}}_{\text{static}}$$

At 180 nm the **dynamic** term dominates. Since $P \propto V_{DD}^2$, the 3.3 V IO ring can consume more power than the entire 1.8 V core despite having far fewer transistors — which is exactly why the supplies are measured separately.

PART 19

Foundry Deliverables Checklist

- The following items constitute the **complete tape-out package** transmitted to the SCL foundry for mask generation and fabrication.
- **Every item must be verified before the package is sent.**
- Once the tape-out package is accepted by the foundry, **no further design changes are possible** without incurring a **mask respin** at significant cost and schedule delay.

Item	File Name	Acceptance Criteria
Tape-out GDSII	`chip_top_tapeout_merged.gds`	Must include: Block + IO Pads + Silicon ID + Seal Ring + Dummy Fill. File size and layer count must match foundry specification.
DRC Report	`chip_top_drc.rpt`	Must show **ZERO** DRC errors. Zero-error DRC is a hard foundry requirement before mask generation.
LVS Report	`chip_top_lvs.rpt`	Must show **CORRECT** — netlist and layout match exactly. No opens, no shorts, no parameter mismatches.
Bonding Diagram	`chip_top_bonding.pdf`	PDF showing all pad coordinates (X,Y in µm from origin), pin names, and signal assignments. Required for wire-bonding machine programming.
Source Netlist	`chip_top_lvs.sp`	Golden SPICE netlist used for LVS. Must be consistent with the GDSII submitted for verification.

Tape-out Best Practice
Always run a **final end-to-end DRC and LVS from the exact GDSII file being submitted** for tape-out — **not** from an intermediate design database. Run these checks on the complete merged file (`chip_top_tapeout_merged.gds`) that includes all finishing structures. **Any manual edits performed after the last DRC run may introduce new violations.**

19.1 Why Each Deliverable Exists

- **GDSII** — the only file the mask writer actually consumes. Everything else is evidence that this file is trustworthy.
- **DRC report** — proves the geometry is **printable**. A single sub-minimum spacing can short two nets across an entire wafer lot.
- **LVS report** — proves the geometry is **the right circuit**. DRC-clean silicon that implements the wrong netlist is still scrap.
- **Bonding diagram** — the **assembly house**, not the fab, needs this. Without exact pad coordinates the wire-bonding machine cannot be programmed.
- **Source netlist** — the reference against which LVS was run. The foundry may re-run LVS independently and needs the same golden source.

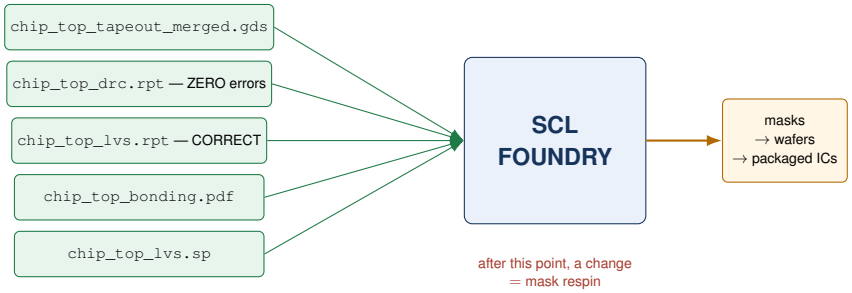


Figure 19.1: *(Explanatory diagram) The five deliverables and the point of no return.*

PART 20

Chapter Summary

- This flow presented a complete, **industry-grade RTL-to-GDSII design flow** for an **8-bit synchronous FIFO** implemented in **SCL 180 nm** process technology.
- The major milestones are summarised below.

Stage	Tool / File	Key Deliverable
RTL Design	`fifo_8bit.v` (Verilog)	Parameterized 8-bit FIFO with N+1 pointer method
Chip Integration	`chip_top.v`	Pad-ring wrapper with SCL IO cells (`pc3d01`, `pt3t03u`)
Functional Verification	`tb_fifo_8bit.v`	Simulated burst write/read; FIFO flags verified
Timing Constraints	`chip_top_final.sdc`	10 ns clock at `PAD_CLK`; I/O delays; DRC constraints
Logic Synthesis	Cadence Genus	Mapped netlist; QoR reports; SDF/SDC/HDL export
Power Pad Insertion	`chip_top_final.v`	$5 \times$ {`pvdi`, `pv0i`, `pvda`, `pv0a`} for balanced ring
MMMC Setup	`chip_top.view`	SS corner (setup) + FF corner (hold)
IO Pad Placement	`chip_top.io`	Corner cells + signal + power pads, clockwise order
Floorplanning	`floorplan.tcl` (Innovus)	Square core, 70 % utilisation, 150 µm pad margins
Power Planning	`power_plan.tcl`	30 µm rings (M3/TOP_M); stripes; SRoute
Physical Cells	`physical_placement.tcl`	End caps (`feedth9`); IO fillers (`pfeed` family)
Standard Cell Placement	`placement.tcl`	`place_opt_design`; congestion + timing analysis
Clock Tree Synthesis	`cts.tcl` (CCOpt)	< 0.3 ns skew; < 0.5 ns transition; propagated timing
Routing	`routing.tcl` (NanoRoute)	Antenna diode insertion; SI-driven; OCV/CPPR signoff
Innovus Sign-off	`signoff.tcl`	Filler insertion; DRC; antenna; GDSII stream-out
Virtuoso Tape-out	Manual + Calibre scripts	GDS stream-in; P/G labels; Silicon ID; Seal Ring
Dummy Fill	`run_dummy.svrf` (Calibre)	AA/GC/M1–M4 fill; merged GDS
Final DRC	`final_drc.svrf`	Zero violations on merged tape-out GDS
Final LVS	`final_lvs.svrf`	CORRECT match; virtual connections; seal ring ground
Post-Layout Simulation	`run_pex.svrf` + `tb_simulation.sp`	RC parasitics extracted; power measured via HSPICE

20.1 Generality of the Flow

- The flow described is **directly applicable to any digital ASIC** targeting a standard **single-supply or dual-supply CMOS process node**.
- The scripts, constraint files and verification decks have been presented in their **complete, executable form** so they can be adapted and deployed in a laboratory or industrial setting.
- The **key principles** apply broadly to both **mature nodes (180 nm, 130 nm)** and **advanced processes (28 nm, 7 nm)**:
  - **MMMC timing closure**;
  - **hierarchical power delivery**;
  - **CCOpt clock distribution**;

  - **DRC/LVS-based sign-off**.
- The **primary differences** at advanced nodes are **tool parameter tuning** and the **complexity of the design rule deck**.

> **What changes below 28 nm (context beyond this flow):** double/multi-patterning colouring rules, FinFET discrete device sizing, placement on a unidirectional routing grid, signal electromigration limits on signal nets (not just power), and statistical (POCV) timing instead of flat OCV derates. The *shape* of the flow in this note is unchanged; the rule decks and the number of corners grow dramatically.

PART 21

Bibliography

The following sources form the technical and scholarly foundation for this RTL-to-GDSII design flow. Entries follow the IEEE reference format.

Textbooks & Monographs

[1] N. H. E. Weste and D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Pearson / Addison-Wesley, 2010.

[2] J. M. Rabaey, A. Chandrakasan, and B. Nikolic, *Digital Integrated Circuits: A Design Perspective*, 2nd ed. Prentice Hall, 2003.

[3] P. P. Chu, *RTL Hardware Design Using VHDL: Coding for Efficiency, Portability, and Scalability*. Wiley-IEEE Press, 2006.

[4] S.-M. Kang and Y. Leblebici, *CMOS Digital Integrated Circuits: Analysis and Design*, 3rd ed. McGraw-Hill, 2003.

[5] M. Keating and P. Bricaud, *Reuse Methodology Manual for System-on-a-Chip Designs*, 3rd ed. Springer, 2002.

[6] R. J. Baker, *CMOS: Circuit Design, Layout, and Simulation*, 3rd ed. Wiley-IEEE Press, 2010.

EDA Tool Documentation

[7] Cadence Design Systems, *Genus Synthesis Solution User Guide*, Version 21.1. Cadence Design Systems, Inc., San Jose, CA, 2021.

[8] Cadence Design Systems, *Innovus Implementation System User Guide*, Version 21.1. Cadence Design Systems, Inc., San Jose, CA, 2021.

[9] Cadence Design Systems, *Virtuoso Layout Suite User Guide*, Version 6.1.8. Cadence Design Systems, Inc., San Jose, CA, 2020.

[10] Siemens EDA (Mentor Graphics), *Calibre Design Solutions User's Manual: nmDRC and nmLVS*. Siemens EDA, Wilsonville, OR, 2022.

[11] Siemens EDA (Mentor Graphics), *Calibre xRC Parasitic Extraction User's Manual*. Siemens EDA, Wilsonville, OR, 2022.

Process Technology & PDK

[12] Semiconductor Laboratory (SCL), *SCL 180 nm CMOS Process Design Kit (PDK) v2: Documentation and Design Rules*. Society for Integrated Circuit Technology and Applied Research (SITAR), Chandigarh, India, 2019.

[13] Semiconductor Laboratory (SCL), *SCL Standard Cell Library tsl18fs120 and IO Library tsl18cio150: Liberty and LEF Reference Manual*. SITAR, Chandigarh, India, 2019.

IEEE & Industry Standards

[14] *IEEE Standard for Verilog Hardware Description Language*, IEEE Std 1364-2005. IEEE, New York, NY, 2006. doi: 10.1109/IEEESTD.2006.99495.

[15] *IEEE Standard for Standard Delay Format (SDF) for the Electronic Design Process*, IEEE Std 1497-2001. IEEE, New York, NY, 2001.

[16] Synopsys Inc., *Design Constraints (SDC) Command Reference Manual*. Synopsys, Mountain View, CA, 2020.

[17] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu, *VLSI Physical Design: From Graph Partitioning to Timing Closure*. Springer, 2011.

[18] J. Lienig and H. Bruemmer, *Fundamentals of Electronic Systems Design*. Springer, 2017.

[19] Open Verilog International (OVI), *Standard Parasitic Extraction Format (SPEF)*, Version 1.0. OVI, San Jose, CA, 2001.

Additional References

[20] S. Golshan, *Physical Design Essentials: An ASIC Design Implementation Perspective*. Springer, 2007.

[21] W. Wolf, *FPGA-Based System Design*. Prentice Hall, 2004.

[22] JEDEC Solid State Technology Association, *FIFO Memory Terminology and Specifications*, JESD22-B111. JEDEC, Arlington, VA, 2003.

APPENDIX A

Quick Revision Sheet — All Numbers and Formulas

A.1 Every Numeric Target in One Place

Quantity	Value	Where it appears
Clock period / frequency	10 ns / 100 MHz	SDC `create_clock`, testbench `#5`
Clock waveform (duty)	{0 5} = 50 %	SDC Section 1
Clock source latency (early/late)	0.30 / 0.50 ns	SDC Section 2
Pre-CTS network latency	0.80 / 1.20 ns	SDC Section 3
Clock uncertainty (setup/hold)	0.20 / 0.10 ns	SDC Section 4
Input delay (min/max)	0.50 / 2.50 ns	SDC Section 5
Output delay (min/max)	0.40 / 2.00 ns	SDC Section 6
Output load	5.0 (pF)	SDC Section 8
Clock transition at pad	0.10 ns	SDC Section 9
Max transition (data)	1.50 ns	SDC Section 10
Max fanout	16	SDC Section 10
Core voltage	1.8 V	dual-domain architecture
IO voltage	3.3 V	dual-domain architecture
SS corner voltage	1.68 V (−7 %)	MMMC setup corner
FF corner voltage	1.98 V (+10 %)	MMMC hold corner
Temperature extremes	−40 / +125 °C	MMMC
Aspect ratio	1.0 (square)	`floorPlan`
Core utilisation	0.7 (70 %)	`floorPlan`, `max_density`
Core margins	150 µm × 4 sides	`floorPlan`
Power ring width	30 µm	`addRing`
Ring spacing / offset	30 / 10 µm	`addRing`
Vertical stripe (TOP_M)	11.2 µm wide, 112 µm pitch	`addStripe`
Horizontal stripe (M3)	5.6 µm wide, 224 µm pitch	`addStripe`
Max distance to a stripe	56 µm (= 112/2)	PDN requirement
Stripe start offsets	84.28 / 120.4 µm	`addStripe`
IO pad spacing	10 µm	`chip_top.io space`
Total edges in `.io`	18	`chip_top.io`
CTS target skew	0.3 ns (300 ps)	`target_skew`
CTS max clock transition	0.5 ns (500 ps)	`target_max_trans`
Buffer drive range (CTS)	1× to 7×	`buffer_cells`
Post-route hold margin	0.05 ns (50 ps)	`holdTargetSlack`
Post-route setup target	0.0 ns	`setupTargetSlack`
Congestion danger threshold	> 2 % overflow	placement analysis
Metal density window	100 × 100 µm	dummy fill
Metal density range	25 %–75 %	dummy fill
GDS database units	1000 per µm (1 nm)	`streamOut -units`

Quantity	Value	Where it appears
Calibre precision / resolution	1000 / 5	`run_dummy.svrf`
Min M4 logo stroke width	≈0.5 µm	silicon ID
Seal ring to pad clearance	> 10 µm	seal ring check
SPICE transient	0.1 ns step, 1000 ns total	`.TRAN`
SPICE reset deassert	55 ns	`PWL`

A.2 Every Formula in One Place

FIFO

$$\text{ADDR_WIDTH} = \log_2(\text{DEPTH}); \qquad \text{pointer width} = \text{ADDR_WIDTH} + 1$$

$$\texttt{empty} = (\texttt{wr_ptr} = \texttt{rd_ptr}); \quad \texttt{full} = (\texttt{wr_ptr}[4] \neq \texttt{rd_ptr}[4]) \wedge (\texttt{wr_ptr}[3{:}0] = \texttt{rd_ptr}[3{:}0])$$

Timing

$$T_{cq} + T_{comb} + T_{setup} + T_{uncert} \leq T_{clk} + T_{skew} \qquad \text{(setup)}$$

$$T_{cq} + T_{comb} \geq T_{hold} + T_{skew} + T_{uncert,hold} \qquad \text{(hold)}$$

$$\text{Slack} = T_{required} - T_{arrival}; \quad \text{WNS} = \min(\text{slack}); \quad \text{TNS} = \textstyle\sum_{\text{slack}<0} \text{slack}$$

$$T_{wire} \approx 0.69\,RC \qquad \text{(Elmore)}$$

Physical

$$\text{Utilization} = \frac{\sum \text{cell area}}{\text{core area}}; \qquad \text{Aspect ratio} = \frac{H}{W}$$

$$\text{Overflow} = \frac{\text{routing demand}}{\text{routing supply}}$$

$$\text{Density} = \frac{\text{metal area}}{\text{window area}}, \quad 0.25 \leq \text{Density} \leq 0.75$$

Reliability and noise

$$\Delta V_{IR} = I \cdot R = I R_{sheet} \frac{L}{W}$$

$$V_{gb} = L \frac{di}{dt} \qquad \text{(ground bounce / SSO)}$$

$$\text{MTTF} = A J^{-n} \exp\left(\frac{E_a}{kT}\right), \; n \approx 2 \qquad \text{(Black's equation)}$$

$$\text{Antenna ratio} = \frac{A_{metal}}{A_{gate}}$$

$$\Delta V_{xtalk} = V_{DD} \frac{C_c}{C_c + C_g}$$

Power

$$P = \alpha C V_{DD}^2 f + I_{SC} V_{DD} + I_{leak} V_{DD}$$

APPENDIX B

Master Command Reference

B.1 Cadence Genus (Synthesis)

Command	Function
`set_db hdl_max_memory_address_range`	Raise elaboration memory limit for large arrays
`set_db library`	Load Liberty (.lib) timing/power/area models
`set_db lef_library`	Load LEF physical models (enables PLE)
`set_db syn_*_effort high`	Maximum optimisation effort
`read_hdl -language sv`	Read RTL source files
`elaborate <top>`	Build internal Boolean network (AIG)
`check_design -all`	Detect undriven / multiply-driven nets
`read_sdc`	Load timing constraints
`syn_generic`	Technology-independent Boolean optimisation
`syn_map`	Map generic gates to SCL 180 nm cells
`syn_opt`	Gate sizing, buffering, logic restructuring
`report_area / report_power / report_timing / report_qor`	QoR reports (acceptance: WNS = TNS = 0)
`write_hdl / write_sdc / write_sdf`	Export netlist / constraints / delays

B.2 Cadence Innovus (Physical Implementation)

Command	Function
`setDesignMode -process 180`	Load process-specific rules
`add_tracks -honor_pitch`	Build routing grid aligned to cell pin pitch
`checkUnique / check_timing`	Pre-floorplan integrity checks
`floorPlan -site -r AR Util M M M M`	Create die and core
`reportGateCount / reportNetStat`	Gate and net statistics
`clearGlobalNets / globalNetConnect`	Logical power/ground connection rules
`setAddRingMode / addRing`	Core power ring
`setAddStripeMode / addStripe`	Power stripe mesh
`setSrouteMode / sroute`	Tie pads and cell rails to the mesh
`verifyConnectivity -connLoop`	PDN opens / shorts / closed-loop check
`addEndCap`	Well termination at row ends
`addIoFiller`	Fill gaps in the IO pad ring
`setPlaceMode / setOptMode`	Configure placer and optimiser
`place_opt_design`	Global place + legalize + PPO
`checkPlace / reportDensityMap / reportCongestion`	Post-placement physical analysis
`timeDesign -preCTS -idealClock`	Pre-CTS timing (optimistic by construction)

Command	Function
`create_ccopt_clock_tree_spec`	Derive clock tree spec from SDC
`set_ccopt_property`	Buffer list, target skew, target transition
`ccopt_design -outDir`	Build and route the clock tree
`report_ccopt_clock_trees`	Achieved skew, latency, buffer count
`setNanoRouteMode`	Antenna diode, timing-driven, SI-driven routing
`routeDesign -globalDetail`	Global + detailed routing
`setAnalysisMode -analysisType onChipVariation -cppr both`	OCV with CPPR
`setDelayCalMode -engine aae -SIAware true`	Crosstalk-aware delay calculation
`optDesign -postRoute -setup -hold`	Final timing repair on real parasitics
`addFiller -doDRC`	Standard-cell row fill
`verify_drc / verifyProcessAntenna`	Geometric and antenna checks
`setExtractRCMode / extractRC / rcOut -spef`	Parasitic extraction to SPEF
`saveNetlist / write_sdf`	Post-layout netlist and delays
`streamOut -mapFile -merge -units -mode ALL`	GDSII export with abstract replacement
`saveDesign`	Database checkpoint

B.3 Siemens Calibre (Verification)

Statement / Command	Function
`LAYOUT SYSTEM / PATH / PRIMARY`	Input GDS and top cell
`PRECISION / RESOLUTION`	Database units and minimum geometry
`DRC RESULTS DATABASE / SUMMARY REPORT`	Output database and report
`DRC MAXIMUM RESULTS ALL`	Do not truncate violations
`#define CHIP`	Enable full-chip rules (pads, seal ring)
`#define 4LM`	Select 4-metal rule set
`#define AA_gen ... M4_gen`	Enable per-layer dummy generation
`SOURCE PATH / PRIMARY / SYSTEM SPICE`	Golden netlist for LVS/PEX
`LVS POWER NAME / LVS GROUND NAME`	Declare all supply aliases
`VIRTUAL CONNECT NAME`	Bridge equivalent net names
`PEX EXTRACT RESISTOR / CAPACITOR`	What parasitics to extract
`PEX NETLIST "..." HSPICE 1`	Output an HSPICE-format RC netlist
`calibredrv -a layout filemerge`	Merge design + dummy GDS
`calibre -xrc -fmt -all`	Run extraction and all formatting phases

APPENDIX C

SCL 180 nm Library Cell Reference

Cell / Family	Category	Role in this design
pc3d01	IO — input pad	Buffers external signal into the core (CLK, RSTN, WR_EN, RD_EN, DIN)
pt3t03u	IO — 3-state output pad	Drives DOUT, FULL, EMPTY; OEN=0 permanently enabled
pvdi	IO — power pad	1.8 V core VDD
pv0i	IO — power pad	0 V core VSS
pvda	IO — power pad	3.3 V IO VDDO
pv0a	IO — power pad	0 V IO VSSO
pfrelr	IO — corner cell	Turns the power ring 90° at each die corner
pfeed30000 ...pfeed00010	IO — filler	Metal-only gap fill in the pad ring (300 µm down to 0.1 µm)
feedth9	Core — end cap	N/P-well termination at both ends of every row
feedth, feedth3, feedth9	Core — row filler	Restore M1 rails and well continuity after routing
bufbd1/2/4/7, buffd1/2/4/7	Core — clock buffers	CCOpt tree, drive strengths 1×–7×
inv0d1/2/4/7, invbd2/4/7	Core — clock inverters	CCOpt tree inverter variants
inv0d2	Core — inverter	Used as the external set_driving_cell model
and2d1 / and2d4	Core — logic	Example of weak vs. strong drive chosen by syn_map
antenna diode	Core — physical	Inserted by NanoRoute at vulnerable gate pins

C.1 Library and Technology Files

File	Purpose
tsl18fs120_scl_ss.lib	Standard-cell Liberty, Slow-Slow corner (setup)
tsl18fs120_scl_ff.lib	Standard-cell Liberty, Fast-Fast corner (hold)
tsl18cio150_min.lib / _max.lib	IO pad Liberty, min/max timing
tsl180l4.lef	Technology LEF (layers, vias, rules)
tsl18fs120_scl.lef	Standard-cell abstract LEF
tsl18cio150_4lm.lef	IO pad abstract LEF
tsl18fs120.gds	Standard-cell full GDS layouts (for -merge)
tsl18cio150_4lm.gds	IO pad full GDS layouts (for -merge)
gds2_fe_4l.map	Layer-name → GDS layer/datatype map
tsl180.tf	Virtuoso master technology file
SCL_TLUPLUS_4M1L_TYP.capTbl	TLU+ capacitance table for RC corners
DRC_TS18SL_SCL_CALIBRE	Foundry DRC rule deck
LVS_TS18SL_SCL_CALIBRE	Foundry LVS rule deck
xRC_TS18SL_SCL_CALIBRE	Foundry parasitic extraction deck

File	Purpose
DUMMYFILL_TS18SL_SCL_CALIBRE	Foundry dummy fill algorithm
SCL_SealRing_Fixed	Fixed-size seal ring GDS cell (to be stretched)

APPENDIX D

Index of Key Terms

References are to the Part numbers of this note.

A — Active-low reset (2, 4); Antenna diode insertion (13, 14); Antenna effect / ratio (13, 14); Aspect ratio (8)

B — Back-annotation (SDF) (6, 14); Bin packing (10, 14); Bonding diagram (19); Buffer insertion (6, 12)

C — CCOpt (12); Cadence Genus (6); Cadence Innovus (7–14); Cadence Virtuoso (15); Calibre (16–18); cap_table / TLU+ (7); Circular buffer (2); Clock domain crossing (2); Clock insertion delay (12); Clock skew (12); Clock tree synthesis (12); Clock uncertainty (5); CMP planarity (16); Congestion (11); Corner cells `pfrelr` (3, 7); CPPR (13); `create_clock` (5)

D — `DATA_WIDTH` / `DEPTH` (2); Delay corner (7); Design Rule Check (14, 17, 19); Design Rule Violations (5, 6); Dishing (16); Dual voltage domain (7, 18); Dummy fill (16)

E — Elaboration (6); Electromigration (7, 9); Empty flag (2, 4); End cap cells `feedth9` (10); ESD protection (3)

F — Fast-Fast corner (7); FIFO, 8-bit synchronous (2); Filler cells, IO `pfeed` (10); Filler cells, standard (14); Floorplanning (8); Foundry deliverables (19); Full flag (2, 4); Functional verification (4)

G — Gate-level simulation (6, 14); GDSII stream-in (15); GDSII stream-out (14); Generic synthesis (6); Ground bounce / SSO (7)

H — Hold timing (5, 7, 12, 13); HSPICE (18)

I — Incremental optimization `syn_opt` (6); IO pad library `tsl18cio150` (3); IO pad placement `chip_top.io` (7); IO pad ring (3); IR drop (7, 9)

L — Latch-up (10); Layer map file (14, 15); LEF files (6, 7); Liberty files (6, 7); Logic synthesis (6); LVS (17, 19)

M — Mask tape-out package (19); `mem[]` array (2); Metal stack 4LM (9, 13); MMMC analysis (7)

N — N+1 pointer method (2); NanoRoute (13); Netlist export (6)

O — OCV (13); Operating conditions, PVT (7); Overflow, routing (11)

P — Pad-limited design (3); Pad ring wrapper `chip_top.v` (3); Parasitic extraction PEX (18); Phase / wrap bit (2); Physical cell placement (10); Physical Layout Estimation (6); Post-layout simulation (18); Power delivery network (9); Power pad insertion (7); Power planning (9); Power stripes (9); Propagated clock (12); PVT corners (7)

Q — QoR (6, 11)

R — RC corner (7); `rd_ptr` / `wr_ptr` (2, 4); Registered read output (2); Reset sequence (4); RTL design (2); RTL testbench (4)

S — SCL 180 nm (1, 6); SDC (5); SDF (6, 14); Seal ring (15, 17); `set_input_delay` / `set_output_delay` (5); `set_max_fanout` (5); `set_max_transition` (5); Setup timing (5, 7, 13); SI-driven routing (13); Silicon ID layer (15); Sign-off, Innovus (14); Slow-Slow corner (6, 7); SPEF (13, 14); SRoute (9); SSO noise (7); Standard cell placement (11); Synchronous operation (2); `syn_generic` / `syn_map` / `syn_opt` (6)

T — Tape-out, Virtuoso (15); `tb_fifo_8bit.v` (4); `tb_simulation.sp` (18); Technology mapping (6); Timing constraints (5); TLU+ cap tables (7); TNS (6); `tsl18cio150` (3); `tsl18fs120_scl` (6)

U — Utilization target 70 % (8)

V — VDD_CORE / VSS_CORE (7, 9); VDDO / VSSO (7, 18); Virtual connect (17, 18); Voltage domain, dual (7, 18)

W — Well tap / end cap cells (10); Wire bonding (7, 19); WNS (6); `wr_en` / `rd_en` (2, 4)